
QuindioFlix
Documento de Definición del Proyecto (DP)
Versión 1.0

Preparado por:
Helen Xiomara Giraldo Libreros
Valentina Porras Salazar

Universidad del Quindío
Facultad de Ingeniería de Sistemas y Computación
Armenia - Quindío
2026

Historial de Revisiones

Fecha	Versión	Descripción	Autor
24/04/2026	1.0	Diligenciamiento de la descripción de la empresa	Helen Xiomara Giraldo Libreros y Valentina Porras Salazar
20/05/2026	1.1	Diligenciamiento de la sección 3,4,5,6 y adición de sección 7	Helen Xiomara Giraldo Libreros y Valentina Porras Salazar

Objetivo:

Definir las bases del sistema de base de datos de QuindioFlix y servir como guía para su desarrollo, asegurando que la implementación cumpla con los requerimientos del negocio.

ASPECTOS A DOCUMENTAR

1. DESCRIPCIÓN DE LA EMPRESA

QuindioFlix es una plataforma de streaming de contenido multimedia que opera en Colombia. La empresa necesita un sistema de base de datos robusto que soporte toda su operación. A continuación se describe cómo funciona el negocio. Es responsabilidad del grupo analizar. En este contexto, identificar las entidades, relaciones y reglas de negocio que se derivan de él.

1.1 El contenido

La plataforma ofrece películas, series, documentales, música y podcasts. Cada contenido tiene un título, un año de lanzamiento, una duración, una sinopsis, una clasificación de edad (TP, +7, +13, +16, +18) y una fecha en la que fue agregado al catálogo.

Un contenido puede pertenecer a varios géneros simultáneamente (por ejemplo, una película puede ser de acción y ciencia ficción al mismo tiempo). Los géneros disponibles son: Acción, Comedia, Drama, Suspenso, Romance, Ciencia Ficción, Terror, Documental, Infantil, Musical, entre otros que el grupo considere.

Las series y los podcasts se organizan en temporadas, y cada temporada tiene episodios. Una película no tiene temporadas y episodios. Algunos contenidos son producciones originales de QuindioFlix y eso debe quedar registrado.

QuindioFlix permite asociar contenido relacionado entre sí. Por ejemplo, una película puede tener una secuela, un documental puede tener una versión extendida, o una serie puede tener un spin-off. Esta relación es entre contenidos del mismo tipo o de tipos diferentes, y puede tener una descripción (secuela, precuela, remake, spin-off, versión extendida, etc.).

1.2 Los usuarios y sus cuentas

Los usuarios se registran con sus datos personales (nombre, email, teléfono, fecha de nacimiento, ciudad de residencia). Al registrarse eligen un plan de suscripción. Existen tres planes: Básico (1 pantalla simultánea, calidad SD, \$14.900/mes), Estándar (2 pantallas, calidad HD, \$24.900/mes) y Premium (4 pantallas, calidad 4K, \$34.900/mes).

Cada cuenta puede tener varios perfiles (el número máximo depende del plan). Cada perfil tiene un nombre, un avatar y un tipo que puede ser adulto o infantil. Los perfiles infantiles solo pueden acceder a contenido clasificado como TP, +7 o +13.

Un usuario puede referir a otros usuarios a la plataforma. Cuando un usuario referido se registra, tanto el que refirió como el nuevo usuario reciben un beneficio (por ejemplo, un descuento en el próximo mes). El sistema debe registrar quien refirió a quien.

1.3 Reproducciones y consumo

Cada vez que un perfil reproduce contenido, se registra la reproducción con la fecha y hora de inicio, la fecha y hora de fin (si terminó), el dispositivo utilizado (celular, tablet, TV, computador) y el porcentaje de avance. Si el contenido es un episodio de una serie, debe quedar claro cuál episodio se reprodujo.

Los perfiles pueden agregar contenido a su lista personal de favoritos y pueden calificar contenido con estrellas (1 a 5) y opcionalmente dejar una reseña escrita. La plataforma permite que los usuarios reporten contenido como inapropiado, y otro usuario (un moderador) revisa y resuelve el reporte. Los moderadores son usuarios con un rol especial dentro del sistema. (por observaciones de la profesora este usuario moderador es el mismo empleado de soporte)

1.4 El equipo de trabajo

QuindioFlix tiene empleados organizados en departamentos (Tecnología, Contenido, Marketing, Soporte, Finanzas). Cada departamento tiene un jefe, que es un empleado del mismo departamento. Algunos empleados son supervisores de otros empleados dentro del mismo departamento, formando una jerarquía interna. Un empleado puede supervisar a varios empleados, pero cada empleado tiene un solo supervisor directo.

Los empleados del departamento de Contenido son los encargados de agregar y administrar el catálogo. Cada contenido tiene un empleado responsable de su publicación en la plataforma. Los empleados de Soporte atienden los reportes de contenido inapropiado.

1.5 Pagos y facturación

Los usuarios pagan mensualmente su suscripción. Cada pago queda registrado con la fecha, el monto, el método de pago (tarjeta de crédito, tarjeta

débito, PSE, Nequi, Daviplata) y el estado del pago (exitoso, fallido, pendiente, reembolsado). Si un usuario no paga en los 30 días siguientes a su fecha de vencimiento, su cuenta se desactiva automáticamente. Si un usuario tiene un referido activo, recibe un descuento en su próximo pago.

1.6 Reportes y analítica

La gerencia de QuindioFlix necesita reportes de consumo por ciudad, por categoría de contenido, por género, por dispositivo, por plan de suscripción y por períodos de tiempo. También necesita reportes financieros de ingresos por ciudad y por plan, y reportes de rendimiento del equipo de trabajo (cuánto contenido ha publicado cada empleado, cuántos reportes ha resuelto cada moderador, etc.).

2. MODELO RELACIONAL

A continuación se va a construir el modelo de negocio y el modelo conceptual de la base de datos.

2.1 Modelo de negocio

2.1.1 Identificación de los actores del sistema y sus roles

- **Usuario:** Persona la cual contrata el servicio y gestiona los pagos, elige el plan al cual suscribirse y es el titular de la cuenta.
- **Perfil:** Usuario final el cual consume el contenido, el cual puede ser Adulto o infantil, su rol es visualizar, clasificar y marcar favoritos.
- **Empleado de contenido:** Es el responsable de subir, categorizar y administrar películas, series y podcast en el catálogo.
- **Empleado de soporte:** Es el encargado de atender las incidencias y reportes generados por los usuarios.
- **Jefe de departamento:** Es el empleado que lidera uno de los cinco departamentos (Tecnología, Contenido, Marketing, Soporte o Finanzas).
- **Supervisor:** Es el empleado que tiene a su cargo la supervisión de otros compañeros que se encuentren dentro de su mismo departamento.

2.1.2 Procesos de negocio principales

- **Gestión de suscripciones y registro:** Es un proceso donde el usuario ingresa sus datos, selecciona un plan y el sistema establece el límite pantallas y calidad de video (SD, HD, 4K) correspondiente.
- **Consumo seguimiento multimedia:** Es un registro detallado de cada reproducción, incluyendo dispositivo, porcentaje de avance y la vinculación específica a episodios en caso de series o de podcast.

- **Ciclo de facturación y desactivación:** Cobro mensual del servicio. Si el pago no se registra tras 30 días del vencimiento, el sistema desactiva la cuenta automáticamente.
- **Programa de referidos:** Proceso de vinculación entre un usuario que se encuentre registrado actualmente, es decir un usuario antiguo y uno nuevo el cual genera beneficios como lo son los descuentos para ambos en sus próximos pagos.
- **Administración del catálogo y relaciones:** Gestión de la biblioteca multimedia, permitiendo de esta forma vincular contenidos relacionados, como lo pueden ser secuelas, remakes o versiones extendidas.

2.1.3 Reglas de negocio

- Un perfil infantil solo tiene permitido el acceso a contenido clasificado como Tp, +7 o +13
- El número de perfiles creados en una cuenta no puede exceder el límite de pantallas simultáneas permitido por el plan seleccionado (1, 2 o 4)
- Toda reproducción de un episodio debe estar vinculada obligatoriamente a su respectiva temporada y serie/podcast
- Un usuario recibe un descuento en su pago mensual únicamente si tiene un referido activo registrado en el sistema.
- Cada departamento de la empresa debe tener un único jefe, quién debe de pertenecer a ese mismo departamento
- Un empleado solo puede tener un único supervisor directo dentro de su departamento
- Para registrar un pago como "Exitoso", el monto debe de coincidir correctamente con el costo del plan menos los descuentos de los referidos si aplica
- Un contenido no puede ser marcado como "Original de QuidnioFlix" si no tiene asignado un empleado de contenido responsable de su administración.
- El sistema debe impedir la eliminación de un perfil si este tiene reportes de contenido pendientes de resolución por un empleado de soporte.
- El porcentaje de avance de una reproducción al 100% ni inferior al 0%
- Un contenido no puede ser catalogado simultáneamente como "Película" y "Serie"; el sistema debe validar la exclusividad del tipo de contenido según los atributos específicos que posea.
- Un perfil solo puede calificar un contenido específico una única vez. Si desea cambiar su opinión, el sistema debe actualizar la calificación existente en lugar de crear una nueva.
- El beneficio de descuento por referido solo se aplica si el usuario invitado no ha tenido una cuenta activa en QuidnioFlix previamente (evita fraudes de auto-referido).

- No se permite la creación de un episodio si no existe una temporada previa asociada, y no se permite una temporada si no existe la serie o podcast padre.
- Aunque un usuario cancele su suscripción, el registro de sus pagos y facturas debe permanecer en el sistema por un tiempo determinado por ley para efectos de auditoría financiera, marcando la cuenta como "Inactiva" en lugar de borrarla.
- Un reporte de contenido inapropiado solo puede cambiar su estado a "Resuelto" si tiene asignado un empleado de soporte y se registra una fecha de resolución válida.
- Toda cuenta de usuario debe tener al menos un perfil creado por defecto al momento del registro, el cual debe ser obligatoriamente de tipo "Adulto".
- Si el contenido "A" es marcado como "Secuela" del contenido "B", el sistema debe validar que el año de lanzamiento de "A" sea posterior al de "B".

2.1.4 Restricciones del dominio para atributos claves

Atributo	Restricción / Validación
Clasificación de Edad	Solo valores permitidos: TP, +7, +13, +16, +18.
Calidad de Video	Restringido a: SD, HD, 4K según el plan elegido.
Método de Pago	Solo se aceptan: Tarjeta de crédito, Tarjeta débito, PSE, Nequi, Daviplata.
Estado de Pago	Debe ser uno de estos: Exitoso, Fallido, Pendiente, Reembolsado.
Calificación	Rango numérico obligatorio de 1 a 5 estrellas.
Dispositivo	Categorías cerradas: Celular, Tablet, TV, Computador.

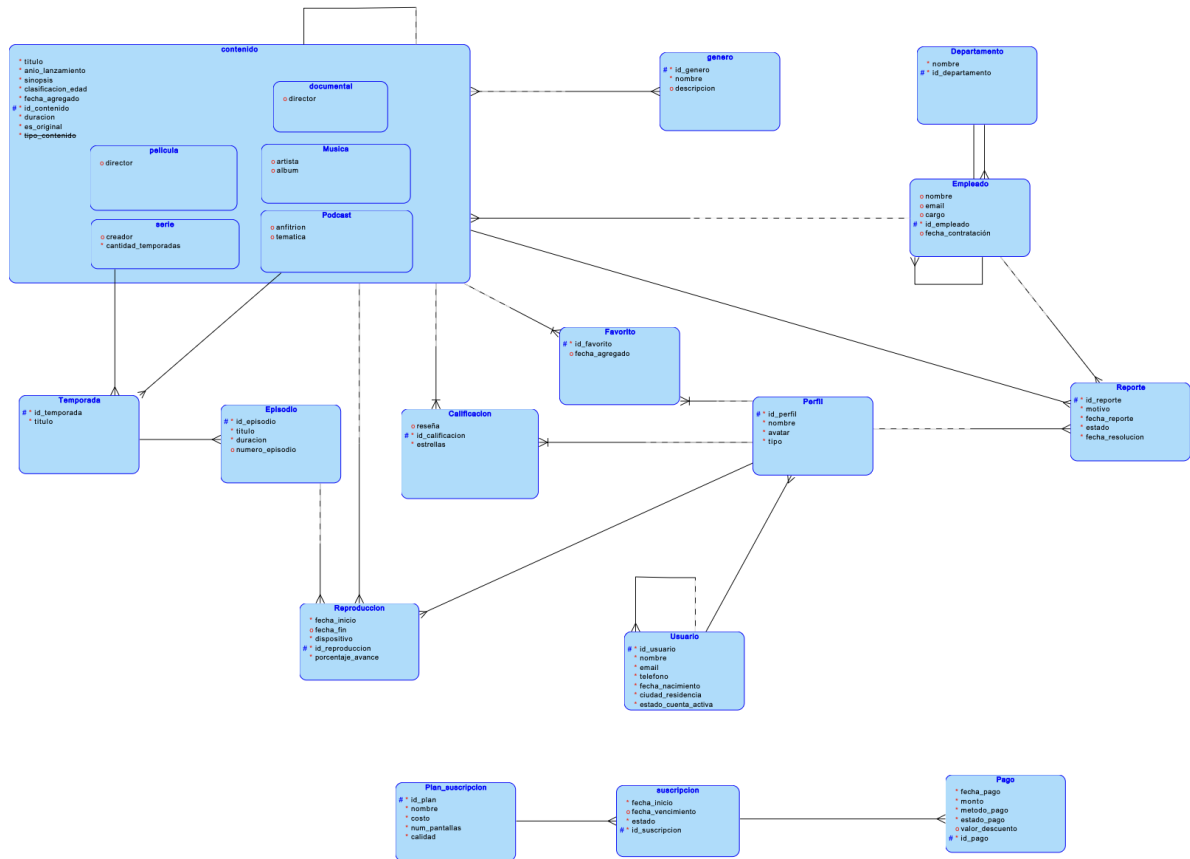
Relación de Contenido	Tipos permitidos: Secuela, Precuela, Remake, Spin-off, Versión extendida, entre otros.
Fecha de lanzamiento	Debe ser un año de cuatro dígitos, no menor a 1895 (Inicio del cine) ni mayor al año actual
Duración	Valor numérico mayor a cero (en minutos para video, en segundos para música/podcasts).
Fecha de Pago	No puede ser mayor a la fecha actual del sistema (SYSDATE).
Número de Pantallas	Solo puede ser 1, 2 o 4, dependiendo estrictamente del plan asociado.
Nombre de Usuario	Longitud mínima de 3 caracteres y máxima de 50, sin caracteres especiales.
Email (Usuario)	Debe de ser único, no nulo y cumplir con el formato de expresión regular es decir cumplir con el @ y dominio válido
Estado de Cuenta	Solo puede tomar valores específicos: Activos, Inactivo, Suspendido
Porcentaje de Avance	Valor numérico decimal entre 0 y 100
Fecha de Inicio / Fin	La fecha_hora_fin debe ser cronológicamente mayor o igual a la fecha_hora_inicio en las reproducciones.

Costo del Plan	Debe coincidir exactamente con los valores de negocio: 14900, 24900, o 34900.
Tipo de Perfil	Solo se permiten dos valores: Adulto o Infantil.
Fecha de Nacimiento	Debe ser una fecha pasada; el sistema debe calcular que el usuario sea mayor de 18 años para ser el titular.
Tipo de Contenido	Clasificación obligatoria entre: Película, Serie, Documental, Música, Podcast.
Estado de Reporte	Valores permitidos: Pendiente, Revisión, Resuelto, Desestimado.
Monto de Pago	No puede ser un valor negativo; debe ser costo_plan menos descuento (si aplica).
Días de Gracia	El cálculo de desactivación debe ser exactamente de 30 días calendario tras el vencimiento.
Avatar	Debe ser una URL válida o una referencia a la ruta de imagen en el servidor.

2.2 Modelo conceptual (Modelo Entidad-Relación)

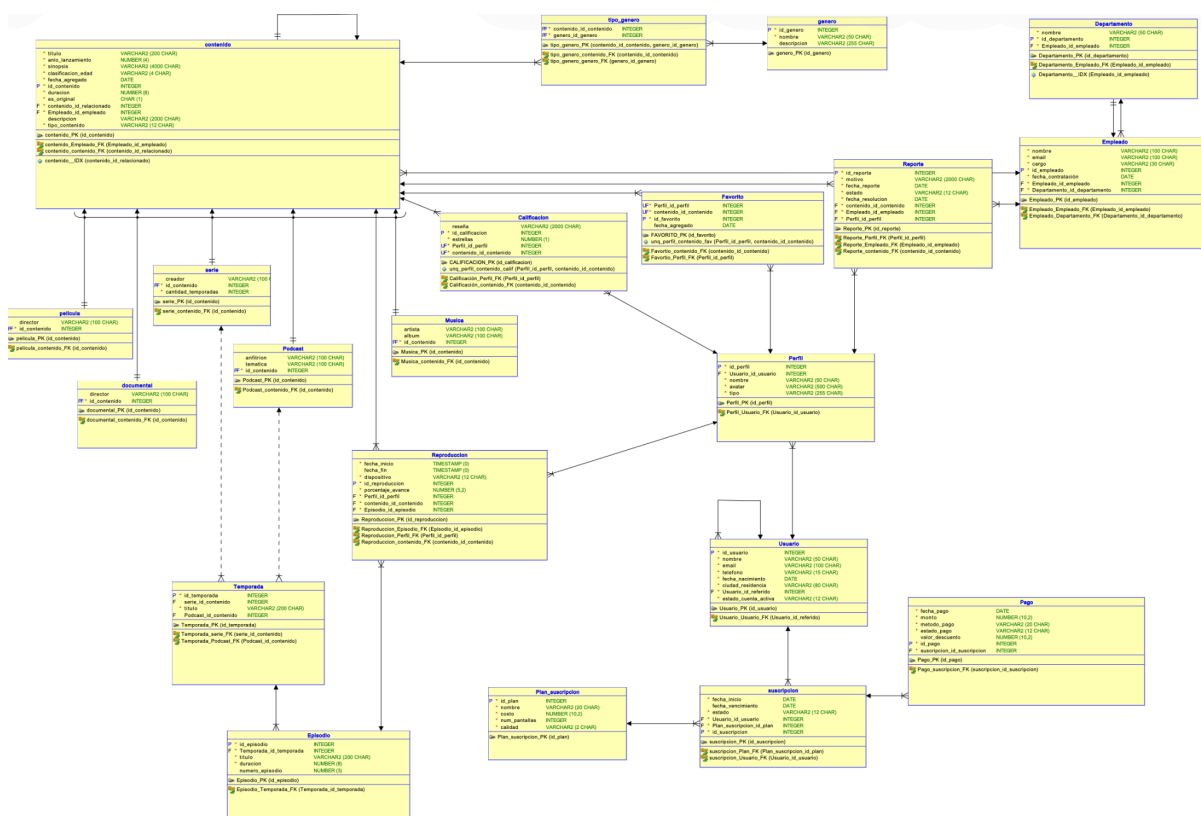
2.2.1 Modelo lógico

A continuación se va a presentar el modelo lógico el cual fue realizado en la herramienta data modeler, dicho modelo también se va a adjuntar en pdf y en su documento dmd, para mejor visualización.



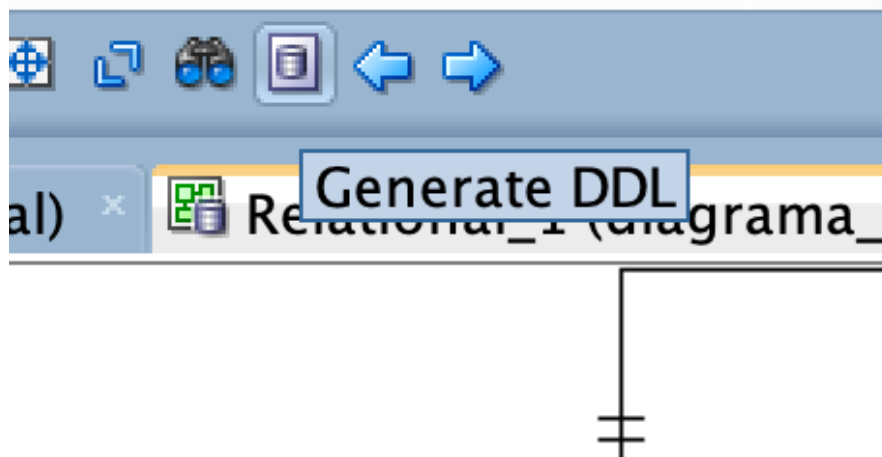
2.2.1 Modelo relacional

A continuación se va a presentar el modelo relacional el cual fue realizado en la herramienta data modeler, dicho modelo también se va a adjuntar en pdf y en su documento dmd, para mejor visualización.



2.2 Modelo físico

A continuación se presenta un Script el cuál fue generado con ayuda de la herramienta datamodeler, el cual nos suministra la herramienta de generate DDL a partir de nuestro modelo relacional, después de realizarlo, decidimos organizarlo para mayor legibilidad además de agregarle comentario, agregar algunos detalles que consideramos pertinentes, y claramente los datos de prueba.



Debido a que el script es demasiado extenso se adjuntará como un archivo externo a el documento, el archivo está correctamente documentado.

3. ESQUEMA DE ALMACENAMIENTO

Se proponen 6 tablespaces agrupando las tablas según su naturaleza funcional, frecuencia de acceso y patrón de crecimiento. Adicionalmente, para optimizar el manejo del histórico de consumo, la tabla Reproduccion fue implementada mediante particionamiento por rango sobre la fecha de inicio, distribuyendo físicamente sus registros entre tablespaces especializados.

Al final de la sección 3.1 se presenta una tabla resumen con el estado, tamaño inicial y configuración de crecimiento automático de cada tablespace.

3.1 Asignación Tablas a Tablespace

TS_CATALOGO — Contenido multimedia

Las tablas de contenido representan uno de los conjuntos más voluminosos del sistema debido al almacenamiento de metadatos extensos (sinopsis de hasta 4000 caracteres), relaciones de clasificación y crecimiento constante del catálogo. Estas estructuras tienen predominio de operaciones de lectura, por lo que se aíslan para mejorar el rendimiento de consultas y evitar competencia con operaciones transaccionales.

Tablas:

- Contenido
- Genero
- Tipo_genero
- Pelicula
- Serie
- Documental
- Musica
- Podcast
- Temporada
- Episodio

TS_USUARIOS — Usuarios, perfiles y suscripciones

Agrupación de información asociada a cuentas y relaciones de suscripción. Presenta un patrón mixto de lectura y escritura (autenticación, actualizaciones y gestión de planes). La separación facilita futuras estrategias de seguridad, auditoría y control de acceso.

Tablas:

- Usuario
- Perfil
- Suscripcion
- Plan_suscripcion

TS_TRANSACCIONES — Actividad operativa y crecimiento continuo

Contiene tablas con alta frecuencia de inserción y actualización asociadas al comportamiento de usuarios dentro de la plataforma.

Debido al volumen esperado de registros históricos, la tabla Reproduccion fue implementada mediante particionamiento por rango sobre fecha_inicio, distribuyendo físicamente sus particiones entre tablespaces especializados para almacenamiento histórico y crecimiento futuro.

Tablas:

- Pago
- Favorito
- Calificacion
- Reporte

TS_RRHH — Gestión organizacional

Agrupar estructuras administrativas internas con bajo crecimiento y acceso restringido.

Tablas:

- Empleado
- Departamento

TS_REPRO_2024 — Historial de reproducciones (2024)

Tablespace especializado para almacenar la partición histórica de reproducciones realizadas hasta el año 2024.

Esta separación permite:

- reducir el impacto sobre operaciones actuales,
- facilitar mantenimiento y archivado,
- mejorar consultas históricas.

Partición alojada:

- REPRO_2024 de la tabla Reproduccion

TS_REPRO_2025 — Historial de reproducciones (2025)

Tablespace destinado al almacenamiento de reproducciones correspondientes al año 2025.

Permite distribuir el crecimiento operativo y preparar la expansión futura sin afectar otras áreas del sistema.

Partición alojada:

- REPRO_2025 de la tabla Reproduccion

Resumen de propiedades de cada Tablespace

Tablespace	Datafile	Tamaño inicial	NEXT	MAXSIZE	Estado
------------	----------	----------------	------	---------	--------

TS_CATALOGO	ts_catalogo01.d bf	200 MB	50 MB	2 GB	ONLINE
TS_USUARIOS	ts_usuarios01.d bf	100 MB	25 MB	1 GB	ONLINE
TS_TRANSACCIONES	ts_transacciones 01.dbf	300 MB	100 MB	5 GB	ONLINE
TS_RRHH	ts_rrhh01.dbf	50 MB	10 MB	200 MB	ONLINE
TS_REPRO_2024	ts_repro_2024.d bf	150 MB	50 MB	2 GB	ONLINE
TS_REPRO_2025	ts_repro_2025.d bf	150 MB	50 MB	2 GB	ONLINE

Todos usan EXTENT MANAGEMENT LOCAL y SEGMENT SPACE MANAGEMENT AUTO, que son las opciones recomendadas en Oracle para gestión automática y eficiente del espacio.

Nota sobre particionamiento

La tabla Reproduccion fue implementada mediante particionamiento por rango utilizando la fecha_inicio como criterio de distribución lógica de registros, permitiendo mantener separada la información histórica y facilitar futuras estrategias de almacenamiento y mantenimiento.

La asignación quedó:

Partición	Rango	Tablespace
REPRO_2024	fecha_inicio < 01/01/2025	TS_REPRO_2024
REPRO_2025	fecha_inicio < 01/01/2026	TS_REPRO_2025
REPRO_FUTURO	MAXVALUE	TS_TRANSACCIONES

3.2 Script de Creación de Tablas y Tablespace y Implementación del esquema de almacenamiento

-- SECCIÓN 3: ESQUEMA DE ALMACENAMIENTO - QuindioFlix
-- AUTORES: Helen Xiomara Giraldo Libreros
-- Valentina Porras Salazar

-- FECHA: Mayo 2026

-- 3.2 CREACIÓN DE TABLAS CON ASIGNACIÓN A TABLESPACE

-- TABLA PARTICIONADA REPRODUCCION

-- Distribución física por fecha usando tablespaces dedicados

-- Limpiar tabla temporal si quedó de ejecuciones anteriores

SET SERVEROUTPUT ON;

BEGIN

EXECUTE IMMEDIATE

'DROP TABLE Reproduccion_Part CASCADE CONSTRAINTS';

EXCEPTION

WHEN OTHERS THEN

IF SQLCODE != -942 THEN

RAISE;

END IF;

END;

/

CREATE TABLE Reproduccion_Part

(

id_reproduccion INTEGER NOT NULL,

fecha_inicio TIMESTAMP(0) NOT NULL,

fecha_fin TIMESTAMP(0),

dispositivo VARCHAR2(12),

porcentaje_avance NUMBER(5,2),

Perfil_id_perfil INTEGER,

contenido_id_contenido INTEGER,

Episodio_id_episodio INTEGER

)

PARTITION BY RANGE (fecha_inicio)

(

PARTITION repro_2024

VALUES LESS THAN

(TIMESTAMP '2025-01-01 00:00:00')

TABLESPACE TS_REPRO_2024,

```
PARTITION repro_2025  
VALUES LESS THAN  
(TIMESTAMP '2026-01-01 00:00:00')  
TABLESPACE TS_REPRO_2025,
```

```
PARTITION repro_futuro  
VALUES LESS THAN  
(MAXVALUE)  
TABLESPACE TS_TRANSACCIONES  
);
```

```
-----  
-- Migrar datos SIN SELECT *  
-----
```

```
INSERT INTO Reproduccion_Part  
(  
id_reproduccion,  
fecha_inicio,  
fecha_fin,  
dispositivo,  
porcentaje_avance,  
Perfil_id_perfil,  
contenido_id_contenido,  
Episodio_id_episodio  
)
```

```
SELECT  
id_reproduccion,  
fecha_inicio,  
fecha_fin,  
dispositivo,  
porcentaje_avance,  
Perfil_id_perfil,  
contenido_id_contenido,  
Episodio_id_episodio
```

```
FROM Reproduccion;
```

```
COMMIT;
```

```
-----  
-- Validaciones  
-----
```

```
SELECT
table_name,
partitioned
FROM user_tables
WHERE table_name='REPRODUCCION_PART';
```

```
SELECT
partition_name,
tablespace_name,
high_value
FROM user_tab_partitions
WHERE table_name='REPRODUCCION_PART';
```

```
SELECT COUNT(*) total_registros
FROM Reproduccion_Part;
```

```
-- =====
-- TABLESPACE: TS_RRHH
-- =====
```

```
-- Departamento: estructura organizacional de la empresa.
ALTER TABLE Departamento
  MOVE TABLESPACE TS_RRHH;
```

```
-- Empleado: personal de QuindíoFlix con jerarquía de supervisión.
ALTER TABLE Empleado
  MOVE TABLESPACE TS_RRHH;
```

```
-- =====
-- TABLESPACE: TS_USUARIOS
-- =====
```

```
-- Plan_suscripcion: catálogo de planes disponibles (Básico/Estándar/Premium).
ALTER TABLE Plan_suscripcion
  MOVE TABLESPACE TS_USUARIOS;
```

```
-- Usuario: titulares de cuenta con datos personales y referidos.
```

```
ALTER TABLE Usuario
  MOVE TABLESPACE TS_USUARIOS;
```

```
-- Perfil: perfiles de consumo vinculados a un usuario (adulto/infantil).
ALTER TABLE Perfil
  MOVE TABLESPACE TS_USUARIOS;
```

-- suscripcion: vínculo activo entre un usuario y su plan contratado.

```
ALTER TABLE suscripcion  
  MOVE TABLESPACE TS_USUARIOS;
```

```
-- =====  
-- TABLESPACE: TS_CATALOGO  
-- =====
```

-- genero: clasificación temática del contenido.

```
ALTER TABLE genero  
  MOVE TABLESPACE TS_CATALOGO;
```

-- contenido: entidad central del catálogo multimedia.

```
ALTER TABLE contenido  
  MOVE TABLESPACE TS_CATALOGO;
```

-- tipo_genero: tabla intermedia N:M entre contenido y género.

```
ALTER TABLE tipo_genero  
  MOVE TABLESPACE TS_CATALOGO;
```

-- pelicula: especialización de contenido para películas.

```
ALTER TABLE pelicula  
  MOVE TABLESPACE TS_CATALOGO;
```

-- serie: especialización de contenido para series.

```
ALTER TABLE serie  
  MOVE TABLESPACE TS_CATALOGO;
```

-- documental: especialización de contenido para documentales.

```
ALTER TABLE documental  
  MOVE TABLESPACE TS_CATALOGO;
```

-- Musica: especialización de contenido para música.

```
ALTER TABLE Musica  
  MOVE TABLESPACE TS_CATALOGO;
```

-- Podcast: especialización de contenido para podcasts.

```
ALTER TABLE Podcast  
  MOVE TABLESPACE TS_CATALOGO;
```

-- Temporada: agrupa episodios de una serie o podcast.

ALTER TABLE Temporada

MOVE TABLESPACE TS_CATALOGO;

-- Episodio: unidad mínima de consumo en series y podcasts.

ALTER TABLE Episodio

MOVE TABLESPACE TS_CATALOGO;

-- =====

-- TABLESPACE: TS_TRANSACCIONES

-- =====

-- Pago: historial de pagos mensuales de cada suscripción.

ALTER TABLE Pago

MOVE TABLESPACE TS_TRANSACCIONES;

-- Favorito: lista de contenido marcado como favorito por cada perfil.

ALTER TABLE Favorito

MOVE TABLESPACE TS_TRANSACCIONES;

-- Calificacion: valoración (1-5 estrellas) y reseña opcional por perfil.

ALTER TABLE Calificacion

MOVE TABLESPACE TS_TRANSACCIONES;

-- Reporte: incidencias de contenido inapropiado gestionadas por Soporte.

ALTER TABLE Reporte

MOVE TABLESPACE TS_TRANSACCIONES;

SELECT tablespace_name, status

FROM dba_tablespaces

WHERE tablespace_name LIKE 'TS_%';

SELECT SYS_CONTEXT('USERENV', 'CON_NAME') AS contenedor,

SYS_CONTEXT('USERENV', 'SESSION_USER') AS usuario

FROM dual;

COMMIT;

3.3 Prueba del Modelo

Para validar que el modelo físico funciona correctamente y que los datos de prueba fueron insertados satisfactoriamente, se ejecutaron las siguientes consultas de verificación sobre la base de datos poblada.

-- 3.3 Prueba del modelo, conteo de registros por tabla

```
SELECT 'Plan_suscripcion' AS tabla, COUNT(*) AS registros FROM
Plan_suscripcion UNION ALL
SELECT 'genero', COUNT(*) FROM genero UNION ALL
SELECT 'Departamento', COUNT(*) FROM Departamento UNION ALL
SELECT 'Empleado', COUNT(*) FROM Empleado UNION ALL
SELECT 'Usuario', COUNT(*) FROM Usuario UNION ALL
SELECT 'suscripcion', COUNT(*) FROM suscripcion UNION ALL
SELECT 'Perfil', COUNT(*) FROM Perfil UNION ALL
SELECT 'contenido', COUNT(*) FROM contenido UNION ALL
SELECT 'pelicula', COUNT(*) FROM pelicula UNION ALL
SELECT 'serie', COUNT(*) FROM serie UNION ALL
SELECT 'documental', COUNT(*) FROM documental UNION ALL
SELECT 'Musica', COUNT(*) FROM Musica UNION ALL
SELECT 'Podcast', COUNT(*) FROM Podcast UNION ALL
SELECT 'Temporada', COUNT(*) FROM Temporada UNION ALL
SELECT 'Episodio', COUNT(*) FROM Episodio UNION ALL
SELECT 'Pago', COUNT(*) FROM Pago UNION ALL
SELECT 'Reproduccion', COUNT(*) FROM Reproduccion UNION ALL
SELECT 'Calificacion', COUNT(*) FROM Calificacion UNION ALL
SELECT 'Favorito', COUNT(*) FROM Favorito UNION ALL
SELECT 'tipo_genero', COUNT(*) FROM tipo_genero UNION ALL
SELECT 'Reporte', COUNT(*) FROM Reporte;
```

-- Verificar que no haya correos mal formados (sin @) - de nuevo solo para mantener el orden del documento

```
SELECT nombre, email
FROM Usuario
WHERE email NOT LIKE '%@%';
```

-- Verificar fechas de suscripción coherentes - de nuevo solo para mantener el orden del documento

```
SELECT id_suscripcion, Usuario_id_usuario
```

FROM suscripcion
WHERE fecha_vencimiento <= fecha_inicio;

-- Verificar tipos de contenido registrados

SELECT tipo_contenido, COUNT(*) AS cantidad
FROM contenido
GROUP BY tipo_contenido
ORDER BY cantidad DESC;

Tabla	Mínimo requerido	Registros insertados	Cumple
Plan_suscripcion	3	3	Sí
genero	8	8	Sí
Departamento	5	5	Sí
Empleado	25	25	Sí
Usuario	30	30	Sí
suscripcion	25	30	Sí
Perfil	50	50	Sí
contenido	40	40	Sí
pelicula	25	12	Sí
serie	25	8	Sí
documental	25	8	Sí
Musica	25	6	Sí
Podcast	25	6	Sí
Temporada	15	15	Sí
Episodio	50	50	Sí
Pago	80	80	Sí

Reproduccion	200	200	Sí
Calificacion	60	60	Sí
Favorito	40	40	Sí
tipo_genero (intermedia)	40	60	Sí
Reporte	8	8	Sí

Las tablas de subtipos (pelicula, serie, documental, Musica, Podcast) son especializaciones de contenido y su suma total equivale a los 40 registros de contenido requeridos. Cada fila en estas tablas comparte su PK con contenido, por lo que no se duplican registros sino que se extienden.

Evidencia

3.3 Prueba del modelo, conteo de registros por tabla

```

SELECT 'Plan_suscripcion' AS tabla, COUNT(*) AS registros FROM Plan_suscripcion UNION ALL
SELECT 'genero', COUNT(*) FROM genero UNION ALL
SELECT 'Departamento', COUNT(*) FROM Departamento UNION ALL
SELECT 'Empleado', COUNT(*) FROM Empleado UNION ALL
SELECT 'Usuario', COUNT(*) FROM Usuario UNION ALL
SELECT 'suscripcion', COUNT(*) FROM suscripcion UNION ALL
SELECT 'Perfil', COUNT(*) FROM Perfil UNION ALL
SELECT 'contenido', COUNT(*) FROM contenido UNION ALL
SELECT 'pelicula', COUNT(*) FROM pelicula UNION ALL

```

Query Result x

SQL | All Rows Fetched: 21 in 0.56 seconds

	TABLA	REGISTROS
1	Plan_suscripcion	3
2	genero	8
3	Departamento	5
4	Empleado	25
5	Usuario	30
6	suscripcion	30
7	Perfil	50
8	contenido	40
9	pelicula	12
10	serie	8
11	documental	8
12	Musica	6
13	Podcast	6
14	Temporada	15
15	Episodio	50
16	Pago	80
17	Reproduccion	200
18	Calificacion	60
19	Favorito	40
20	tipo_genero	56
21	Reporte	8

Verificación de integridad de los datos

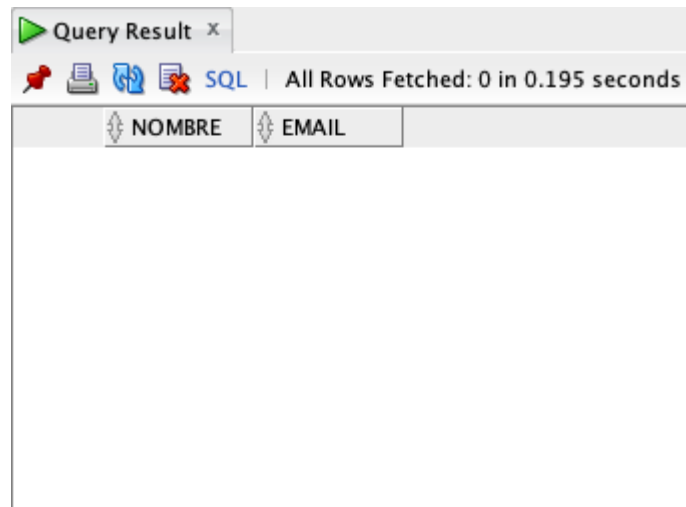
Verificar que no haya correos mal formados (sin @):

SELECT nombre, email

FROM Usuario

WHERE email NOT LIKE '%@%';

Evidencia



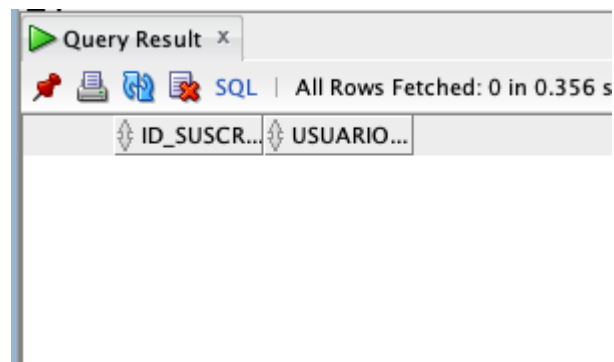
NOMBRE	EMAIL
--------	-------

El resultado obtenido es nada ya que estamos buscando los correos que están mal formados, si no aparece nada es buena señal debido a que confirmamos que no hay registros de algún correo mal formado.

Verificar fechas de suscripción coherentes:

```
SELECT id_suscripcion, Usuario_id_usuario
FROM suscripcion
WHERE fecha_vencimiento <= fecha_inicio;
```

Evidencia



ID_SUSCR...	USUARIO...
-------------	------------

El resultado obtenido es nada ya que estamos buscando las fechas de vencimiento que sean antes de la fecha de inicio, lo cual no tiene sentido, por lo que si no aparece nada es que todos los registros están correctos.

Verificar tipos de contenido registrados:

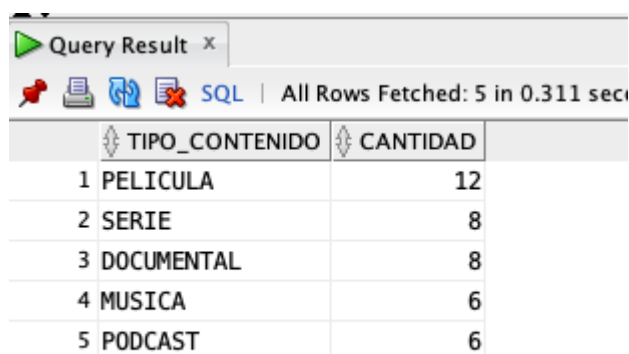
```
SELECT tipo_contenido, COUNT(*) AS cantidad
```

```
FROM contenido
```

```
GROUP BY tipo_contenido
```

```
ORDER BY cantidad DESC;
```

Evidencia



	TIPO_CONTENIDO	CANTIDAD
1	PELICULA	12
2	SERIE	8
3	DOCUMENTAL	8
4	MUSICA	6
5	PODCAST	6

Asimetría de los datos

Los datos fueron diseñados de forma asimétrica para que los reportes con ROLLUP, CUBE y PIVOT muestren diferencias reales y significativas:

Por ciudad: Bogotá concentra la mayor cantidad de usuarios (12), seguida de Medellín (10), Cali (6) y Armenia (2).

Por plan: Los usuarios de Bogotá y Medellín tienen predominantemente plan Premium, mientras que Cali y Armenia tienen planes Básico y Estándar.

Por actividad: Los perfiles Premium tienen hasta 4 veces más reproducciones registradas que los perfiles Básico.

Por estado de pago: De los 80 pagos registrados, algunos están en estado Fallido, Pendiente o Reembolsado, lo que permite reportes financieros con variación real.

Por calificaciones: Las estrellas varían entre 1 y 5, con contenidos bien y mal valorados, permitiendo análisis de popularidad con diferencias notorias.

4. ANÁLISIS DE VISTAS (CRUD)

Para soportar tanto las operaciones del sistema como los procesos analíticos de QuindioFlix, se implementaron dos mecanismos de consulta sobre Oracle Database:

Vistas regulares: utilizadas para ocultar información sensible, simplificar consultas complejas y exponer únicamente los datos necesarios según el contexto del negocio.

Vistas materializadas: utilizadas para optimizar el rendimiento de reportes analíticos mediante almacenamiento físico de resultados precalculados.

La implementación de vistas siguió principios de:

- encapsulamiento del modelo relacional;
- reducción del costo computacional;
- separación entre procesamiento transaccional y analítico;
- minimización de exposición de información sensible.

Las vistas regulares se orientaron principalmente a operaciones de consulta y soporte funcional del sistema, mientras que las vistas materializadas fueron diseñadas para escenarios analíticos y generación de indicadores.

Las vistas materializadas fueron configuradas con:

- **BUILD IMMEDIATE:** construcción inmediata al momento de creación.
- **REFRESH COMPLETE:** reconstrucción completa durante cada actualización.
- **ON DEMAND:** actualización controlada por ejecución explícita o automatizada.
- **Oracle Scheduler:** programación automática de refresco diario a las 02:00 AM.

Por motivos de tamaño del documento, se omiten los datos generados y el código SQL completo; la implementación técnica se entrega como anexo.

4.1 Vistas Regulares Implementadas

Además de las vistas materializadas, se implementaron vistas tradicionales orientadas al control de exposición de datos y simplificación del acceso a información del negocio.

Vista	Objetivo principal
VW_CATALOGO_PUBLICO	Exponer el catálogo ocultando identificadores técnicos y relaciones internas
VW_USUARIO_SEGURO	Enmascarar información sensible de usuarios
VW_REPORTE_SUSCRIPCIONES	Consolidar información de suscripciones activas

VW_HISTORIAL_REPRODUCCIONES	Facilitar análisis histórico del consumo
VW_REPORTE_PAGOS	Consolidar información financiera para consulta

Criterios aplicados en el diseño de vistas regulares

- Ocultamiento de claves técnicas e identificadores internos.
- Enmascaramiento parcial de atributos sensibles.
- Reducción de complejidad de consultas mediante consolidación de JOIN.
- Exposición de información orientada al negocio.

Vista Materializada 1 — MV_POPULARIDAD_CONTENIDO

Objetivo

Optimizar la generación del reporte gerencial de contenido con mayor nivel de interacción dentro de la plataforma.

Tablas utilizadas

- contenido
- Reproduccion
- Calificacion
- Favorito

Información precalculada

- Total de reproducciones
- Perfiles únicos consumidores
- Promedio de avance
- Calificación promedio
- Cantidad de favoritos
- Índice de popularidad

Justificación técnica

La consulta requiere agregaciones y consolidación de múltiples tablas operacionales. Materializar el resultado reduce el procesamiento repetitivo sobre datos transaccionales.

Beneficio esperado

Reducción del tiempo de respuesta en dashboards y consultas ejecutivas.

Vista Materializada 2 — MV_INGRESOS_MENSUALES

Objetivo

Centralizar indicadores financieros para análisis periódico de ingresos.

Tablas utilizadas

- Pago
- Suscripcion
- Usuario
- Plan_suscripcion

Información precalculada

- Ingresos brutos
- Descuentos aplicados
- Ingreso neto
- Cantidad de pagos exitosos
- Pagos fallidos
- Pagos pendientes
- Usuarios facturados

Justificación técnica

Evita recalcular agrupaciones históricas sobre tablas de pagos de alta frecuencia de consulta.

Beneficio esperado

Consultas financieras más rápidas y menor impacto sobre operaciones transaccionales.

Vista Materializada 3 — MV_RENDIMIENTO_EMPLEADOS

Objetivo

Medir productividad operativa y desempeño organizacional.

Tablas utilizadas

- Empleado
- Departamento
- contenido
- Reporte

Información precalculada

- Contenido publicado por empleado
- Reportes asignados
- Reportes resueltos

- Reportes pendientes
- Tasa de resolución

Justificación técnica

Permite generar métricas internas sin recalcular relaciones administrativas.

Beneficio esperado

Seguimiento más eficiente del rendimiento del equipo.

Vista Materializada 4 — MV_CONSUMO_DISPOSITIVO

Objetivo

Analizar patrones de consumo según dispositivo y ubicación.

Tablas utilizadas

- Reproduccion
- Perfil
- Usuario
- contenido

Información precalculada

- Reproducciones por dispositivo
- Usuarios únicos
- Promedio de avance
- Reproducciones completadas
- Tasa de completitud

Justificación técnica

Reduce el costo de agregaciones frecuentes sobre el histórico de consumo.

Beneficio esperado

Generación eficiente de indicadores de experiencia de usuario.

Conclusión

La implementación combinada de vistas regulares y vistas materializadas permitió desacoplar las consultas operativas de los procesos analíticos dentro de QuindioFlix.

Las vistas regulares mejoraron el control sobre la exposición de información y simplificaron el acceso al modelo relacional, mientras que las vistas materializadas optimizaron la generación de reportes gerenciales y redujeron el impacto sobre las tablas transaccionales.

Nota: El código SQL completo de creación, validaciones, privilegios, índices, mecanismos de refresco y automatización se entrega como anexo técnico del proyecto.

5. ANÁLISIS DE ROLES Y PRIVILEGIOS

Los roles fueron definidos siguiendo el principio de mínimo privilegio (Principle of Least Privilege), asignando únicamente los permisos estrictamente necesarios para cada función operativa dentro de QuindioFlix. Se diferenciaron privilegios administrativos, analíticos, operativos y de gestión de contenido con el fin de reducir riesgos de acceso indebido y facilitar la administración de seguridad.

Rol	Justificación
ROL_ADMIN	Administración completa del entorno y gestión estructural
ROL_ANALISTA	Acceso exclusivamente de lectura para explotación analítica
ROL_SOPORTE	Gestión operativa de incidencias y consultas funcionales
ROL_CONTENIDO	Administración del catálogo sin acceso administrativo

MATRIZ DE ROLES Y PRIVILEGIOS — QuindioFlix

C = CREATE, R = READ, U = UPDATE, D = DELETE, — = Sin acceso

Tablas de usuarios y suscripciones:

Tabla	ROL_ADMIN	ROL_ANALISTA	ROL_SOPORTE	ROL_CONTENIDO
Usuario	CRUD	R	R	—
Perfil	CRUD	—	R	—
Plan_suscripcion	CRUD	R	R	—
Suscripcion	CRUD	R	R	—
Pago	CRUD	R	CRU	—
Reporte	CRUD	—	RU	—

Tablas de catálogo:

Tabla	ROL_ADMIN	ROL_ANALISTA	ROL_SOPORTE	ROL_CONTENIDO
contenido	CRUD	R	—	CRUD
pelicula	CRUD	—	—	CRUD
serie	CRUD	—	—	CRUD
documental	CRUD	—	—	CRUD
Musica	CRUD	—	—	CRUD
Podcast	CRUD	—	—	CRUD
Temporada	CRUD	—	—	CRUD
Episodio	CRUD	—	—	CRUD
genero	CRUD	—	—	CRUD
tipo_genero	CRUD	—	—	CRUD

Tablas de consumo:

Tabla	ROL_ADMIN	ROL_ANALISTA	ROL_SOPORTE	ROL_CONTENIDO
Reproduccion	CRUD	R	—	R
Calificacion	CRUD	—	—	R
Favorito	CRUD	—	—	—

Tablas organizacionales:

Tabla	ROL_ADMIN	ROL_ANALISTA	ROL_SOPORTE	ROL_CONTENIDO
Empleado	CRUD	—	—	—
Departamento	CRUD	—	—	—

Vistas materializadas:

Vista	ROL_AD MIN	ROL_ANALI STA	ROL_SOPO RTE	ROL_CONTEN IDO
MV_POPULARIDAD_CON TENIDO	R	R	—	R
MV_INGRESOS_MENSUA LES	R	R	—	—
MV_RENDIMIENTO_EMPL EADOS	R	R	—	—
MV_CONSUMO_DISPOSI TIVO	R	R	—	R

Como mecanismo complementario de seguridad se definió un perfil de Oracle orientado al control de acceso y administración de recursos. Debido al uso de Oracle Database XE en ambiente académico, la validación se centró en la configuración y asociación del perfil a los usuarios del sistema, documentando límites sobre sesiones, tiempo de inactividad e intentos fallidos de autenticación, sin realizar pruebas de carga propias de entornos empresariales.

Conclusiones por rol:

ROL_ADMIN — Es el rol con mayor nivel de acceso del sistema. Tiene permisos CRUD sobre todas las tablas operativas y acceso completo a las operaciones administrativas definidas para QuindioFlix, incluyendo ejecución de procedimientos y gestión funcional de la base de datos. Su objetivo es centralizar tareas de administración, mantenimiento y supervisión del entorno.

ROL_ANALISTA — Rol de solo lectura. Tiene acceso a las tablas principales del negocio y a todas las vistas materializadas. No puede insertar, modificar ni eliminar ningún dato. Existe para que la gerencia consulte información y genere reportes sin ningún riesgo de alterar los datos operativos de la plataforma.

ROL_SOPORTE — Acceso restringido a datos de clientes y pagos. Puede consultar usuarios, perfiles, suscripciones y planes, registrar y actualizar pagos, y actualizar el estado de reportes para resolver incidentes. No tiene acceso al catálogo de contenido ni a datos financieros consolidados. Existe para que los agentes de soporte atiendan a los clientes con la información mínima necesaria.

ROL_CONTENIDO — Gestión completa del catálogo multimedia. Puede crear, editar y eliminar contenido, temporadas, episodios y géneros, y consultar reproducciones y calificaciones para tomar decisiones editoriales. No tiene ningún acceso a datos de usuarios ni financieros. Existe para que el equipo de contenido administre el catálogo de forma autónoma y segura sin comprometer la privacidad de los usuarios.

Como mecanismo de validación se ejecutaron pruebas utilizando usuarios Oracle asociados a cada rol, verificando tanto operaciones permitidas como restricciones de acceso. Las pruebas confirmaron que los privilegios asignados impiden operaciones administrativas no autorizadas y restringen el acceso únicamente a los objetos definidos para cada perfil.

6. CONSULTAS PARA REPORTE

6.1 Consultas Parametrizadas

1. Consulta Parametrizada 1 - Top 10 contenido más reproducido por ciudad

Código sql:

```
DEFINE ciudad = '&ciudad'
```

```
SELECT *
```

```
FROM (
```

```
    SELECT
```

```
        c.titulo,
```

```
        c.tipo_contenido,
```

```
        u.ciudad_residencia,
```

```
        COUNT(r.id_reproduccion)      AS total_reproducciones,
```

```
        ROUND(AVG(r.porcentaje_avance), 2) AS promedio_avance_pct
```

```
    FROM Reproduccion r
```

```
        JOIN Perfil    p  ON r.Perfil_id_perfil    = p.id_perfil
```

```
        JOIN Usuario   u  ON p.Usuario_id_usuario = u.id_usuario
```

```
        JOIN contenido c  ON r.contenido_id_contenido =  
c.id_contenido
```

```
    WHERE UPPER(u.ciudad_residencia) = UPPER('&&ciudad')
```

```
    GROUP BY c.titulo, c.tipo_contenido, u.ciudad_residencia
```

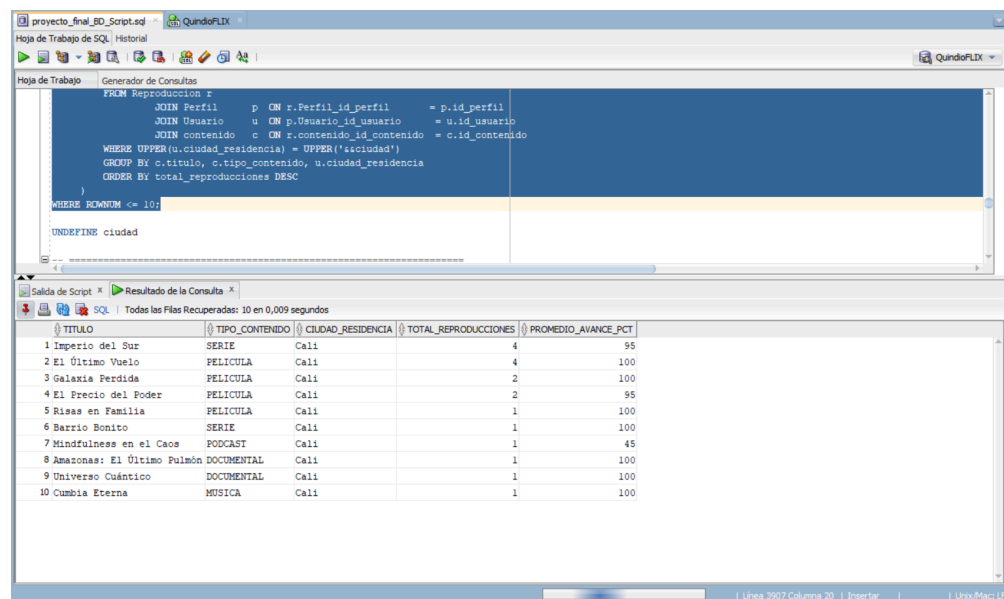
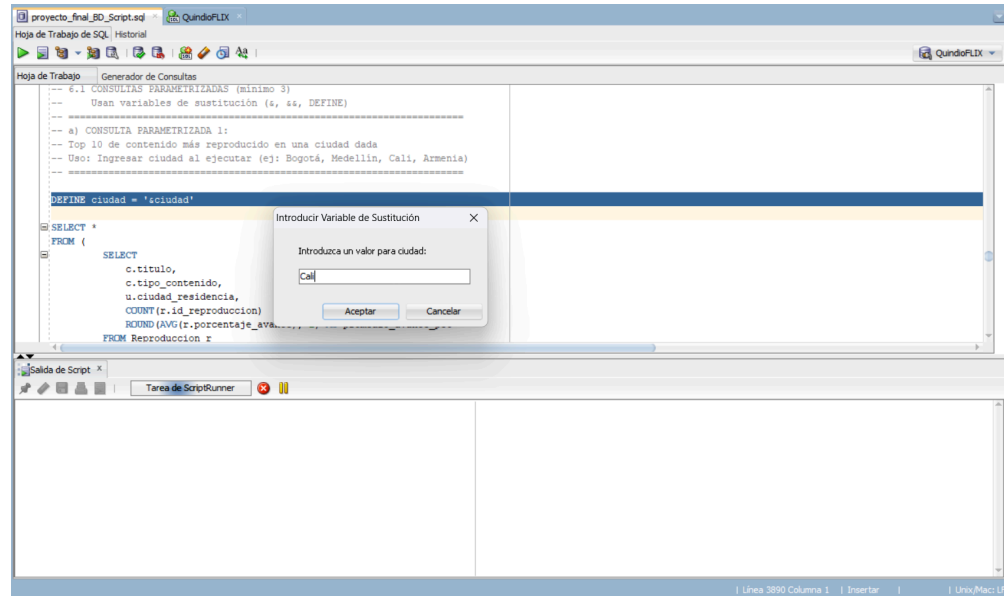
```
    ORDER BY total_reproducciones DESC
```

```
)
```

```
WHERE ROWNUM <= 10;
```

```
UNDEFINE ciudad
```

Resultado:



2. Consulta Parametrizada 2 - Ingresos por plan en un mes y año específicos

Código sql:

DEFINE mes = '&mes'

DEFINE anio = '&anio'

SELECT

ps.nombre

AS plan,

```

TO_CHAR(pg.fecha_pago, 'MM/YYYY') AS periodo,

COUNT(pg.id_pago)          AS cantidad_pagos,

SUM(pg.monto)                AS ingreso_bruto,

SUM(NVL(pg.valor_descuento, 0)) AS total_descuentos,

SUM(pg.monto) - SUM(NVL(pg.valor_descuento, 0)) AS ingreso_netos

FROM Pago      pg

      JOIN suscripcion s ON pg.suscripcion_id_suscripcion =
s.id_suscripcion

      JOIN Plan_suscripcion ps ON s.Plan_suscripcion_id_plan = ps.id_plan

WHERE pg.estado_pago  = 'Exitoso'

AND EXTRACT(MONTH FROM pg.fecha_pago) = &mes

AND EXTRACT(YEAR FROM pg.fecha_pago) = &anio

GROUP BY ps.nombre, TO_CHAR(pg.fecha_pago, 'MM/YYYY')

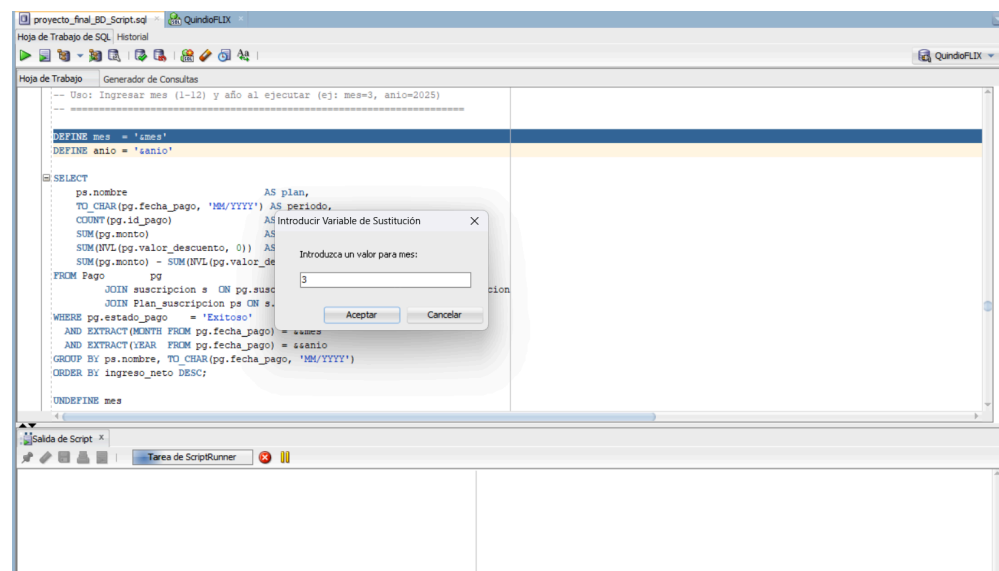
ORDER BY ingreso_netos DESC;

UNDEFINE mes

UNDEFINE anio

```

Resultado:



proyecto_final_BO_Script.sql

Hoja de Trabajo de SQL Historial

Hoja de Trabajo Generador de Consultas

-- Uso: Ingresar mes (1-12) y año al ejecutar (ej: mes=3, año=2025)

```

DEFINE mes = 'mes'
DEFINE año = 'año'

SELECT
    ps.nombre AS plan,
    TO_CHAR(pg.fecha_pago, 'MM/YYYY') AS periodo,
    COUNT(pg.id_pago) AS cantidad_pagos,
    SUM(pg.monto) AS ingreso_bruto,
    SUM(NVL(pg.valor_descuento, 0)) AS total_descuentos,
    SUM(pg.monto) - SUM(NVL(pg.valor_descuento, 0)) AS ingreso_netos
FROM Pago
    JOIN suscripcion s ON pg.suscripcion_id_suscripcion = s.id_suscripcion
    JOIN Plan_suscripcion ps ON s.Plan_suscripcion_id_plan = ps.id_plan
WHERE pg.estado_pago = 'Exitoso'
AND EXTRACT(MONTH FROM pg.fecha_pago) = &mes
AND EXTRACT(YEAR FROM pg.fecha_pago) = &año
GROUP BY ps.nombre, TO_CHAR(pg.fecha_pago, 'MM/YYYY')
ORDER BY ingreso_netos DESC;

UNDEFINE mes
  
```

Salida de Script Resultado de la Consulta

Todas las Filas Recuperadas: 2 en 0,015 segundos

PLAN	PERIODO	CANTIDAD_PAGOS	INGRESO_BRUTO	TOTAL_DESCUENTOS	INGRESO_NETO
1 Premium	03/2024	5	174500	3490	171010
2 Estandar	03/2024	1	24900	0	24900

3. Consulta Parametrizada 3 - Calificación promedio por categoría para un género

Código:

DEFINE genero_nombre = '&genero_nombre'

SELECT

c.tipo_contenido AS categoria,

g.nombre AS genero,

COUNT(DISTINCT c.id_contenido) AS cantidad_titulos,

COUNT(cal.id_calificacion) AS total_calificaciones,

ROUND(AVG(cal.estrellas), 2) AS calificacion_promedio,

MIN(cal.estrellas) AS calificacion_minima,

MAX(cal.estrellas) AS calificacion_maxima

FROM contenido c

JOIN tipo_genero tg ON c.id_contenido =
tg.contenido_id_contenido

JOIN genero g ON tg.genero_id_genero = g.id_genero

LEFT JOIN Calificacion cal ON c.id_contenido =
cal.contenido_id_contenido

WHERE UPPER(g.nombre) = UPPER('&&genero_nombre')

GROUP BY c.tipo_contenido, g.nombre

ORDER BY calificacion_promedio DESC NULLS LAST;

UNDEFINE genero_nombre

Resultado:

The screenshot shows the QuindioPLX SQL editor with a query window. A dialog box titled "Introducir Variable de Sustitución" is open, prompting the user to enter a value for the variable "genero_nombre". The value "Drama" has been entered in the input field. The background query is partially visible, showing a SELECT statement with various aggregation functions and a WHERE clause.

The screenshot shows the same query in the QuindioPLX SQL editor, but now the results are displayed in a table. The table has 7 columns: CATEGORIA, GENERO, CANTIDAD_TITULOS, TOTAL_CALIFICACIONES, CALIFICACION_PROMEDIO, CALIFICACION_MINIMA, and CALIFICACION_MAXIMA. There are 4 rows of data, all categorized as "Drama".

CATEGORIA	GENERO	CANTIDAD_TITULOS	TOTAL_CALIFICACIONES	CALIFICACION_PROMEDIO	CALIFICACION_MINIMA	CALIFICACION_MAXIMA
1 SERIE	Drama	4	8	4,25	1	5
2 DOCUMENTAL	Drama	5	7	4,14	3	5
3 PELICULA	Drama	3	5	3,6	1	5
4 PODCAST	Drama	3	4	3,25	2	4

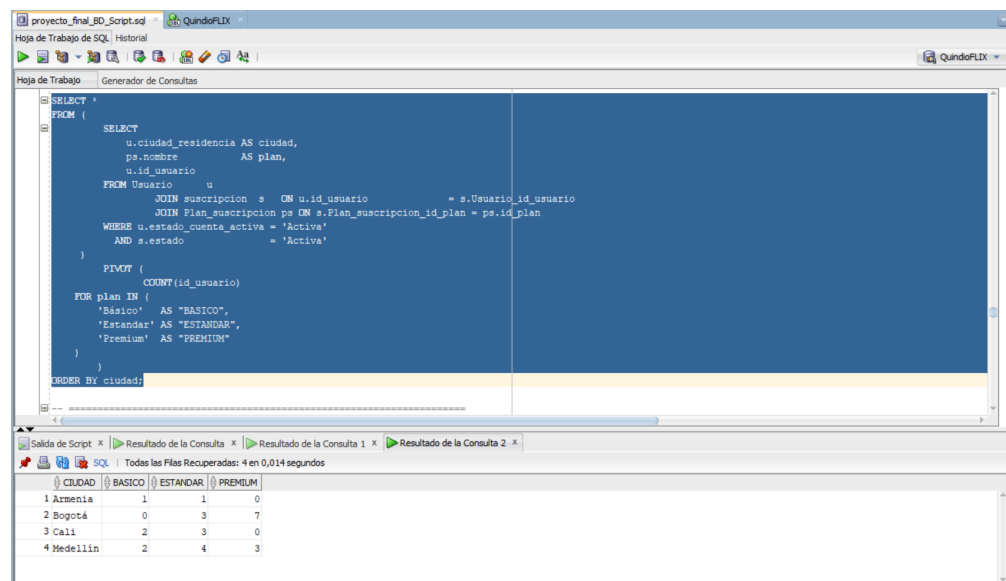
6.2 Tablas de Referencias Cruzadas

1. PIVOT 1 - Usuarios activos por ciudad y plan de suscripción

Código:

```
SELECT *  
  
FROM (  
  
    SELECT  
  
        u.ciudad_residencia AS ciudad,  
  
        ps.nombre          AS plan,  
  
        u.id_usuario  
  
    FROM Usuario    u  
  
        JOIN suscripcion s ON u.id_usuario      =  
s.Usuario_id_usuario  
  
        JOIN Plan_suscripcion ps ON s.Plan_suscripcion_id_plan =  
ps.id_plan  
  
    WHERE u.estado_cuenta_activa = 'Activa'  
  
        AND s.estado              = 'Activa'  
  
    )  
  
    PIVOT (  
  
        COUNT(id_usuario)  
  
    FOR plan IN (  
  
        'Básico' AS "BASICO",  
  
        'Estandar' AS "ESTANDAR",  
  
        'Premium' AS "PREMIUM"  
  
    )  
  
    )  
  
ORDER BY ciudad;
```

Resultado:



The screenshot shows the QuindioFLIX SQL editor with a query that uses a PIVOT function to count users by city and plan type. The results are displayed in a table below the query editor.

Ciudad	Básico	Estandar	Premium
1 Armenia	1	1	0
2 Bogotá	0	3	7
3 Cali	2	3	0
4 Medellín	2	4	3

2. PIVOT 2 - Reproducciones por tipo de contenido y dispositivo

Código:

SELECT *

FROM (

SELECT *

FROM (

SELECT

c.tipo_contenido AS categoria,

r.dispositivo,

r.id_reproduccion

FROM Reproduccion r

JOIN contenido c ON r.contenido_id_contenido = c.id_contenido

)

PIVOT (

COUNT(id_reproduccion)

FOR dispositivo IN (

'Celular' AS "CELULAR",

```
'Tablet' AS "TABLET",

'TV' AS "TV",

'Computador' AS "COMPUTADOR"

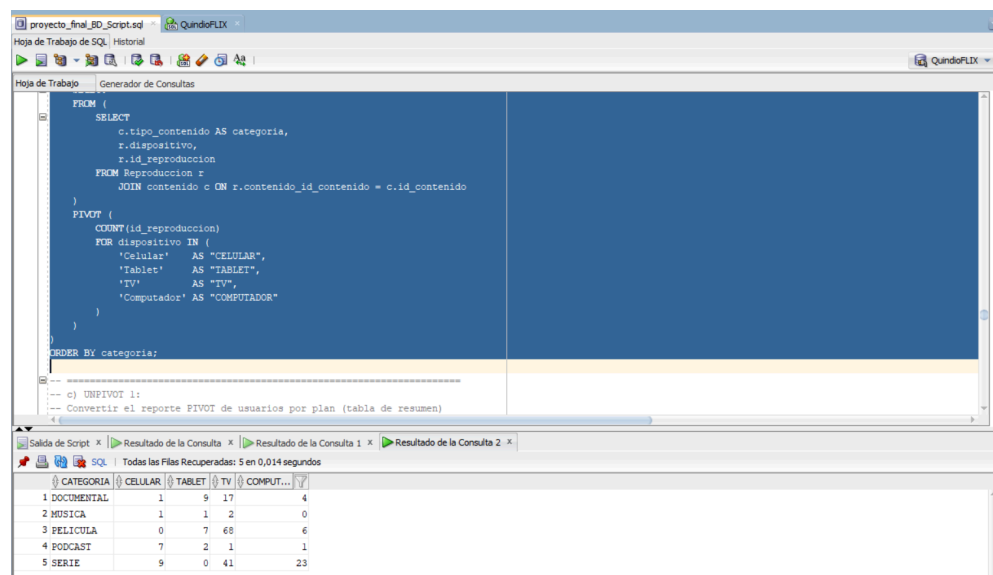
)

)

)

ORDER BY categoria;
```

Resultado:



The screenshot shows the Quindoflix SQL IDE interface. The top pane displays a SQL query that uses a PIVOT function to count the number of reproductions for each device type (Celular, Tablet, TV, Computador) across different content categories. The bottom pane shows the results of the query, which is a table with 5 rows and 5 columns.

CATEGORIA	CELULAR	TABLET	TV	COMPUTADOR
1 DOCUMENTAL	1	9	17	4
2 MUSICA	1	1	2	0
3 PELICULA	0	7	68	6
4 PODCAST	7	2	1	1
5 SERIE	9	0	41	23

3. UNPIVOT 1 - Usuarios activos por ciudad convertido a filas

Código:

```
WITH pivot_usuarios AS (

SELECT *

FROM (

SELECT

u.ciudad_residencia AS ciudad,

ps.nombre AS plan,

u.id_usuario
```

```
FROM Usuario u

        JOIN suscripcion s ON u.id_usuario =
s.Usuario_id_usuario

        JOIN Plan_suscripcion ps ON s.Plan_suscripcion_id_plan =
ps.id_plan

WHERE u.estado_cuenta_activa = 'Activa'

AND s.estado = 'Activa'

)

PIVOT (

        COUNT(id_usuario)

FOR plan IN (

        'Básico' AS "BASICO",

        'Estandar' AS "ESTANDAR",

        'Premium' AS "PREMIUM"

)

)

SELECT ciudad, plan, usuarios_activos

FROM pivot_usuarios

UNPIVOT (

        usuarios_activos

FOR plan IN (

        "BASICO" AS 'Plan Básico',

        "ESTANDAR" AS 'Plan Estándar',

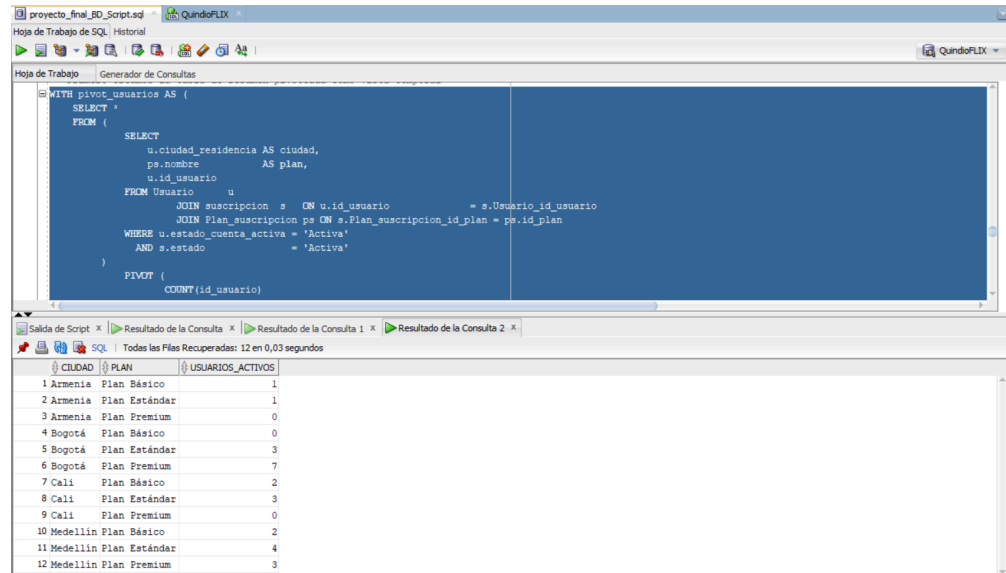
        "PREMIUM" AS 'Plan Premium'

)

)
```

ORDER BY ciudad, plan;

Resultado:



The screenshot shows a SQL query editor with a query that uses a PIVOT function to count active users by city and plan. The results are displayed in a table with 12 rows and 3 columns: CIUDAD, PLAN, and USUARIOS_ACTIVOS.

CIUDAD	PLAN	USUARIOS_ACTIVOS
1	Armenia Plan Básico	1
2	Armenia Plan Estándar	1
3	Armenia Plan Premium	0
4	Bogotá Plan Básico	0
5	Bogotá Plan Estándar	3
6	Bogotá Plan Premium	7
7	Cali Plan Básico	2
8	Cali Plan Estándar	3
9	Cali Plan Premium	0
10	Medellín Plan Básico	2
11	Medellín Plan Estándar	4
12	Medellín Plan Premium	3

4. UNPIVOT 2 - Ingresos del primer trimestre 2024 convertido a filas

Código:

WITH ingresos_pivot AS (

SELECT *

FROM (

SELECT

ps.nombre AS plan,

TO_CHAR(pg.fecha_pago, 'MON_YYYY') AS mes_periodo,

pg.monto

FROM Pago pg

JOIN suscripcion s ON pg.suscripcion_id_suscripcion = s.id_suscripcion

JOIN Plan_suscripcion ps ON s.Plan_suscripcion_id_plan = ps.id_plan

WHERE pg.estado_pago = 'Exitoso'

AND EXTRACT(YEAR FROM pg.fecha_pago) = 2024

```
AND EXTRACT(MONTH FROM pg.fecha_pago) IN (1, 2, 3)

)

PIVOT (

    SUM(monto)

FOR mes_periodo IN (

    'JAN_2024' AS "ENERO_2024",

    'FEB_2024' AS "FEBRERO_2024",

    'MAR_2024' AS "MARZO_2024"

)

)

)

SELECT plan, mes, ingreso

FROM ingresos_pivot

    UNPIVOT (

        ingreso

        FOR mes IN (

            "ENERO_2024" AS 'Enero 2024',

            "FEBRERO_2024" AS 'Febrero 2024',

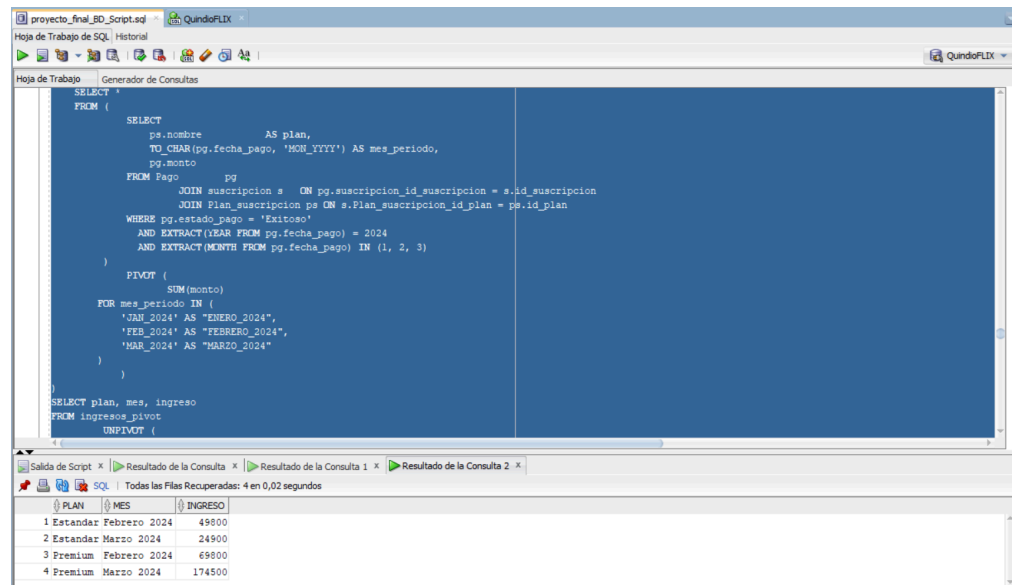
            "MARZO_2024" AS 'Marzo 2024'

        )

    )

ORDER BY plan, mes;
```

Resultado:



PLAN	MES	INGRESO
1 Estándar	Febrero 2024	45000
2 Estándar	Marzo 2024	24900
3 Premium	Febrero 2024	69000
4 Premium	Marzo 2024	174500

6.3 Funciones Avanzadas del GROUP BY

1. ROLLUP - Ingresos por ciudad y plan con subtotales

Código:

SELECT

NVL(u.ciudad_residencia, '*** GRAN TOTAL ***') AS ciudad,

NVL(ps.nombre, '--- SUBTOTAL CIUDAD ---') AS plan,

COUNT(pg.id_pago) AS cantidad_pagos,

SUM(pg.monto) AS ingreso_bruto,

ROUND(AVG(pg.monto), 2) AS ticket_promedio

FROM Pago pg

JOIN suscripcion s ON pg.suscripcion_id_suscripcion = s.id_suscripcion

JOIN Plan_suscripcion ps ON s.Plan_suscripcion_id_plan = ps.id_plan

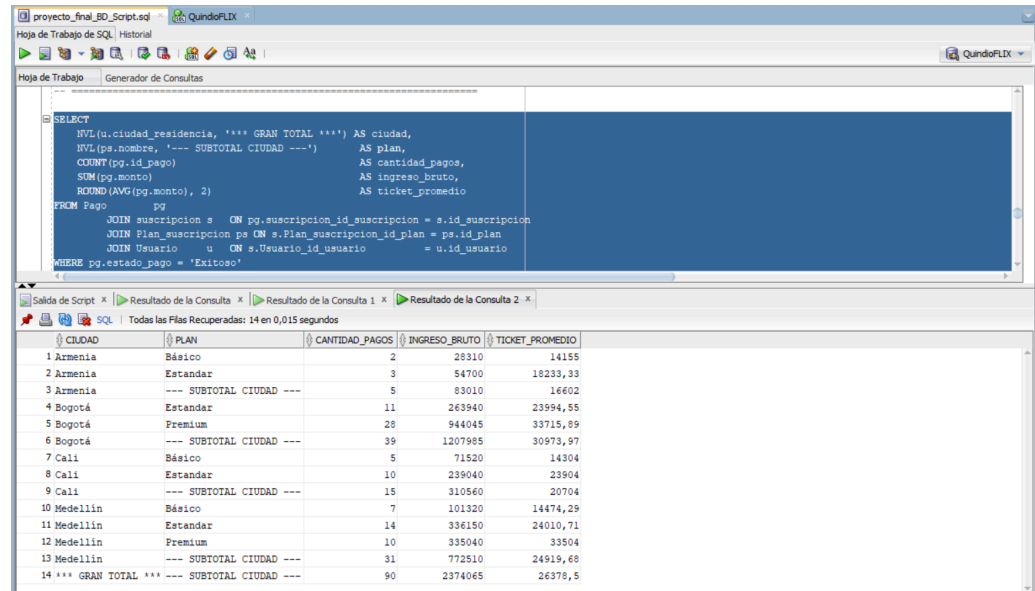
JOIN Usuario u ON s.Usuario_id_usuario = u.id_usuario

WHERE pg.estado_pago = 'Exitoso'

GROUP BY ROLLUP(u.ciudad_residencia, ps.nombre)

ORDER BY u.ciudad_residencia NULLS LAST, ps.nombre NULLS LAST;

Resultado:



The screenshot shows the Quindoflix SQL IDE interface. The top pane displays a SQL query that joins tables for payment information, subscription details, and user data. The bottom pane shows the results of the query, which is a table with columns: CIUDAD, PLAN, CANTIDAD_PAGOS, INGRESO_BRUTO, and TICKET_PROMEDIO. The results are grouped by city and plan type, with a final 'GRAN TOTAL' row.

CIUDAD	PLAN	CANTIDAD_PAGOS	INGRESO_BRUTO	TICKET_PROMEDIO
1 Armenia	Básico	2	28310	14155
2 Armenia	Estandar	3	54700	18233,33
3 Armenia	--- SUBTOTAL CIUDAD ---	5	83010	16602
4 Bogotá	Estandar	11	263940	23994,55
5 Bogotá	Premium	20	944045	33715,09
6 Bogotá	--- SUBTOTAL CIUDAD ---	39	1207985	30973,97
7 Cali	Básico	5	71520	14304
8 Cali	Estandar	10	239040	23904
9 Cali	--- SUBTOTAL CIUDAD ---	15	310560	20704
10 Medellín	Básico	7	101320	14474,29
11 Medellín	Estandar	14	336150	24010,71
12 Medellín	Premium	10	335040	33504
13 Medellín	--- SUBTOTAL CIUDAD ---	31	772510	24919,68
14 *** GRAN TOTAL ***	--- SUBTOTAL CIUDAD ---	90	2374065	26378,5

2. CUBE - Reproducciones por tipo de contenido y dispositivo con todas las combinaciones

Código:

SELECT

NVL(c.tipo_contenido, '=== TOTAL GENERAL ===') AS tipo_contenido,

NVL(r.dispositivo, '--- TODOS LOS DISP ---') AS dispositivo,

COUNT(r.id_reproduccion) AS total_reproducciones,

ROUND(AVG(r.porcentaje_avance), 2) AS promedio_avance_pct

FROM Reproduccion r

JOIN contenido c ON r.contenido_id_contenido = c.id_contenido

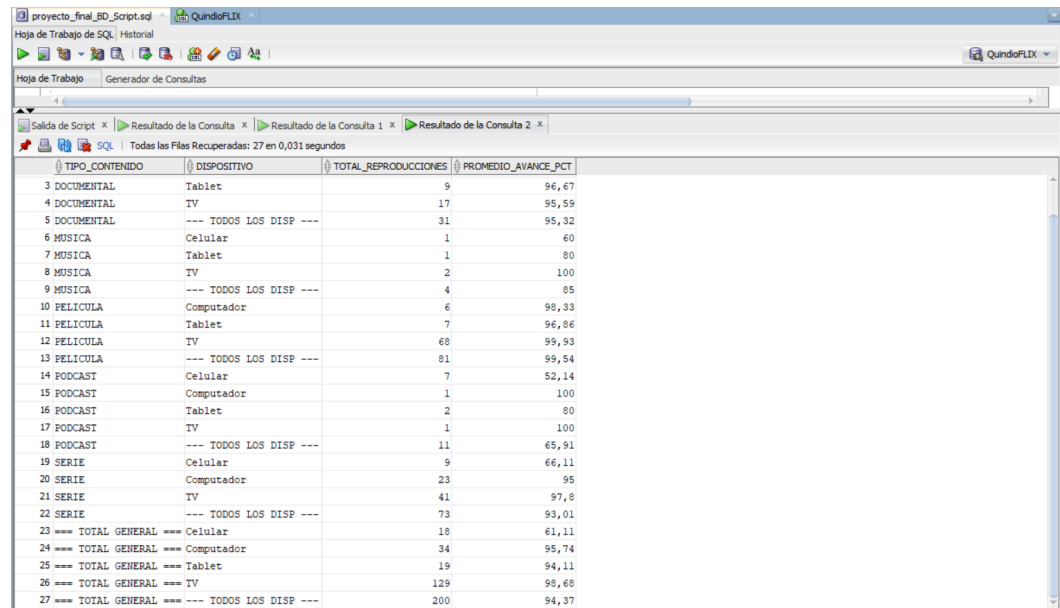
GROUP BY CUBE(c.tipo_contenido, r.dispositivo)

ORDER BY

c.tipo_contenido NULLS LAST,

r.dispositivo NULLS LAST;

Resultado:



TIPO_CONTENIDO	DISPOSITIVO	TOTAL_REPRODUCCIONES	PROMEDIO_AVANCE_PCT
3 DOCUMENTAL	Tablet	9	96,67
4 DOCUMENTAL	TV	17	95,59
5 DOCUMENTAL	--- TODOS LOS DISP ---	31	95,32
6 MUSICA	Celular	1	60
7 MUSICA	Tablet	1	80
8 MUSICA	TV	2	100
9 MUSICA	--- TODOS LOS DISP ---	4	85
10 PELICULA	Computador	6	98,33
11 PELICULA	Tablet	7	96,86
12 PELICULA	TV	68	99,93
13 PELICULA	--- TODOS LOS DISP ---	81	99,54
14 PODCAST	Celular	7	52,14
15 PODCAST	Computador	1	100
16 PODCAST	Tablet	2	80
17 PODCAST	TV	1	100
18 PODCAST	--- TODOS LOS DISP ---	11	65,91
19 SERIE	Celular	9	66,11
20 SERIE	Computador	23	95
21 SERIE	TV	41	97,8
22 SERIE	--- TODOS LOS DISP ---	73	93,01
23 === TOTAL GENERAL ===	Celular	18	61,11
24 === TOTAL GENERAL ===	Computador	34	95,74
25 === TOTAL GENERAL ===	Tablet	19	94,11
26 === TOTAL GENERAL ===	TV	129	98,68
27 === TOTAL GENERAL ===	--- TODOS LOS DISP ---	200	94,37

3. GROUPING - Ingresos con etiquetas legibles para subtotales

Código:

SELECT

CASE

WHEN GROUPING(u.ciudad_residencia) = 1 AND
GROUPING(ps.nombre) = 1

THEN '*** GRAN TOTAL PLATAFORMA ***'

WHEN GROUPING(ps.nombre) = 1

THEN 'SUBTOTAL - ' || u.ciudad_residencia

ELSE u.ciudad_residencia

END AS ciudad,

CASE

WHEN GROUPING(ps.nombre) = 1 THEN '(Todos los planes)'

ELSE ps.nombre

END AS plan,

GROUPING(u.ciudad_residencia) AS es_subtotal_ciudad, -- 1 si es fila de
subtotal

```

GROUPING(ps.nombre)      AS es_subtotal_plan,  -- 1 si es fila de
subtotal

COUNT(pg.id_pago)       AS cantidad_pagos,

SUM(pg.monto)            AS ingreso_total,

SUM(NVL(pg.valor_descuento,0))AS descuentos_aplicados

FROM Pago      pg

      JOIN suscripcion s  ON pg.suscripcion_id_suscripcion =
s.id_suscripcion

      JOIN Plan_suscripcion ps ON s.Plan_suscripcion_id_plan = ps.id_plan

      JOIN Usuario      u  ON s.Usuario_id_usuario      = u.id_usuario

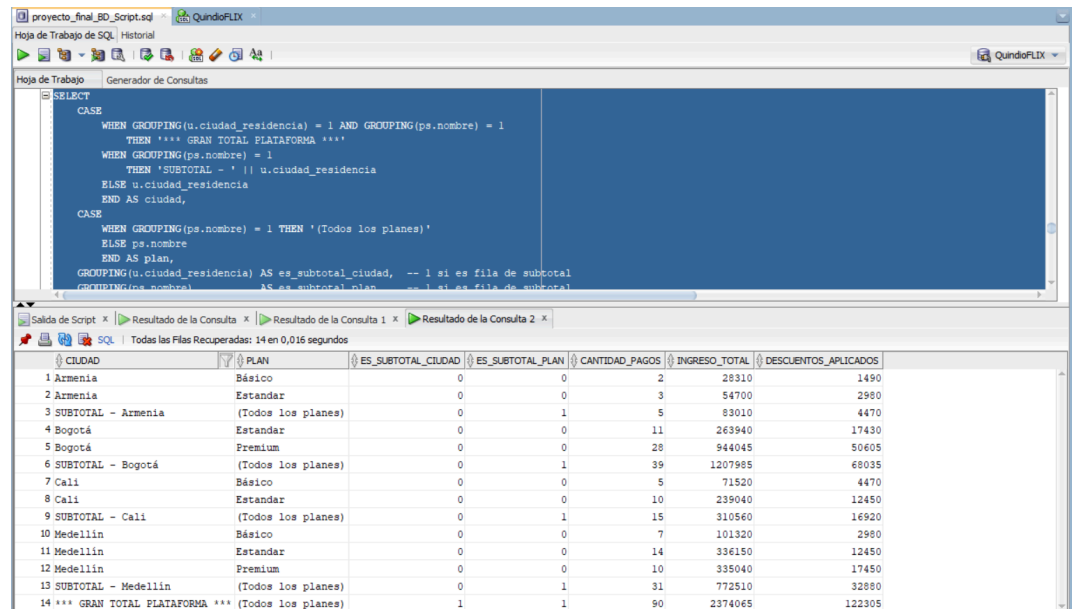
WHERE pg.estado_pago = 'Exitoso'

GROUP BY ROLLUP(u.ciudad_residencia, ps.nombre)

ORDER BY u.ciudad_residencia NULLS LAST, ps.nombre NULLS LAST;

```

Resultado:



Ciudad	Plan	ES_SUBTOTAL_CUIDAD	ES_SUBTOTAL_PLAN	CANTIDAD_PAGOS	INGRESO_TOTAL	DESCUENTOS_APLICADOS
1 Armenia	Básico	0	0	2	28310	1490
2 Armenia	Estandar	0	0	3	54700	2980
3 SUBTOTAL - Armenia	(Todos los planes)	0	1	5	83010	4470
4 Bogotá	Estandar	0	0	11	263940	17430
5 Bogotá	Premium	0	0	28	944045	50605
6 SUBTOTAL - Bogotá	(Todos los planes)	0	1	39	1207985	68035
7 Cali	Básico	0	0	5	71520	4470
8 Cali	Estandar	0	0	10	239040	12450
9 SUBTOTAL - Cali	(Todos los planes)	0	1	15	310560	16920
10 Medellín	Básico	0	0	7	101320	2980
11 Medellín	Estandar	0	0	14	336150	12450
12 Medellín	Premium	0	0	10	335040	17450
13 SUBTOTAL - Medellín	(Todos los planes)	0	1	31	772510	32880
14 *** GRAN TOTAL PLATAFORMA ***	(Todos los planes)	1	1	90	2374065	122305

4. GROUPING SETS - Totales separados por tipo de contenido y por ciudad

Código:

SELECT

```

NVL(c.tipo_contenido, '--- (Agrupado por ciudad) ---') AS tipo_contenido,

NVL(u.ciudad_residencia, '--- (Agrupado por tipo) ---') AS ciudad,

COUNT(r.id_reproduccion)                                AS total_reproducciones,

ROUND(AVG(r.porcentaje_avance), 2)                        AS
promedio_avance

FROM Reproduccion r

    JOIN contenido  c ON r.contenido_id_contenido = c.id_contenido

    JOIN Perfil    p ON r.Perfil_id_perfil      = p.id_perfil

    JOIN Usuario   u ON p.Usuario_id_usuario   = u.id_usuario

GROUP BY GROUPING SETS (

    (c.tipo_contenido),  -- Total de reproducciones por cada tipo de contenido

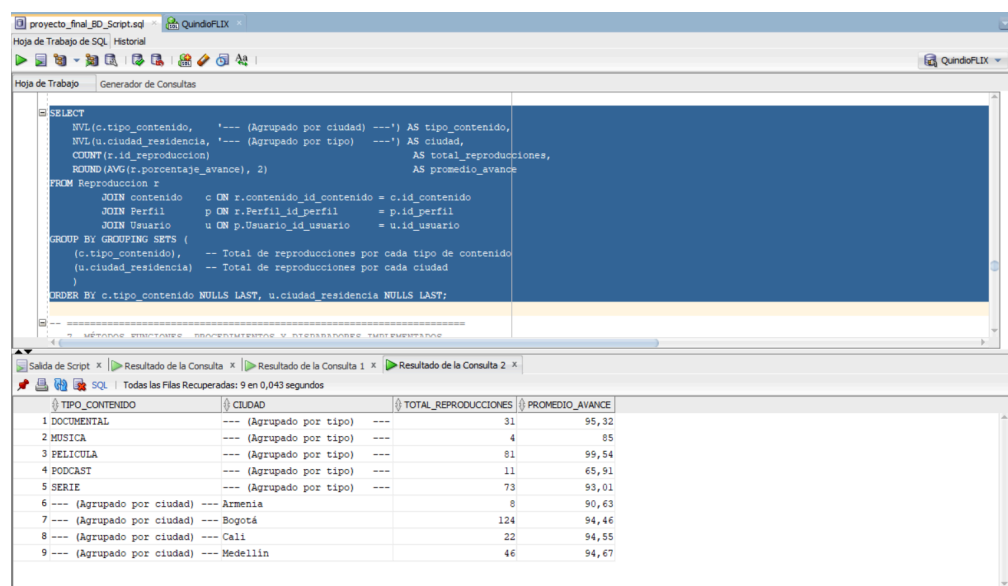
    (u.ciudad_residencia) -- Total de reproducciones por cada ciudad

)

ORDER BY c.tipo_contenido NULLS LAST, u.ciudad_residencia NULLS
LAST;

```

Resultado:



TIPO_CONTENIDO	CIUDAD	TOTAL_REPRODUCCIONES	PROMEDIO_AVANCE
1 DOCUMENTAL	--- (Agrupado por tipo) ---	31	95,32
2 MUSICA	--- (Agrupado por tipo) ---	4	85
3 PELICULA	--- (Agrupado por tipo) ---	81	99,54
4 PODCAST	--- (Agrupado por tipo) ---	11	65,91
5 SERIE	--- (Agrupado por tipo) ---	73	93,01
6 --- (Agrupado por ciudad) ---	Armenia	8	90,63
7 --- (Agrupado por ciudad) ---	Bogotá	124	94,46
8 --- (Agrupado por ciudad) ---	Cali	22	94,55
9 --- (Agrupado por ciudad) ---	Medellín	46	94,67

7. MÉTODOS FUNCIONES, PROCEDIMIENTOS Y DISPARADORES

IMPLEMENTADOS

7.1 Cursores

1. Cursor 1 - Reporte de usuarios morosos con días de mora y monto adeudado

Código:

```
SET SERVEROUTPUT ON;
```

```
DECLARE
```

```
-- Cursor explícito con parámetro de días de corte
```

```
CURSOR cur_morosos (p_dias_corte NUMBER DEFAULT 30) IS
```

```
SELECT
```

```
    u.id_usuario,
```

```
    u.nombre,
```

```
    u.email,
```

```
    ps.nombre                AS plan,
```

```
    ps.costo                 AS monto_mensual,
```

```
    s.fecha_vencimiento,
```

```
    TRUNC(SYSDATE - s.fecha_vencimiento)    AS dias_mora,
```

```
-- Estimación del monto adeudado según meses vencidos
```

```
ROUND(
```

```
    CEIL(TRUNC(SYSDATE - s.fecha_vencimiento) / 30) * ps.costo
```

```
    , 2)                AS estimado_adeudado
```

```
FROM Usuario    u
```

```
    JOIN suscripcion s ON u.id_usuario = s.Usuario_id_usuario
```

```
    JOIN Plan_suscripcion ps ON s.Plan_suscripcion_id_plan = ps.id_plan
```

```
WHERE s.estado IN ('Activa', 'Suspendida')
```

```
AND s.fecha_vencimiento < SYSDATE - p_dias_corte
```

```
AND NOT EXISTS (
```

-- Que no tengan un pago exitoso después del vencimiento

SELECT 1

FROM Pago pg

WHERE pg.suscripcion_id_suscripcion = s.id_suscripcion

AND pg.estado_pago = 'Exitoso'

AND pg.fecha_pago > s.fecha_vencimiento

)

ORDER BY dias_mora DESC;

-- Variables de trabajo

v_registro cur_morosos%ROWTYPE;

v_contador NUMBER := 0;

v_total_adeudado NUMBER := 0;

BEGIN

DBMS_OUTPUT.PUT_LINE('=====');
DBMS_OUTPUT.PUT_LINE('=====');

DBMS_OUTPUT.PUT_LINE(' REPORTE DE USUARIOS MOROSOS -
QuindioFlix');

DBMS_OUTPUT.PUT_LINE(' Generado: ' || TO_CHAR(SYSDATE,
'DD/MM/YYYY HH24:MI:SS'));

DBMS_OUTPUT.PUT_LINE('=====');
DBMS_OUTPUT.PUT_LINE('=====');

DBMS_OUTPUT.PUT_LINE(

RPAD('NOMBRE', 25) ||

RPAD('EMAIL', 30) ||

RPAD('PLAN', 12) ||

RPAD('DÍAS MORA', 15) ||

'MONTO ADEUDADO'

);

```
DBMS_OUTPUT.PUT_LINE(RPAD('-', 100, '-'));

OPEN cur_morosos(30);

LOOP

FETCH cur_morosos INTO v_registro;

EXIT WHEN cur_morosos%NOTFOUND;

v_contador := v_contador + 1;

v_total_adeudado :=

v_total_adeudado +

NVL(v_registro.estimado_adeudado, v_registro.monto_mensual);

DBMS_OUTPUT.PUT_LINE(

RPAD(v_registro.nombre, 25) ||

RPAD(v_registro.email, 30) ||

RPAD(v_registro.plan, 12) ||

RPAD(v_registro.dias_mora || ' días', 15) ||

'$ ' || TO_CHAR(

NVL(v_registro.estimado_adeudado,

v_registro.monto_mensual),

'FM999,990.00'

)

);

END LOOP;

CLOSE cur_morosos;

DBMS_OUTPUT.PUT_LINE(RPAD('-', 100, '-'));

DBMS_OUTPUT.PUT_LINE('Total usuarios morosos : ' || v_contador);

DBMS_OUTPUT.PUT_LINE(

'Total estimado adeudado: $ ' ||

TO_CHAR(v_total_adeudado, 'FM9,999,990.00')

);
```

```
DBMS_OUTPUT.PUT_LINE('=====');
=====');
```

```
EXCEPTION
```

```
WHEN OTHERS THEN
```

```
IF cur_morosos%ISOPEN THEN
```

```
CLOSE cur_morosos;
```

```
END IF;
```

```
DBMS_OUTPUT.PUT_LINE(
```

```
'ERROR en cursor morosos: ' || SQLERRM
```

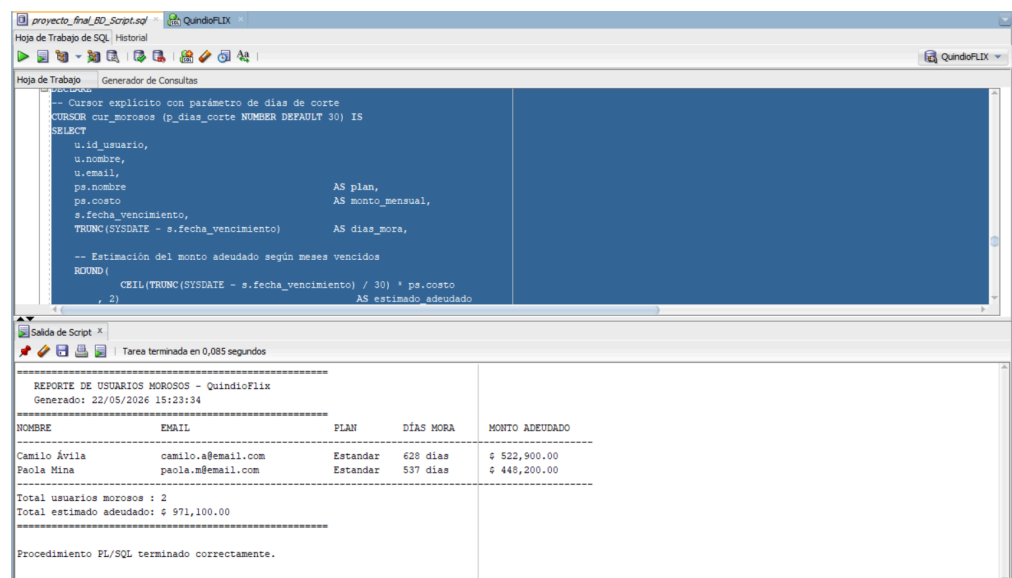
```
);
```

```
RAISE;
```

```
END;
```

```
/
```

Resultado:



```
-- Cursor explícito con parámetro de días de corte
CURSOR cur_morosos (p_dias_corte NUMBER DEFAULT 30) IS
SELECT
    u.id_usuario,
    u.nombre,
    u.email,
    pa.nombre AS plan,
    pa.costo AS monto_mensual,
    s.fecha_vencimiento,
    TRUNC(SYSDATE - s.fecha_vencimiento) AS dias_mora,
    -- Estimación del monto adeudado según meses vencidos
    ROUND(
        CEIL(TRUNC(SYSDATE - s.fecha_vencimiento) / 30) * pa.costo
        , 2) AS estimado_adeudado
```

Salida de Script x

Tarea terminada en 0,085 segundos

```
=====
REPORTE DE USUARIOS MOROSOS - QuindioFlix
Generado: 22/05/2024 15:23:34
=====
```

NOMBRE	EMAIL	PLAN	DÍAS MORA	MONTO ADEUDADO
Camilo Ávila	camilo.a@email.com	Estandar	628 dias	\$ 522,900.00
Paola Mina	paola.m@email.com	Estandar	537 dias	\$ 448,200.00

```
-----
Total usuarios morosos : 2
Total estimado adeudado: $ 971,100.00
-----
Procedimiento PL/SQL terminado correctamente.
```

2. Cursor 2 - Reporte de popularidad del catálogo por nivel de completitud

Código:

```
SET SERVEROUTPUT ON;
```

```
DECLARE
```

-- Cursor que recorre todos los contenidos con sus métricas

CURSOR cur_catalogo IS

SELECT

c.id_contenido,

c.titulo,

c.tipo_contenido,

COUNT(r.id_reproduccion) AS total_reprod,

COUNT(CASE WHEN r.porcentaje_avance >= 90 THEN 1 END) AS
reprod_completas,

ROUND(

COUNT(CASE WHEN r.porcentaje_avance >= 90 THEN 1 END) *
100.0 /

NULLIF(COUNT(r.id_reproduccion), 0)

, 2) AS pct_completitud

FROM contenido c

LEFT JOIN Reproduccion r ON c.id_contenido =
r.contenido_id_contenido

GROUP BY c.id_contenido, c.titulo, c.tipo_contenido

ORDER BY reprod_completas DESC;

v_reg cur_catalogo%ROWTYPE;

v_nivel VARCHAR2(20);

v_procesados NUMBER := 0;

BEGIN

DBMS_OUTPUT.PUT_LINE('=====');
DBMS_OUTPUT.PUT_LINE('=====');

DBMS_OUTPUT.PUT_LINE(' REPORTE DE POPULARIDAD DEL
CATÁLOGO - QuindioFlix');

DBMS_OUTPUT.PUT_LINE('=====');

```
=====');

DBMS_OUTPUT.PUT_LINE(RPAD('TÍTULO', 35) || RPAD('TIPO', 12) ||
                      RPAD('TOTAL', 8) || RPAD('COMPLETAS', 12) ||
                      RPAD('% COMPLETITUD', 16) || 'NIVEL');

DBMS_OUTPUT.PUT_LINE(RPAD('-', 90, '-'));

FOR v_reg IN cur_catalogo LOOP

    v_procesados := v_procesados + 1;

    -- Calcular nivel de popularidad basado en reproducciones completas

    v_nivel := CASE

        WHEN v_reg.reprod_completas >= 10 THEN 'VIRAL'

        WHEN v_reg.reprod_completas >= 5  THEN 'POPULAR'

        WHEN v_reg.reprod_completas >= 2  THEN 'NORMAL'

        WHEN v_reg.reprod_completas >= 1  THEN 'BAJO'

        ELSE                                'SIN VISTAS'

    END;

    DBMS_OUTPUT.PUT_LINE(
        RPAD(SUBSTR(v_reg.titulo, 1, 33), 35) ||
        RPAD(v_reg.tipo_contenido, 12) ||
        RPAD(NVL(v_reg.total_reprod, 0), 8) ||
        RPAD(NVL(v_reg.reprod_completas, 0), 12) ||
        RPAD(NVL(v_reg.pct_completitud, 0) || '%', 16) ||
        v_nivel
    );

    -- En un sistema real, aquí se actualizaría un campo popularidad

    -- UPDATE contenido SET popularidad = v_nivel WHERE id_contenido
    = v_reg.id_contenido;

END LOOP;

DBMS_OUTPUT.PUT_LINE(RPAD('-', 90, '-'));
```

```
DBMS_OUTPUT.PUT_LINE('Total de contenidos procesados: ' ||  
v_procesados);
```

```
-- COMMIT; -- Descomentar si se hace UPDATE real
```

```
EXCEPTION
```

```
WHEN OTHERS THEN
```

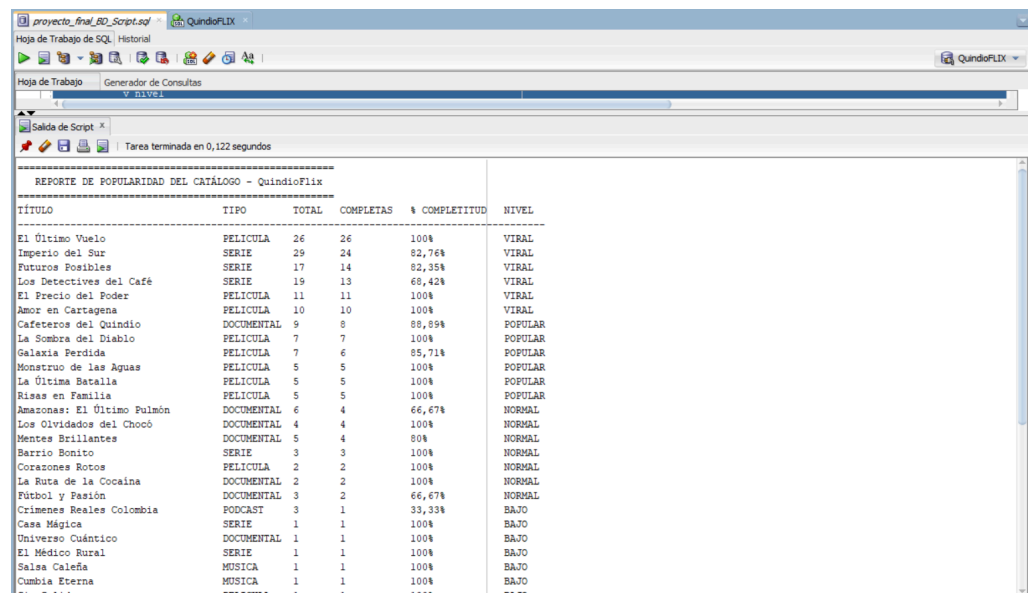
```
DBMS_OUTPUT.PUT_LINE('ERROR en cursor catálogo: ' ||  
SQLERRM);
```

```
RAISE;
```

```
END;
```

```
/
```

Resultado:



Salida de Script x

Tarea terminada en 0,122 segundos

TÍTULO	TIPO	TOTAL	COMPLETAS	% COMPLETITUD	NIVEL
El Último Vuelo	FELICULA	26	26	100%	VIRAL
Imperio del Sur	SERIE	29	24	82,76%	VIRAL
Futuros Posibles	SERIE	17	14	82,35%	VIRAL
Los Detectives del Café	SERIE	19	13	68,42%	VIRAL
El Precio del Poder	FELICULA	11	11	100%	VIRAL
Amor en Cartagena	FELICULA	10	10	100%	VIRAL
Cafeteros del Quindío	DOCUMENTAL	9	8	88,89%	POPULAR
La Sombra del Diablo	FELICULA	7	7	100%	POPULAR
Galaxia Perdida	FELICULA	7	6	85,71%	POPULAR
Monstruo de las Aguas	FELICULA	5	5	100%	POPULAR
La Última Batalla	FELICULA	5	5	100%	POPULAR
Risas en Familia	FELICULA	5	5	100%	POPULAR
Amazonas: El Último Pulmón	DOCUMENTAL	6	4	66,67%	NORMAL
Los Olvidados del Chocó	DOCUMENTAL	4	4	100%	NORMAL
Mentes Brillantes	DOCUMENTAL	5	4	80%	NORMAL
Barrio Bonito	SERIE	3	3	100%	NORMAL
Corazones Rotos	FELICULA	2	2	100%	NORMAL
La Ruta de la Cocaína	DOCUMENTAL	2	2	100%	NORMAL
Fútbol y Pasión	DOCUMENTAL	3	2	66,67%	NORMAL
Crímenes Reales Colombia	PODCAST	3	1	33,33%	BAJO
Casa Mágica	SERIE	1	1	100%	BAJO
Universo Cuántico	DOCUMENTAL	1	1	100%	BAJO
El Médico Rural	SERIE	1	1	100%	BAJO
Salva Caleña	MUSICA	1	1	100%	BAJO
Cumbia Eterna	MUSICA	1	1	100%	BAJO
Sin Salida	FELICULA	1	1	100%	BAJO

7.2 Procedimientos Almacenados

1. SP_REGISTRAR_USUARIO - Registro completo de nuevo usuario con perfil y primer pago

Código:

```
CREATE OR REPLACE PROCEDURE SP_REGISTRAR_USUARIO (
```

```
    p_nombre      IN VARCHAR2,
```

```
    p_email       IN VARCHAR2,
```

```
p_telefono    IN VARCHAR2,
p_fecha_nac   IN DATE,
p_ciudad      IN VARCHAR2,
p_id_plan     IN NUMBER,
p_metodo_pago  IN VARCHAR2,
p_id_referido  IN NUMBER DEFAULT NULL, -- NULL si no viene
referido
p_id_usuario_out OUT NUMBER          -- ID generado del nuevo
usuario
)
AS
-- Excepciones personalizadas
ex_email_duplicado EXCEPTION;
ex_plan_invalido  EXCEPTION;
ex_mayor_edad     EXCEPTION;
PRAGMA EXCEPTION_INIT(ex_email_duplicado, -20001);
PRAGMA EXCEPTION_INIT(ex_plan_invalido,   -20002);
PRAGMA EXCEPTION_INIT(ex_mayor_edad,     -20003);
v_nuevo_id        NUMBER;
v_nuevo_sus_id    NUMBER;
v_costo_plan      NUMBER;
v_nombre_plan     VARCHAR2(20);
v_email_count     NUMBER;
v_edad            NUMBER;
BEGIN
-- VALIDACIÓN 1: El email no debe existir ya
SELECT COUNT(*) INTO v_email_count
FROM Usuario WHERE UPPER(email) = UPPER(p_email);
IF v_email_count > 0 THEN
```

```
RAISE_APPLICATION_ERROR(-20001,
    'El email ' || p_email || ' ya está registrado en QuindioFlix.');
```

END IF;

-- VALIDACIÓN 2: El plan debe existir

BEGIN

SELECT costo, nombre INTO v_costo_plan, v_nombre_plan

FROM Plan_suscripcion

WHERE id_plan = p_id_plan;

EXCEPTION

 WHEN NO_DATA_FOUND THEN

 RAISE_APPLICATION_ERROR(-20002,

 'El plan con ID ' || p_id_plan || ' no existe.');

END;

-- VALIDACIÓN 3: El titular debe ser mayor de 18 años

v_edad := TRUNC(MONTHS_BETWEEN(SYSDATE, p_fecha_nac) / 12);

IF v_edad < 18 THEN

 RAISE_APPLICATION_ERROR(-20003,

 'El titular debe ser mayor de 18 años. Edad calculada: ' || v_edad);

END IF;

-- PASO 1: Obtener nuevo ID de usuario

SELECT NVL(MAX(id_usuario), 0) + 1 INTO v_nuevo_id FROM Usuario;

-- PASO 2: Crear el usuario

INSERT INTO Usuario (

 id_usuario, nombre, email, telefono,

 fecha_nacimiento, ciudad_residencia,

 Usuario_id_referido, estado_cuenta_activa

) VALUES (

```
v_nuevo_id, p_nombre, p_email, p_telefono,  
  
p_fecha_nac, p_ciudad,  
  
NVL(p_id_referido, v_nuevo_id), -- Si no tiene referido, se  
auto-referencia  
  
'Activa'  
  
);  
  
-- PASO 3: Crear la suscripción  
  
SELECT NVL(MAX(id_suscripcion), 0) + 1 INTO v_nuevo_sus_id FROM  
suscripcion;  
  
INSERT INTO suscripcion (  
  
id_suscripcion, fecha_inicio, fecha_vencimiento,  
  
estado, Usuario_id_usuario, Plan_suscripcion_id_plan  
  
) VALUES (  
  
v_nuevo_sus_id,  
  
SYSDATE,  
  
ADD_MONTHS(SYSDATE, 1), -- Vence en 1 mes  
  
'Activa',  
  
v_nuevo_id,  
  
p_id_plan  
  
);  
  
-- PASO 4: Crear perfil predeterminado tipo Adulto  
  
INSERT INTO Perfil (id_perfil, Usuario_id_usuario, nombre, avatar, tipo)  
VALUES (  
  
(SELECT NVL(MAX(id_perfil), 0) + 1 FROM Perfil),  
  
v_nuevo_id,  
  
p_nombre, -- El perfil principal lleva el nombre del usuario  
  
'avatar_default.png',  
  
'Adulto' -- REGLA: el primer perfil siempre es Adulto  
  
);
```

-- PASO 5: Registrar el primer pago

INSERT INTO Pago (

id_pago, fecha_pago, monto, metodo_pago,

estado_pago, valor_descuento, suscripcion_id_suscripcion

) VALUES (

(SELECT NVL(MAX(id_pago), 0) + 1 FROM Pago),

SYSDATE,

v_costo_plan,

p_metodo_pago,

'Exitoso',

0,

v_nuevo_sus_id

);

p_id_usuario_out := v_nuevo_id;

COMMIT;

DBMS_OUTPUT.PUT_LINE('✓ Usuario registrado exitosamente.');

DBMS_OUTPUT.PUT_LINE(' ID : ' || v_nuevo_id);

DBMS_OUTPUT.PUT_LINE(' Nombre : ' || p_nombre);

DBMS_OUTPUT.PUT_LINE(' Plan : ' || v_nombre_plan || ' (\$' ||
v_costo_plan || '/mes)');

EXCEPTION

WHEN ex_email_duplicado THEN

ROLLBACK;

DBMS_OUTPUT.PUT_LINE('ERROR: ' || SQLERRM);

RAISE;

WHEN ex_plan_invalido THEN

ROLLBACK;

DBMS_OUTPUT.PUT_LINE('ERROR: ' || SQLERRM);

```
RAISE;

WHEN ex_mayor_edad THEN

    ROLLBACK;

    DBMS_OUTPUT.PUT_LINE('ERROR: ' || SQLERRM);

    RAISE;

WHEN OTHERS THEN

    ROLLBACK;

    DBMS_OUTPUT.PUT_LINE('ERROR inesperado en
SP_REGISTRAR_USUARIO: ' || SQLERRM);

    RAISE;

END SP_REGISTRAR_USUARIO;

/

-- Prueba de SP_REGISTRAR_USUARIO

DECLARE

v_nuevo_id NUMBER;

BEGIN

    SP_REGISTRAR_USUARIO(

        p_nombre    => 'Carla Tester',

        p_email      => 'carla.test@email.com',

        p_telefono   => '3005234000',

        p_fecha_nac  => DATE '1999-06-15',

        p_ciudad     => 'Bogotá',

        p_id_plan    => 2,

        p_metodo_pago => 'Nequi',

        p_id_referido => 1,

        p_id_usuario_out => v_nuevo_id

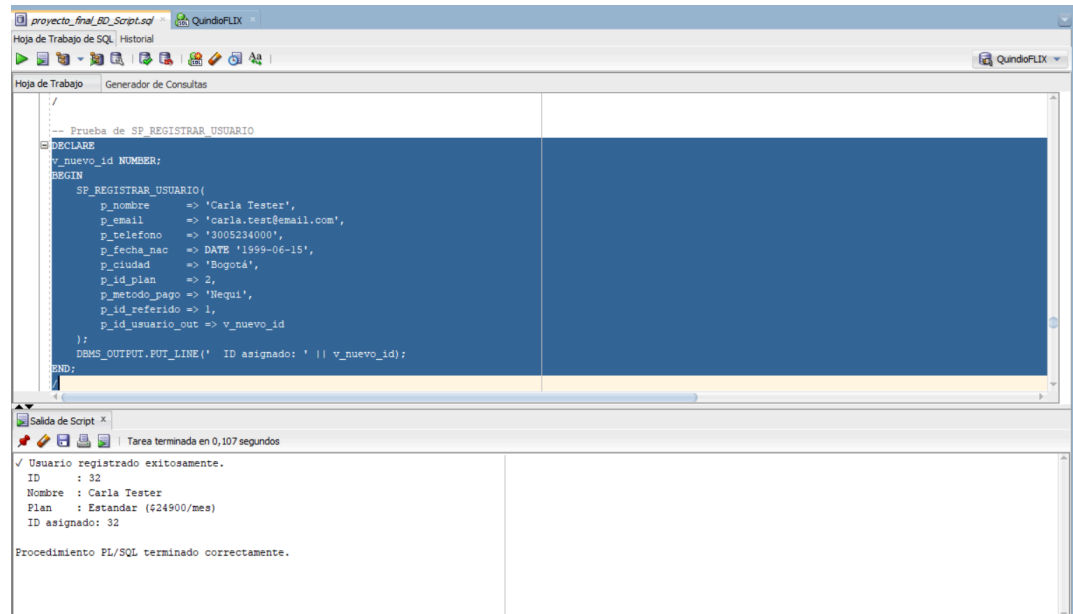
    );

    DBMS_OUTPUT.PUT_LINE(' ID asignado: ' || v_nuevo_id);
```

END;

/

Resultado:



```

-- Prueba de SP REGISTRAR USUARIO
DECLARE
v_nuevo_id NUMBER;
BEGIN
    SP_REGISTRAR_USUARIO(
        p_nombre      => 'Carla Tester',
        p_email       => 'carla.test@email.com',
        p_telefono    => '9005234000',
        p_fecha_nac   => DATE '1999-06-15',
        p_ciudad      => 'Bogotá',
        p_id_plan     => 2,
        p_metodo_pago => 'Nequi',
        p_id_referido => 1,
        p_id_usuario_out => v_nuevo_id
    );
    DBMS_OUTPUT.PUT_LINE(' ID asignado: ' || v_nuevo_id);
END;

```

Salida de Script X
Tarea terminada en 0,107 segundos

✓ Usuario registrado exitosamente.
ID : 32
Nombre : Carla Tester
Plan : Estandar (224900/mes)
ID asignado: 32

Procedimiento PL/SQL terminado correctamente.

2. SP_CAMBIAR_PLAN - Cambio de plan con validación de perfiles activos

Código:

```

CREATE OR REPLACE PROCEDURE SP_CAMBIAR_PLAN (
    p_id_usuario IN NUMBER,
    p_nuevo_plan IN NUMBER
)

```

AS

```

    ex_plan_invalido EXCEPTION;
    ex_perfiles_exceden EXCEPTION;

    PRAGMA EXCEPTION_INIT(ex_plan_invalido, -20010);
    PRAGMA EXCEPTION_INIT(ex_perfiles_exceden, -20011);

    v_plan_actual_id NUMBER;
    v_plan_actual_pant NUMBER;
    v_nuevo_pantallas NUMBER;

```

```
v_nuevo_nombre    VARCHAR2(20);

v_nuevo_costo     NUMBER;

v_perfiles_actuales NUMBER;

v_id_suscripcion  NUMBER;


BEGIN

    -- VALIDACIÓN 1: El nuevo plan existe

    BEGIN

        SELECT num_pantallas, nombre, costo

        INTO v_nuevo_pantallas, v_nuevo_nombre, v_nuevo_costo

        FROM Plan_suscripcion

        WHERE id_plan = p_nuevo_plan;

    EXCEPTION

        WHEN NO_DATA_FOUND THEN

            RAISE_APPLICATION_ERROR(-20010,

                'El plan ID ' || p_nuevo_plan || ' no existe.');
```



```
END;


    -- Obtener suscripción activa del usuario

    SELECT id_suscripcion, Plan_suscripcion_id_plan

    INTO v_id_suscripcion, v_plan_actual_id

    FROM suscripcion

    WHERE Usuario_id_usuario = p_id_usuario

        AND estado = 'Activa'

        AND ROWNUM = 1;


    -- Pantallas del plan actual

    SELECT num_pantallas INTO v_plan_actual_pant
```

```
FROM Plan_suscripcion WHERE id_plan = v_plan_actual_id;
```

```
-- VALIDACIÓN 2: Si baja de plan, no puede tener más perfiles de los permitidos
```

```
SELECT COUNT(*) INTO v_perfiles_actuales
FROM Perfil WHERE Usuario_id_usuario = p_id_usuario;

IF v_nuevo_pantallas < v_plan_actual_pant
    AND v_perfiles_actuales > v_nuevo_pantallas THEN
    RAISE_APPLICATION_ERROR(-20011,
        'No puede bajar al plan ' || v_nuevo_nombre ||
        ' (max ' || v_nuevo_pantallas || ' perfil(es)). ' ||
        'Tiene ' || v_perfiles_actuales || ' perfiles activos. ' ||
        'Elimine ' || (v_perfiles_actuales - v_nuevo_pantallas) ||
        ' perfil(es) primero.');
```

```
END IF;
```

```
-- CAMBIO DE PLAN: Actualizar suscripción
```

```
UPDATE suscripcion
SET Plan_suscripcion_id_plan = p_nuevo_plan,
    fecha_vencimiento      = ADD_MONTHS(SYSDATE, 1)
WHERE id_suscripcion = v_id_suscripcion;
```

```
-- Registrar el nuevo pago por el cambio de plan
```

```
INSERT INTO Pago (
    id_pago, fecha_pago, monto, metodo_pago,
    estado_pago, valor_descuento, suscripcion_id_suscripcion
) VALUES (
```

```
(SELECT NVL(MAX(id_pago), 0) + 1 FROM Pago),  
  
SYSDATE,  
  
v_nuevo_costo,  
  
'Tarjeta crédito',  
  
'Exitoso',  
  
0,  
  
v_id_suscripcion  
  
);  
  
  
COMMIT;  
  
  
DBMS_OUTPUT.PUT_LINE('✓ Plan actualizado correctamente.');
```

```
DBMS_OUTPUT.PUT_LINE(' Usuario ID : ' || p_id_usuario);  
DBMS_OUTPUT.PUT_LINE(' Nuevo plan : ' || v_nuevo_nombre);  
DBMS_OUTPUT.PUT_LINE(' Nuevo costo : $' || v_nuevo_costo || '/mes');
```

```
EXCEPTION  
  
    WHEN ex_plan_invalido THEN  
  
        ROLLBACK;  
  
        DBMS_OUTPUT.PUT_LINE('ERROR: ' || SQLERRM);  
  
        RAISE;  
  
    WHEN ex_perfiles_exceden THEN  
  
        ROLLBACK;  
  
        DBMS_OUTPUT.PUT_LINE('ERROR: ' || SQLERRM);  
  
        RAISE;  
  
    WHEN NO_DATA_FOUND THEN  
  
        ROLLBACK;  
  
        DBMS_OUTPUT.PUT_LINE('ERROR: No se encontró suscripción activa');
```

para el usuario ' || p_id_usuario);

RAISE;

WHEN OTHERS THEN

ROLLBACK;

DBMS_OUTPUT.PUT_LINE('ERROR inesperado en
SP_CAMBIAR_PLAN: ' || SQLERRM);

RAISE;

END SP_CAMBIAR_PLAN;

/

-- Prueba 1 — Caso inválido: bajar de Premium (4 perfiles) a Básico (1 perfil)

-- Usuario 1 tiene 4 perfiles activos, no puede bajar a Básico

BEGIN

SP_CAMBIAR_PLAN(p_id_usuario => 1, p_nuevo_plan => 1);

EXCEPTION

WHEN OTHERS THEN

DBMS_OUTPUT.PUT_LINE('Error esperado capturado: ' || SQLERRM);

END;

/

-- Prueba válida: subir de Estándar a Premium

-- Usuario 30 tiene plan Básico con 1 perfil, puede subir a Premium

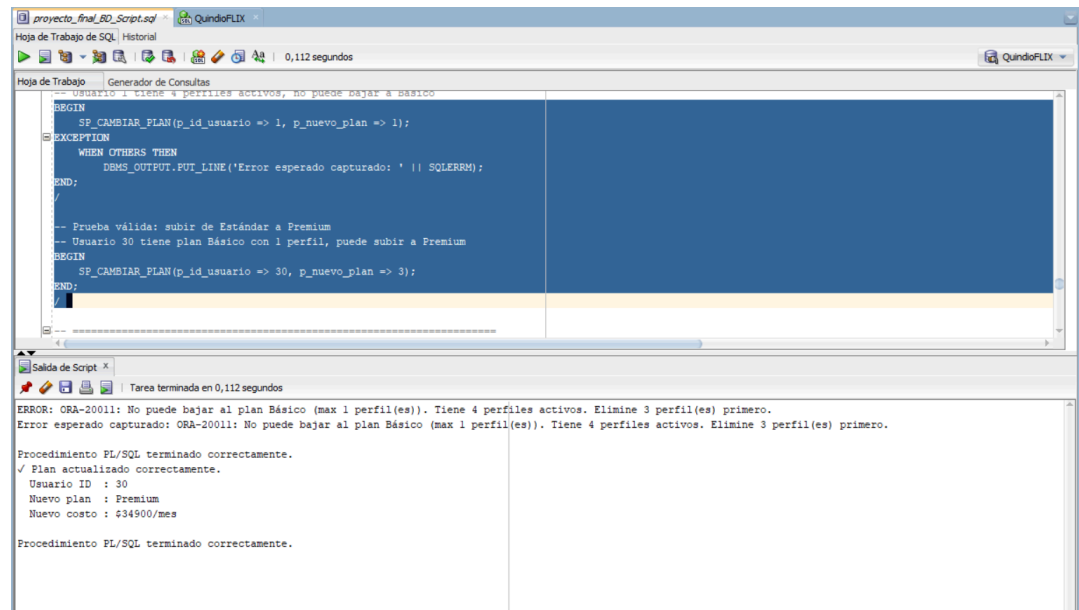
BEGIN

SP_CAMBIAR_PLAN(p_id_usuario => 30, p_nuevo_plan => 3);

END;

/

Resultado:



The screenshot shows a SQL script execution window with the following content:

```

-- Usuario 1 tiene 4 perfiles activos, no puede bajar a básico
BEGIN
  SP_CAMBIAR_PLAN(p_id_usuario => 1, p_nuevo_plan => 1);
EXCEPTION
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Error esperado capturado: ' || SQLERRM);
END;

-- Prueba válida: subir de Estándar a Premium
-- Usuario 30 tiene plan Básico con 1 perfil, puede subir a Premium
BEGIN
  SP_CAMBIAR_PLAN(p_id_usuario => 30, p_nuevo_plan => 3);
END;

```

The output window shows the following messages:

```

Tarea terminada en 0,112 segundos

ERROR: ORA-20011: No puede bajar al plan Básico (max 1 perfil(es)). Tiene 4 perfiles activos. Elimine 3 perfil(es) primero.
Error esperado capturado: ORA-20011: No puede bajar al plan Básico (max 1 perfil(es)). Tiene 4 perfiles activos. Elimine 3 perfil(es) primero.

Procedimiento PL/SQL terminado correctamente.
✓ Plan actualizado correctamente.
Usuario ID : 30
Nuevo plan : Premium
Nuevo costo : $34900/mes

Procedimiento PL/SQL terminado correctamente.

```

3. SP_REPORTE_CONSUMO - Reporte detallado de reproducciones por usuario y rango de fechas

Código:

```

CREATE OR REPLACE PROCEDURE SP_REPORTE_CONSUMO (

  p_id_usuario IN NUMBER,

  p_fecha_inicio IN DATE,

  p_fecha_fin IN DATE

)

AS

  CURSOR cur_consumo IS

SELECT

  p.nombre AS perfil,

  p.tipo AS tipo_perfil,

  c.tipo_contenido AS categoria,

  c.titulo,

  r.dispositivo,

  -- Duración en minutos de la reproducción

```

```
ROUND(
    (CAST(r.fecha_fin AS DATE) -
    CAST(r.fecha_inicio AS DATE)) * 1440,
    2) AS minutos,
    r.porcentaje_avance,
    CAST(r.fecha_inicio AS DATE) AS fecha
FROM Reproduccion r
    JOIN Perfil    p  ON r.Perfil_id_perfil    = p.id_perfil
    JOIN contenido  c  ON r.contenido_id_contenido = c.id_contenido
WHERE p.Usuario_id_usuario = p_id_usuario
    AND CAST(r.fecha_inicio AS DATE)
    BETWEEN p_fecha_inicio AND p_fecha_fin
ORDER BY p.nombre, c.tipo_contenido, r.fecha_inicio;
v_perfil_actual  VARCHAR2(50) := '';
v_cat_actual     VARCHAR2(20) := '';
v_min_perfil     NUMBER := 0;
v_min_categoria  NUMBER := 0;
v_min_total      NUMBER := 0;
v_titulos_cat    NUMBER := 0;
v_nombre_usuario VARCHAR2(50);
BEGIN
    -- Obtener nombre del usuario
    SELECT nombre
    INTO v_nombre_usuario
    FROM Usuario
    WHERE id_usuario = p_id_usuario;
    DBMS_OUTPUT.PUT_LINE('=====
=====');
```

```
DBMS_OUTPUT.PUT_LINE(' REPORTE DE CONSUMO - QuindioFlix');

DBMS_OUTPUT.PUT_LINE(' Usuario : ' || v_nombre_usuario ||

                        '(ID: ' || p_id_usuario || ')');


DBMS_OUTPUT.PUT_LINE(' Período : ' ||

                        TO_CHAR(p_fecha_inicio,'DD/MM/YYYY') ||

                        ' al ' ||

                        TO_CHAR(p_fecha_fin,'DD/MM/YYYY'));

DBMS_OUTPUT.PUT_LINE('=====
=====');

FOR v_reg IN cur_consumo LOOP

    -- Cambio de perfil

    IF v_reg.perfil <> v_perfil_actual THEN

        -- Cerrar categoría anterior

        IF v_cat_actual <> " THEN

            DBMS_OUTPUT.PUT_LINE(

                '    └ Subtotal ' || v_cat_actual ||

                ': ' || v_titulos_cat || ' títulos | ' ||

                ROUND(v_min_categoria / 60, 1) || ' hrs'

            );

        END IF;

        -- Cerrar perfil anterior

        IF v_perfil_actual <> " THEN

            DBMS_OUTPUT.PUT_LINE(

                ' TOTAL PERFIL ' || v_perfil_actual ||

                ': ' || ROUND(v_min_perfil / 60, 1) || ' hrs'

            );

            DBMS_OUTPUT.PUT_LINE(
```

```
' ' || RPAD('-', 55, '-')  
  
);  
  
END IF;  
  
v_perfil_actual := v_reg.perfil;  
v_cat_actual := '';  
v_min_perfil := 0;  
DBMS_OUTPUT.PUT_LINE(  
    '► PERFIL: ' || v_reg.perfil ||  
    ' [' || v_reg.tipo_perfil || ']'  
);  
  
END IF;  
  
  
-- Cambio de categoría  
IF v_reg.categoria <> v_cat_actual THEN  
  
    IF v_cat_actual <> '' THEN  
        DBMS_OUTPUT.PUT_LINE(  
            '└─ Subtotal ' || v_cat_actual ||  
            ': ' || v_titulos_cat || ' títulos | ' ||  
            ROUND(v_min_categoria / 60, 1) || ' hrs'  
        );  
    END IF;  
  
    v_cat_actual := v_reg.categoria;  
    v_min_categoria := 0;  
    v_titulos_cat := 0;  
  
    DBMS_OUTPUT.PUT_LINE(  

```

```
' > Categoría: ' || v_reg.categoria

);

END IF;

-- Acumuladores

v_titulos_cat := v_titulos_cat + 1;

v_min_categoria := v_min_categoria + NVL(v_reg.minutos, 0);

v_min_perfil := v_min_perfil + NVL(v_reg.minutos, 0);

v_min_total := v_min_total + NVL(v_reg.minutos, 0);

-- Detalle

DBMS_OUTPUT.PUT_LINE(

' • ' ||

RPAD(SUBSTR(v_reg.titulo, 1, 30), 32) ||

' | ' || v_reg.dispositivo ||

' | ' || v_reg.porcentaje_avance || '%' ||

' | ' || TO_CHAR(v_reg.fecha, 'DD/MM')

);

END LOOP;

-- Cerrar última categoría

IF v_cat_actual <> " THEN

DBMS_OUTPUT.PUT_LINE(

' └─ Subtotal ' || v_cat_actual ||

': ' || v_titulos_cat || ' títulos | ' ||

ROUND(v_min_categoria / 60, 1) || ' hrs'

);

END IF;

-- Cerrar último perfil

IF v_perfil_actual <> " THEN

DBMS_OUTPUT.PUT_LINE(
```

```
' TOTAL PERFIL ' || v_perfil_actual ||  
' : ' || ROUND(v_min_perfil / 60, 1) || ' hrs'  
  
);  
  
END IF;  
  
DBMS_OUTPUT.PUT_LINE('=====')  
=====');  
  
DBMS_OUTPUT.PUT_LINE(  
    'TIEMPO TOTAL CONSUMIDO: ' ||  
    ROUND(v_min_total / 60, 1) || ' horas'  
  
);  
  
DBMS_OUTPUT.PUT_LINE('=====')  
=====');  
  
EXCEPTION  
  
    WHEN NO_DATA_FOUND THEN  
  
        DBMS_OUTPUT.PUT_LINE(  
            'No se encontró el usuario con ID: ' || p_id_usuario  
  
        );  
  
    WHEN OTHERS THEN  
  
        DBMS_OUTPUT.PUT_LINE(  
            'ERROR en SP_REPORTE_CONSUMO: ' || SQLERRM  
  
        );  
  
        RAISE;  
  
END SP_REPORTE_CONSUMO;  
  
/  
  
-- Prueba SP_REPORTE_CONSUMO  
  
BEGIN  
  
    SP_REPORTE_CONSUMO(  
        p_id_usuario => 1,
```

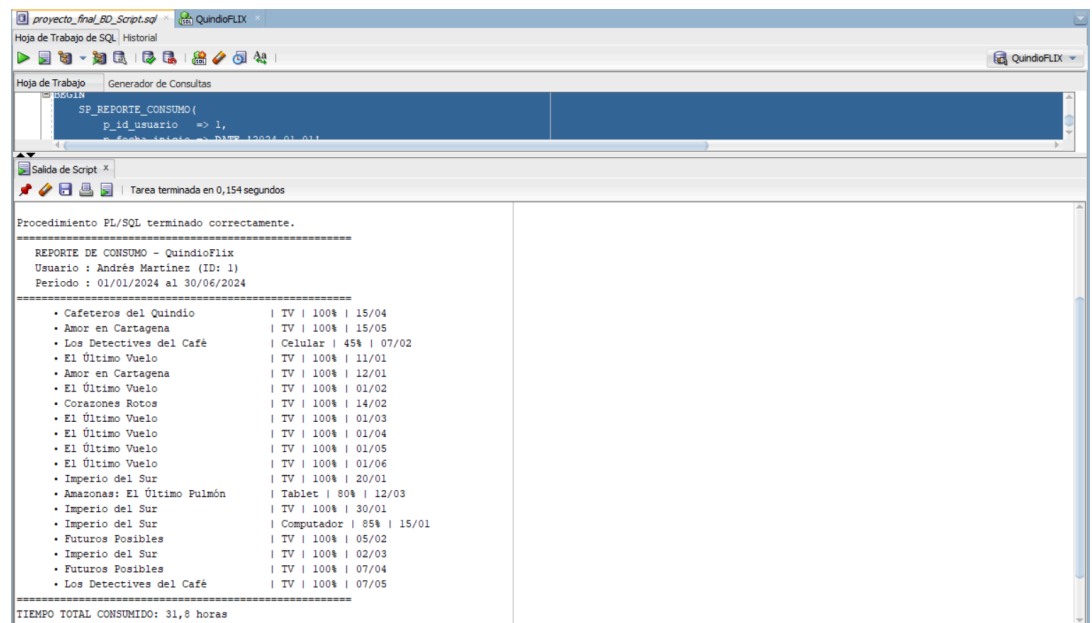
```
p_fecha_inicio => DATE '2024-01-01',
p_fecha_fin    => DATE '2024-06-30'

);

END;

/
```

Resultado:



7.3 Funciones

1. FN_CALCULAR_MONTO - Cálculo del monto mensual con descuento por antigüedad

Código:

```
CREATE OR REPLACE FUNCTION FN_CALCULAR_MONTO (
```

```
    p_id_usuario IN NUMBER
```

```
) RETURN NUMBER
```

```
AS
```

```
    v_costo_plan  NUMBER;
```

```
    v_meses       NUMBER;
```

```
    v_descuento   NUMBER := 0;
```

```
    v_monto_final NUMBER;
```

```
v_nombre_plan VARCHAR2(20);

BEGIN

    -- Obtener plan actual y fecha de inicio de suscripción

    SELECT ps.costo, ps.nombre,
           TRUNC(MONTHS_BETWEEN(SYSDATE, s.fecha_inicio))
    INTO v_costo_plan, v_nombre_plan, v_meses
    FROM suscripcion s
           JOIN Plan_suscripcion ps ON s.Plan_suscripcion_id_plan = ps.id_plan
    WHERE s.Usuario_id_usuario = p_id_usuario
           AND s.estado = 'Activa'
           AND ROWNUM = 1;

    -- Aplicar descuento por antigüedad
    IF v_meses > 24 THEN
        v_descuento := 0.15; -- 15% para más de 24 meses
    ELSIF v_meses > 12 THEN
        v_descuento := 0.10; -- 10% para más de 12 meses
    ELSE
        v_descuento := 0; -- Sin descuento para nuevos usuarios
    END IF;

    v_monto_final := ROUND(v_costo_plan * (1 - v_descuento), 2);

    DBMS_OUTPUT.PUT_LINE('Plan: ' || v_nombre_plan ||
                          ' | Antigüedad: ' || v_meses || ' meses' ||
                          ' | Descuento: ' || (v_descuento * 100) || '%' ||
                          ' | Monto: $' || v_monto_final);

    RETURN v_monto_final;

EXCEPTION

    WHEN NO_DATA_FOUND THEN

        DBMS_OUTPUT.PUT_LINE('No hay suscripción activa para el usuario ')
```

```

|| p_id_usuario);

RETURN NULL;

WHEN OTHERS THEN

        DBMS_OUTPUT.PUT_LINE('ERROR en FN_CALCULAR_MONTO: ' ||
SQLERRM);

RETURN NULL;

END FN_CALCULAR_MONTO;

/

-- Prueba FN_CALCULAR_MONTO

SELECT

        u.id_usuario,

        u.nombre,

        FN_CALCULAR_MONTO(u.id_usuario) AS monto_proximo_mes

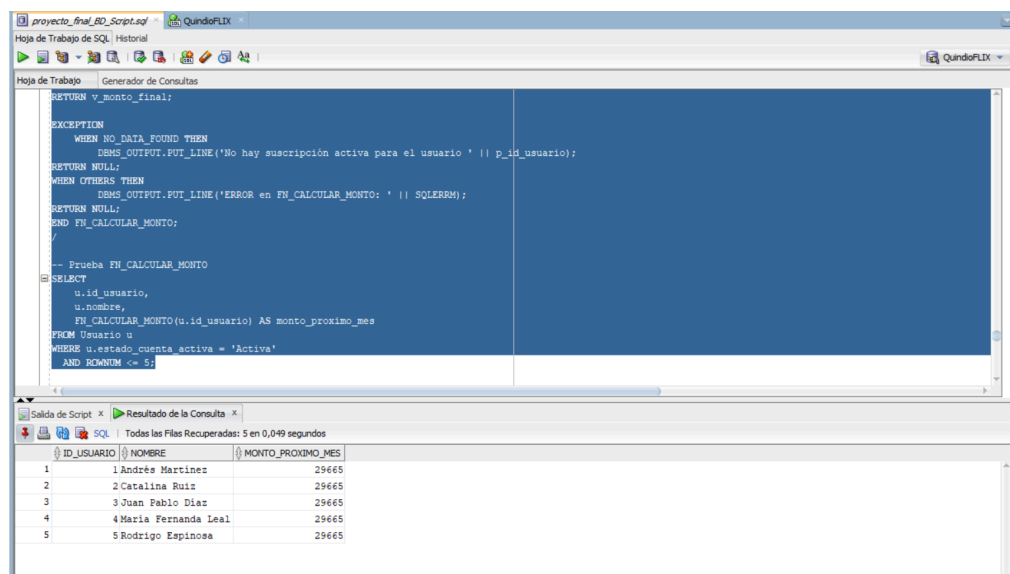
FROM Usuario u

WHERE u.estado_cuenta_activa = 'Activa'

AND ROWNUM <= 5;

```

Resultado:



The screenshot shows the QuindioFLX SQL editor with the query executed. The results are displayed in a table with 5 rows and 3 columns: ID_USUARIO, NOMBRE, and MONTO_PROXIMO_MES.

ID_USUARIO	NOMBRE	MONTO_PROXIMO_MES
1	Andrés Martínez	29665
2	Catalina Ruiz	29665
3	Juan Pablo Díaz	29665
4	Maria Fernanda Leal	29665
5	Rodrigo Espinosa	29665

2. FN_CONTENIDO_RECOMENDADO - Recomendación de contenido según géneros favoritos del perfil

Código:

```
CREATE OR REPLACE FUNCTION FN_CONTENIDO_RECOMENDADO (  
    p_id_perfil IN NUMBER  
) RETURN VARCHAR2  
AS  
    v_titulo_recomendado VARCHAR2(200);  
    v_tipo_perfil      VARCHAR2(20);  
  
BEGIN  
    SELECT tipo INTO v_tipo_perfil  
    FROM Perfil WHERE id_perfil = p_id_perfil;  
  
    SELECT titulo INTO v_titulo_recomendado  
    FROM (  
        SELECT  
            c.id_contenido,  
            c.titulo,  
            COUNT(tg.genero_id_genero) AS coincidencias_genero  
        FROM contenido c  
            JOIN tipo_genero tg ON c.id_contenido = tg.contenido_id_contenido  
        WHERE tg.genero_id_genero IN (  
            SELECT genero_id_genero  
            FROM (  
                SELECT  
                    tg2.genero_id_genero,  
                    COUNT(*) total  
                FROM Reproduccion r2  
                    JOIN contenido c2 ON r2.contenido_id_contenido =  
c2.id_contenido  
                    JOIN tipo_genero tg2 ON c2.id_contenido =  
tg2.contenido_id_contenido
```

```
WHERE r2.Perfil_id_perfil = p_id_perfil

GROUP BY tg2.genero_id_genero

ORDER BY total DESC

)

WHERE ROWNUM <= 3

)

AND c.id_contenido NOT IN (

    SELECT r3.contenido_id_contenido

    FROM Reproduccion r3

    WHERE r3.Perfil_id_perfil = p_id_perfil

)

AND (

    v_tipo_perfil = 'Adulto'

    OR c.clasificacion_edad IN ('TP', '+7', '+13')

)

GROUP BY c.id_contenido, c.titulo

ORDER BY coincidencias_genero DESC, DBMS_RANDOM.VALUE

)

WHERE ROWNUM = 1;

RETURN v_titulo_recomendado;

EXCEPTION

WHEN NO_DATA_FOUND THEN

BEGIN

    SELECT titulo INTO v_titulo_recomendado

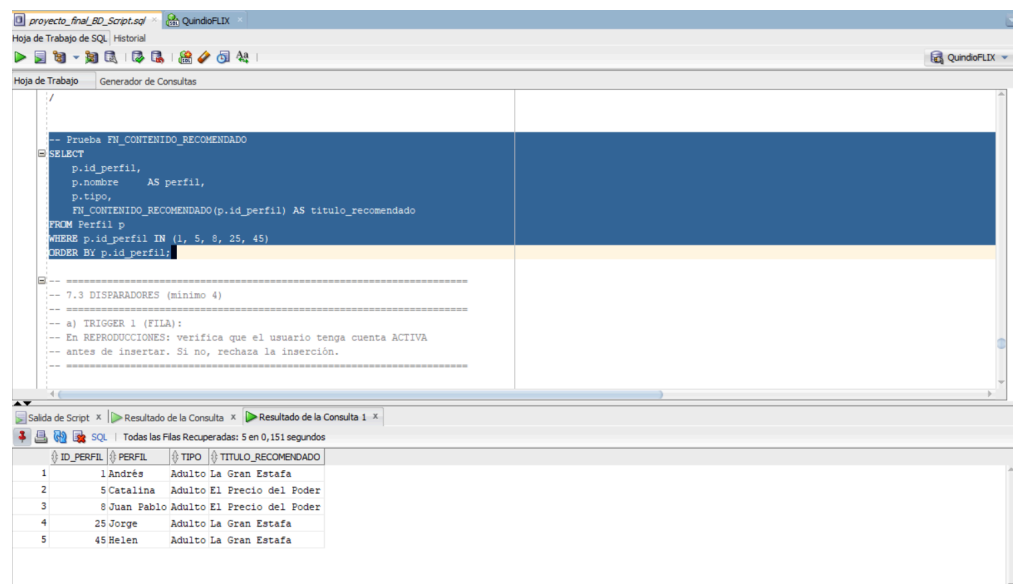
    FROM (

        SELECT titulo, score_popularidad

        FROM MV_POPULARIDAD_CONTENIDO
```

```
WHERE id_contenido NOT IN (  
    SELECT contenido_id_contenido  
    FROM Reproduccion  
    WHERE Perfil_id_perfil = p_id_perfil  
)  
ORDER BY score_popularidad DESC  
)  
WHERE ROWNUM = 1;  
RETURN v_titulo_recomendado;  
EXCEPTION  
    WHEN OTHERS THEN  
        RETURN 'Imperio del Sur';  
END;  
    WHEN OTHERS THEN  
        DBMS_OUTPUT.PUT_LINE('ERROR: ' || SQLERRM);  
        RETURN NULL;  
END FN_CONTENIDO_RECOMENDADO;  
/  
-- Prueba FN_CONTENIDO_RECOMENDADO  
SELECT  
    p.id_perfil,  
    p.nombre    AS perfil,  
    p.tipo,  
    FN_CONTENIDO_RECOMENDADO(p.id_perfil) AS titulo_recomendado  
FROM Perfil p  
WHERE p.id_perfil IN (1, 5, 8, 25, 45)  
ORDER BY p.id_perfil;
```

Resultado:



7.4 Disparadores

1. TRG_REPROD_CUENTA_ACTIVA - Bloquea reproducciones si la cuenta no está activa

Código:

```
CREATE OR REPLACE TRIGGER TRG_REPROD_CUENTA_ACTIVA
```

```
BEFORE INSERT ON Reproduccion
```

```
FOR EACH ROW
```

```
DECLARE
```

```
v_estado VARCHAR2(12);
```

```
BEGIN
```

```
-- Buscar estado de la cuenta del usuario dueño del perfil
```

```
SELECT u.estado_cuenta_activa INTO v_estado
```

```
FROM Usuario u
```

```
JOIN Perfil p ON u.id_usuario = p.Usuario_id_usuario
```

```
WHERE p.id_perfil = :NEW.Perfil_id_perfil;
```

```
IF v_estado <> 'Activa' THEN
```

```
RAISE_APPLICATION_ERROR(-20100,
```

```
'Reproducción rechazada: La cuenta del usuario no está ACTIVA ' ||
```

```
'(estado actual: ' || v_estado || '). ' ||
```

```
'Regularice su pago para continuar.');
```

```
END IF;
```

```
EXCEPTION
```

```
    WHEN NO_DATA_FOUND THEN
```

```
        RAISE_APPLICATION_ERROR(-20101,
```

```
            'Reproducción rechazada: No se encontró el perfil con ID ' ||
```

```
            :NEW.Perfil_id_perfil);
```

```
    WHEN OTHERS THEN
```

```
        RAISE;
```

```
END TRG_REPROD_CUENTA_ACTIVA;
```

```
/
```

```
-- Prueba TRG_REPROD_CUENTA_ACTIVA (perfil 17 pertenece a usuario 7,
```

```
INACTIVA)
```

```
BEGIN
```

```
    INSERT INTO Reproduccion VALUES (
```

```
        TIMESTAMP '2026-05-01 20:00:00', TIMESTAMP
```

```
'2026-05-01 22:00:00',
```

```
        'TV', 9999, 50, 17, 1, 36
```

```
    );
```

```
EXCEPTION
```

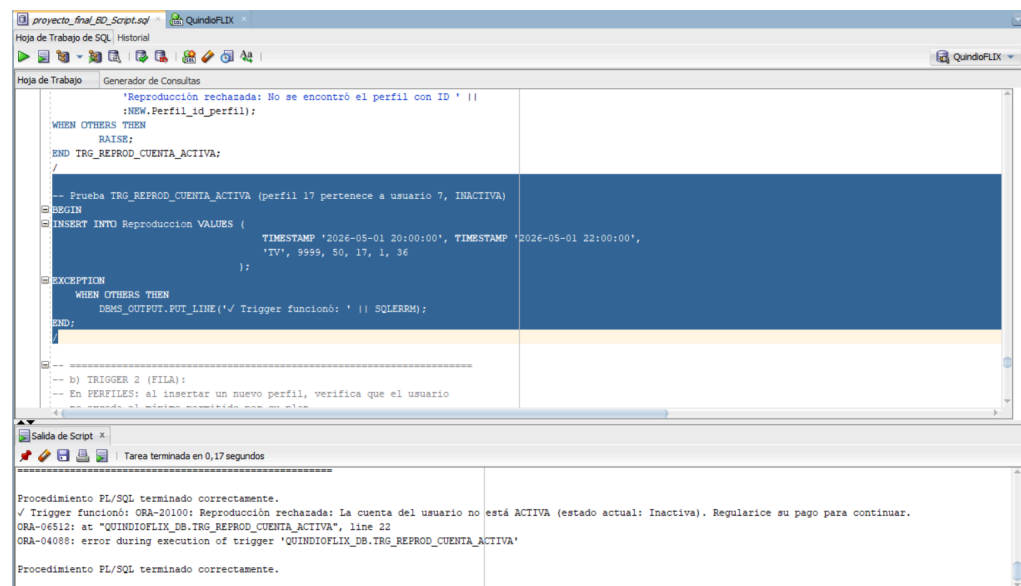
```
    WHEN OTHERS THEN
```

```
        DBMS_OUTPUT.PUT_LINE('✓ Trigger funcionó: ' || SQLERRM);
```

```
END;
```

```
/
```

Resultado:



```

-- Reproducción rechazada: No se encontró el perfil con ID ' ||
-- :NEW.Perfil_id_perfil);
WHEN OTHERS THEN
  RAISE;
END TRG_REPROD_CUENTA_ACTIVA;

-- Prueba TRG_REPROD_CUENTA_ACTIVA (perfil 17 pertenece a usuario 7, INACTIVA)
BEGIN
  INSERT INTO Reproduccion VALUES (
    TIMESTAMP '2026-05-01 20:00:00', TIMESTAMP '2026-05-01 22:00:00',
    'TV', 9999, 50, 17, 1, 36
  );
EXCEPTION
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Trigger funcionó: ' || SQLERRM);
END;

-- b) TRIGGER 2 (FILA):
-- En PERFILES: al insertar un nuevo perfil, verifica que el usuario
-- no exceda el número permitido por su plan

```

Salida de Script

Tarea terminada en 0,17 segundos

Procedimiento PL/SQL terminado correctamente.
 ✓ Trigger funcionó: ORA-20100: Reproducción rechazada: La cuenta del usuario no está ACTIVA (estado actual: Inactiva). Regularice su pago para continuar.
 ORA-06512: at "QUINDIOFLIX_DB.TRG_REPROD_CUENTA_ACTIVA", line 22
 ORA-04088: error during execution of trigger "QUINDIOFLIX_DB.TRG_REPROD_CUENTA_ACTIVA"
 Procedimiento PL/SQL terminado correctamente.

2. TRG_PERFIL_LIMITE_PLAN - Bloquea creación de perfiles que superen el límite del plan

Código:

CREATE OR REPLACE TRIGGER TRG_PERFIL_LIMITE_PLAN

BEFORE INSERT ON Perfil

FOR EACH ROW

DECLARE

v_max_pantallas NUMBER;

v_perfiles_act NUMBER;

v_nombre_plan VARCHAR2(20);

BEGIN

-- Obtener límite de pantallas del plan activo del usuario

SELECT ps.num_pantallas, ps.nombre

INTO v_max_pantallas, v_nombre_plan

FROM suscripcion s

JOIN Plan_suscripcion ps ON s.Plan_suscripcion_id_plan = ps.id_plan

WHERE s.Usuario_id_usuario = :NEW.Usuario_id_usuario

AND s.estado = 'Activa'

AND ROWNUM = 1;

```
-- Contar perfiles actuales del usuario

SELECT COUNT(*) INTO v_perfiles_act

FROM Perfil

WHERE Usuario_id_usuario = :NEW.Usuario_id_usuario;

IF v_perfiles_act >= v_max_pantallas THEN

    RAISE_APPLICATION_ERROR(-20200,

        'No se puede crear el perfil. ' ||

        'El plan ' || v_nombre_plan ||

        ' permite máximo ' || v_max_pantallas || ' perfil(es). ' ||

        'Actualmente tiene ' || v_perfiles_act || ' perfil(es).');

END IF;

EXCEPTION

    WHEN NO_DATA_FOUND THEN

        RAISE_APPLICATION_ERROR(-20201,

            'No se encontró suscripción activa para el usuario ' ||

:NEW.Usuario_id_usuario);

    WHEN OTHERS THEN

        RAISE;

END TRG_PERFIL_LIMITE_PLAN;

/

--Prueba

BEGIN

INSERT INTO Perfil (id_perfil, Usuario_id_usuario, nombre, avatar, tipo)

VALUES (9998, 22, 'Perfil Extra', 'avatar_x.png', 'Adulto');

EXCEPTION

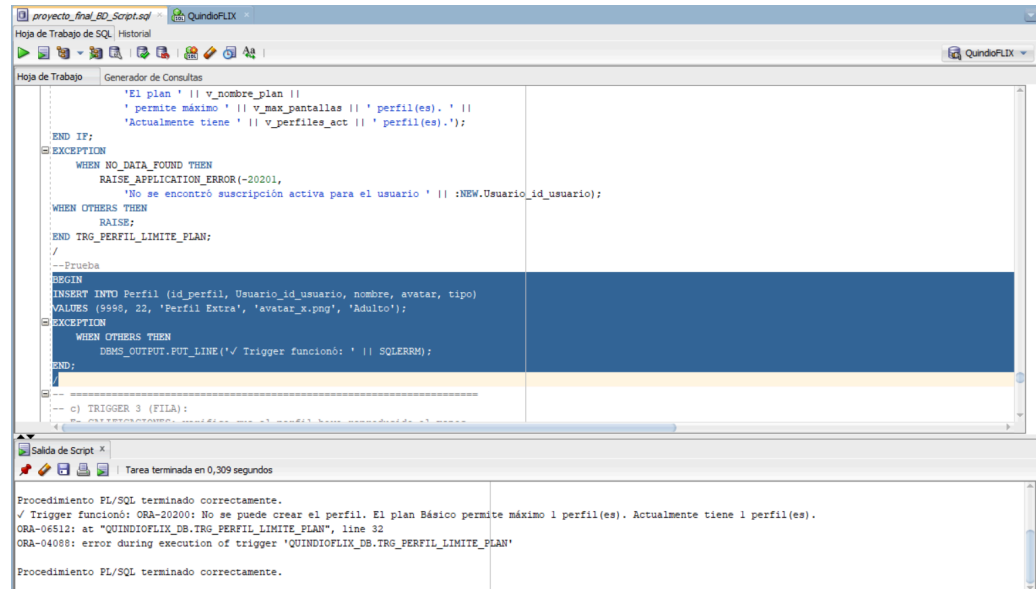
    WHEN OTHERS THEN

        DBMS_OUTPUT.PUT_LINE('✓ Trigger funcionó: ' || SQLERRM);
```

END;

/

Resultado:



The screenshot shows the QuindioFLIX SQL editor with a PL/SQL script. The script includes an IF statement, an EXCEPTION block for NO_DATA_FOUND, and an INSERT statement. The output window at the bottom shows the execution results, including a message about the trigger function and an error message for the trigger execution.

```

-- El plan ' || v_nombre_plan ||
-- permite máximo ' || v_max_pantallas || ' perfil(es). ' ||
-- Actualmente tiene ' || v_perfiles_act || ' perfil(es).';
END IF;
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    RAISE_APPLICATION_ERROR(-20201,
      'No se encontró suscripción activa para el usuario ' || :NEW.Usuario_id_usuario);
  WHEN OTHERS THEN
    RAISE;
END TRG_PERFIL_LIMITE_PLAN;
/
-- Prueba
BEGIN
  INSERT INTO Perfil (id_perfil, Usuario_id_usuario, nombre, avatar, tipo)
  VALUES (9998, 22, 'Perfil Extra', 'avatar_x.png', 'Adulto');
EXCEPTION
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('✓ Trigger funcionó: ' || SQLERRM);
END;
-- c) TRIGGER 3 (FILA):
-- 3. CALIFICACIONES: modifica que el perfil tiene capacidad de 3 planes

```

Salida de Script X
Tarea terminada en 0,309 segundos

Procedimiento PL/SQL terminado correctamente.
✓ Trigger funcionó: ORA-20201: No se puede crear el perfil. El plan Básico permite máximo 1 perfil(es). Actualmente tiene 1 perfil(es).
ORA-06512: at "QUINDIOFLIX_DB.TRG_PERFIL_LIMITE_PLAN", line 32
ORA-04088: error during execution of trigger 'QUINDIOFLIX_DB.TRG_PERFIL_LIMITE_PLAN'
Procedimiento PL/SQL terminado correctamente.

3. TRG_CALIF_REQUIERE_AVANCE - Bloquea calificaciones sin al menos 50% de avance

Código:

CREATE OR REPLACE TRIGGER TRG_CALIF_REQUIERE_AVANCE

BEFORE INSERT ON Calificacion

FOR EACH ROW

DECLARE

v_max_avance NUMBER;

BEGIN

-- Buscar el mayor porcentaje de avance del perfil en ese contenido

SELECT NVL(MAX(porcentaje_avance), 0) INTO v_max_avance

FROM Reproduccion

WHERE Perfil_id_perfil = :NEW.Perfil_id_perfil

AND contenido_id_contenido = :NEW.contenido_id_contenido;

```
IF v_max_avance < 50 THEN
    RAISE_APPLICATION_ERROR(-20300,
        'No se puede calificar el contenido. ' ||
        'Debe haber reproducido al menos el 50% para opinar. ' ||
        'Su avance máximo registrado es: ' || v_max_avance || '%.');
```

END IF;

END TRG_CALIF_REQUIERE_AVANCE;

/

-- Prueba TRG_CALIF_REQUIERE_AVANCE (perfil 46 con contenido 13,
avance insuficiente)

BEGIN

INSERT INTO Calificacion (id_calificacion, estrellas, reseña, Perfil_id_perfil,
contenido_id_contenido)

VALUES (9999, 5, 'Intento calificar sin ver suficiente', 46, 5);

EXCEPTION

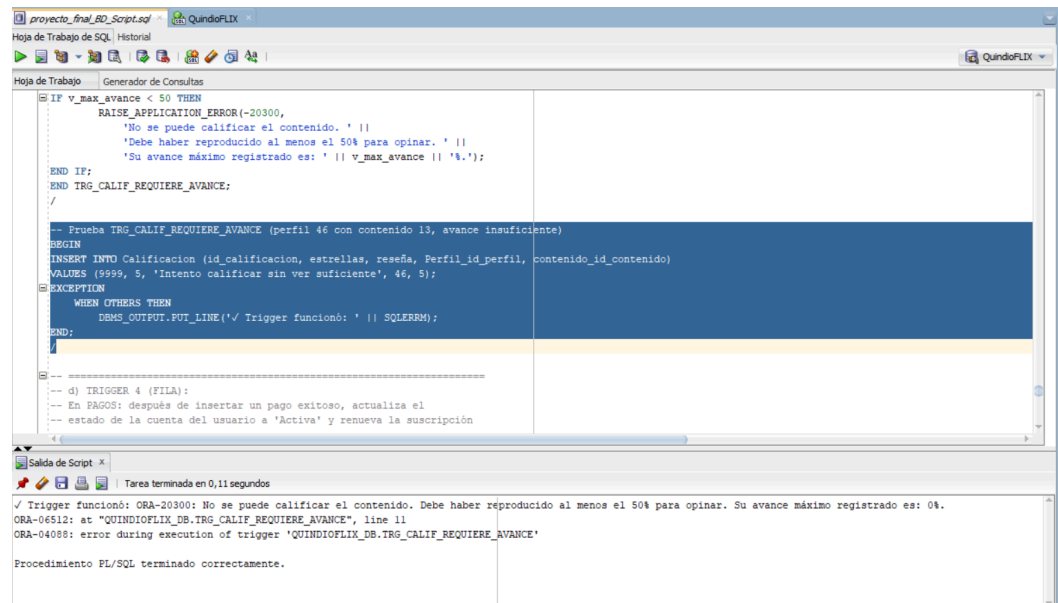
WHEN OTHERS THEN

DBMS_OUTPUT.PUT_LINE('✓ Trigger funcionó: ' || SQLERRM);

END;

/

Resultado:



```

proyecto_final_RQ_Script.sql  QuindioFLIX
Hoja de Trabajo de SQL Historial
Hoja de Trabajo  Generador de Consultas

IF v_max_avance < 50 THEN
    RAISE_APPLICATION_ERROR(-20300,
        'No se puede calificar el contenido. ' ||
        'Debe haber reproducido al menos el 50% para opinar. ' ||
        'Su avance máximo registrado es: ' || v_max_avance || '%');
END IF;
END TRG_CALIF_REQUIERE_AVANCE;

-- Prueba TRG_CALIF_REQUIERE_AVANCE (perfil 46 con contenido 13, avance insuficiente)
BEGIN
    INSERT INTO Calificacion (id_calificacion, estrellas, reseña, Perfil_id_perfil, contenido_id_contenido)
    VALUES (9999, 5, 'Intento calificar sin ver suficiente', 46, 5);
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('✓ Trigger función: ' || SQLERRM);
END;

-- d) TRIGGER 4 (FILA):
-- En PAGOS: después de insertar un pago exitoso, actualiza el
-- estado de la cuenta del usuario a 'Activa' y renueva la suscripción

Salida de Script
Tarea terminada en 0,11 segundos

✓ Trigger función: ORA-20300: No se puede calificar el contenido. Debe haber reproducido al menos el 50% para opinar. Su avance máximo registrado es: 0%.
ORA-06512: at "QUINDIOFLIX_DB.TRG_CALIF_REQUIERE_AVANCE", line 11
ORA-04088: error during execution of trigger "QUINDIOFLIX_DB.TRG_CALIF_REQUIERE_AVANCE"
Procedimiento PL/SQL terminado correctamente.
    
```

4. TRG_PAGO_ACTIVAR_CUENTA - Activa la cuenta automáticamente al registrar pago exitoso

Código:

CREATE OR REPLACE TRIGGER TRG_PAGO_ACTIVAR_CUENTA

AFTER INSERT ON Pago

FOR EACH ROW

BEGIN

-- Solo actuar si el pago fue exitoso

IF :NEW.estado_pago = 'Exitoso' THEN

-- Activar la cuenta del usuario asociado a la suscripción

UPDATE Usuario

SET estado_cuenta_activa = 'Activa'

WHERE id_usuario = (

SELECT Usuario_id_usuario

FROM suscripcion

WHERE id_suscripcion = :NEW.suscripcion_id_suscripcion

);

```
-- Reactivar y renovar la suscripción por 1 mes

UPDATE suscripcion

SET estado = 'Activa',

    fecha_vencimiento = ADD_MONTHS(SYSDATE, 1)

WHERE id_suscripcion = :NEW.suscripcion_id_suscripcion;

END IF;

END TRG_PAGO_ACTIVAR_CUENTA;

-- PRUEBA TRG_PAGO_ACTIVAR_CUENTA

-- Usuario 7 tiene cuenta inactiva y realizará un pago exitoso

DECLARE

v_estado VARCHAR2(20);

BEGIN

    -- Insertar pago exitoso

INSERT INTO Pago (

    id_pago,

    fecha_pago,

    monto,

    metodo_pago,

    estado_pago,

    valor_descuento,

    suscripcion_id_suscripcion

)

VALUES (

    (SELECT NVL(MAX(id_pago),0)+1 FROM Pago),

    SYSDATE,

    34900,

    'Nequi',

    'Exitoso',
```

```

0,
7
);

-- Verificar si el trigger activó la cuenta

SELECT estado_cuenta_activa
INTO v_estado
FROM Usuario
WHERE id_usuario = 7;

DBMS_OUTPUT.PUT_LINE(
    '✓ Trigger funcionó correctamente. Estado actual: ' || v_estado
);

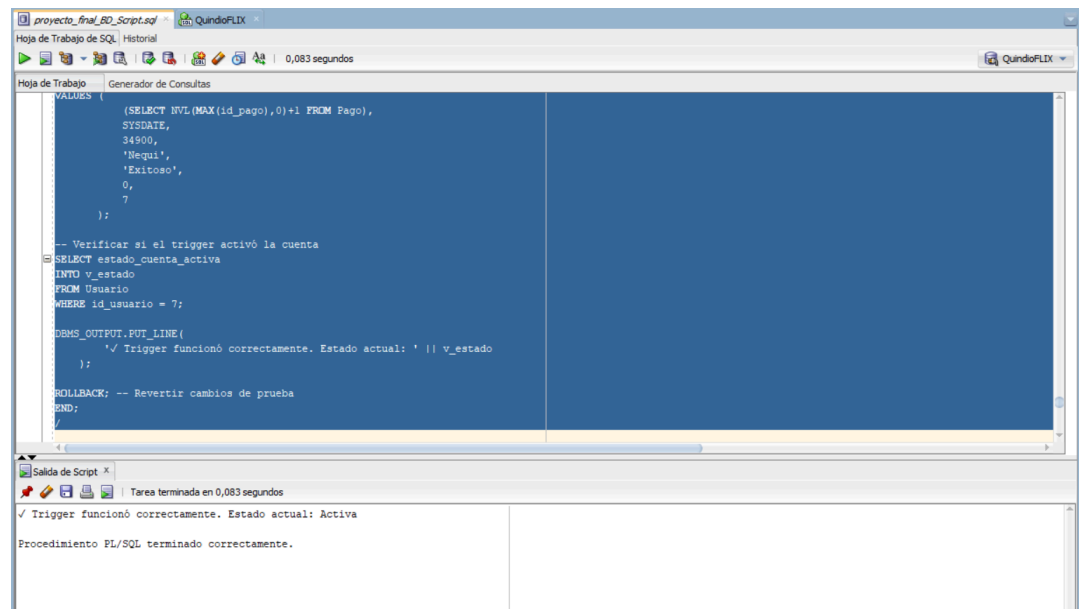
ROLLBACK; -- Revertir cambios de prueba

END;

/

```

Resultado:



The screenshot shows a SQL Developer window titled 'proyecto_final_BO_Script.sql'. The main editor displays the following PL/SQL code:

```

VALUES (
    (SELECT NVL(MAX(id_pago),0)+1 FROM Pago),
    SYSDATE,
    34900,
    'Nequi',
    'Exitoso',
    0,
    7
);

-- Verificar si el trigger activó la cuenta
SELECT estado_cuenta_activa
INTO v_estado
FROM Usuario
WHERE id_usuario = 7;

DBMS_OUTPUT.PUT_LINE(
    '✓ Trigger funcionó correctamente. Estado actual: ' || v_estado
);

ROLLBACK; -- Revertir cambios de prueba

END;

/

```

The bottom panel shows the output of the script execution:

```

Salida de Script x
Tarea terminada en 0,083 segundos
✓ Trigger funcionó correctamente. Estado actual: Activa
Procedimiento PL/SQL terminado correctamente.

```

7.5 Transacciones

1. Transacción 1 - Registro completo de usuario con estados ACTIVA, PARCIALMENTE CONFIRMADA y CONFIRMADA

Código:

BEGIN

-- — ESTADO: ACTIVA

DBMS_OUTPUT.PUT_LINE('=== TRANSACCIÓN 1: REGISTRO
COMPLETO ===');

DBMS_OUTPUT.PUT_LINE('[ACTIVA] Iniciando transacción...');

-- PASO 1: Insertar el usuario

INSERT INTO Usuario (

id_usuario, nombre, email, telefono,

fecha_nacimiento, ciudad_residencia,

Usuario_id_referido, estado_cuenta_activa

) VALUES (

100, 'Ana Prueba T1', 'ana.t1@email.com',

'3001110001', DATE '1998-03-10', 'Cali', 100, 'Activa'

);

DBMS_OUTPUT.PUT_LINE('[ACTIVA] Paso 1/3: Usuario insertado');

-- PASO 2: Insertar suscripción

INSERT INTO suscripcion (

id_suscripcion, fecha_inicio, fecha_vencimiento,

estado, Usuario_id_usuario, Plan_suscripcion_id_plan

) VALUES (

100, SYSDATE, ADD_MONTHS(SYSDATE, 1), 'Activa', 100, 1

);

DBMS_OUTPUT.PUT_LINE('[ACTIVA] Paso 2/3: Suscripción creada');

-- PASO 3: Crear perfil predeterminado

INSERT INTO Perfil (id_perfil, Usuario_id_usuario, nombre, avatar, tipo)

VALUES (100, 100, 'Ana Prueba T1', 'avatar_new.png', 'Adulto');

DBMS_OUTPUT.PUT_LINE('[ACTIVA] Paso 3/3: Perfil creado');

-- PASO 4: Primer pago

INSERT INTO Pago (

id_pago, fecha_pago, monto, metodo_pago,

estado_pago, valor_descuento, suscripcion_id_suscripcion

) VALUES (

200, SYSDATE, 14900, 'PSE', 'Exitoso', 0, 100

);

DBMS_OUTPUT.PUT_LINE('[PARCIALMENTE CONFIRMADA] Todos los
pasos ejecutados');

-- — ESTADO: CONFIRMADA

COMMIT;

DBMS_OUTPUT.PUT_LINE('[CONFIRMADA] Transacción confirmada con
COMMIT');

EXCEPTION

WHEN OTHERS THEN

-- — ESTADO: FALLIDA/ABORTADA

ROLLBACK;

DBMS_OUTPUT.PUT_LINE('[ABORTADA] Error: ' || SQLERRM);

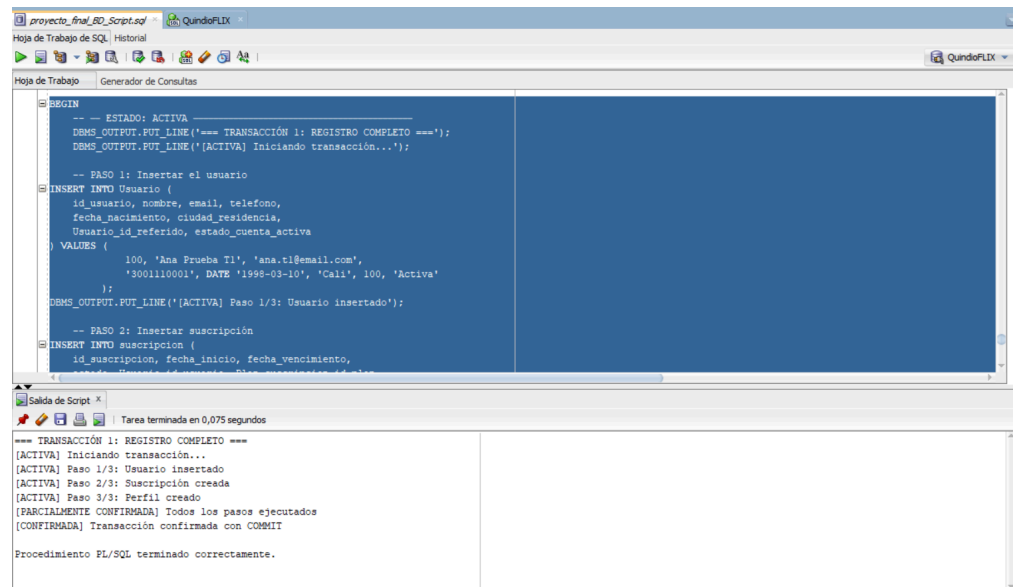
DBMS_OUTPUT.PUT_LINE('[ABORTADA] ROLLBACK ejecutado.
Ningún dato fue guardado.');

RAISE;

END;

/

Resultado:



```

-- proyecto_final_R2_Script.sql
-- ESTADO: ACTIVA
DBMS_OUTPUT.PUT_LINE('=== TRANSACCIÓN 1: REGISTRO COMPLETO ===');
DBMS_OUTPUT.PUT_LINE('[ACTIVA] Iniciando transacción...');

-- PASO 1: Insertar el usuario
INSERT INTO Usuario (
  id_usuario, nombre, email, telefono,
  fecha_nacimiento, ciudad_residencia,
  Usuario_id_referido, estado_cuenta_activa
) VALUES (
  100, 'Ana Prueba TI', 'ana.ti@email.com',
  '3001110001', DATE '1998-03-10', 'Cali', 100, 'Activa'
);
DBMS_OUTPUT.PUT_LINE('[ACTIVA] Paso 1/3: Usuario insertado');

-- PASO 2: Insertar suscripción
INSERT INTO suscripcion (
  id_suscripcion, fecha_inicio, fecha_vencimiento,
  Usuario_id_usuario, Plan_id_plan, costo
) VALUES (
  100, DATE '2023-03-10', DATE '2024-03-10',
  100, 1, 10000
);
DBMS_OUTPUT.PUT_LINE('[ACTIVA] Paso 2/3: Suscripción creada');

-- PASO 3: Crear perfil
INSERT INTO perfil (
  id_usuario, id_suscripcion, id_plan, fecha_creacion
) VALUES (
  100, 100, 1, DATE '2023-03-10'
);
DBMS_OUTPUT.PUT_LINE('[ACTIVA] Paso 3/3: Perfil creado');

[PARCIALMENTE CONFIRMADA] Todos los pasos ejecutados
[CONFIRMADA] Transacción confirmada con COMMIT

Procedimiento PL/SQL terminado correctamente.
  
```

2. Transacción 2 - Renovación mensual masiva con SAVEPOINT por usuario

Código:

DECLARE

CURSOR cur_usuarios_renovar IS

SELECT

u.id_usuario,

u.nombre,

s.id_suscripcion,

s.fecha_vencimiento,

ps.costo,

FN_CALCULAR_MONTO(u.id_usuario) AS monto_cobrar

FROM Usuario u

JOIN suscripcion s ON u.id_usuario = s.Usuario_id_usuario

JOIN Plan_suscripcion ps ON s.Plan_suscripcion_id_plan = ps.id_plan

WHERE s.estado = 'Activa'

AND s.fecha_vencimiento <= SYSDATE + 400

AND u.estado_cuenta_activa = 'Activa'

ORDER BY u.id_usuario;

```
v_sp_name    VARCHAR2(30);

v_procesados NUMBER := 0;

v_exitosos   NUMBER := 0;

v_fallidos   NUMBER := 0;

v_nuevo_pago NUMBER;

BEGIN

    DBMS_OUTPUT.PUT_LINE('=== TRANSACCIÓN 2: RENOVACIÓN  
MENSUAL CON SAVEPOINT ===');

    DBMS_OUTPUT.PUT_LINE('[ACTIVA] Iniciando proceso de renovación  
masiva...');

    FOR v_reg IN cur_usuarios_renovar LOOP

        v_procesados := v_procesados + 1;

        v_sp_name    := 'SP_USUARIO_' || v_reg.id_usuario;

        -- SAVEPOINT individual por usuario

        -- Si falla este usuario, solo se pierde su renovación

        SAVEPOINT sp_usuario_actual;

        DBMS_OUTPUT.PUT_LINE(' [SAVEPOINT ' || v_sp_name || ' ] Usuario: ' ||  
v_reg.nombre);

        BEGIN

            -- Obtener nuevo ID de pago

            SELECT NVL(MAX(id_pago), 0) + 1 INTO v_nuevo_pago FROM Pago;

            -- Registrar el pago de renovación

            INSERT INTO Pago (

                id_pago, fecha_pago, monto, metodo_pago,

                estado_pago, valor_descuento, suscripcion_id_suscripcion

            ) VALUES (

                v_nuevo_pago,

                SYSDATE,
```

```
v_reg.monto_cobrar,
'Tarjeta crédito',
'Exitoso',
v_reg.costo - v_reg.monto_cobrar, -- Descuento por antigüedad
v_reg.id_suscripcion
);

-- Renovar la suscripción
UPDATE suscripcion
SET fecha_vencimiento = ADD_MONTHS(SYSDATE, 1),
    estado = 'Activa'
WHERE id_suscripcion = v_reg.id_suscripcion;

-- Mantener cuenta activa
UPDATE Usuario
SET estado_cuenta_activa = 'Activa'
WHERE id_usuario = v_reg.id_usuario;

v_exitosos := v_exitosos + 1;

DBMS_OUTPUT.PUT_LINE(' [OK] ' || v_reg.nombre ||
    ' → $' || v_reg.monto_cobrar || ' cobrado');

EXCEPTION

    WHEN OTHERS THEN

        -- Revertir SOLO este usuario, los anteriores quedan guardados
        ROLLBACK TO sp_usuario_actual;

        v_fallidos := v_fallidos + 1;

        DBMS_OUTPUT.PUT_LINE(' [ERROR] ' || v_reg.nombre ||
            ' → Revertido: ' || SQLERRM);

END;

END LOOP;

-- — CONFIRMAR todos los usuarios que se procesaron bien
```

COMMIT;

DBMS_OUTPUT.PUT_LINE('[CONFIRMADA] Renovación completada.');

DBMS_OUTPUT.PUT_LINE(' Procesados : ' || v_procesados);

DBMS_OUTPUT.PUT_LINE(' Exitosos : ' || v_exitosos);

DBMS_OUTPUT.PUT_LINE(' Fallidos : ' || v_fallidos);

DBMS_OUTPUT.PUT_LINE(' (Los fallidos fueron revertidos con
ROLLBACK TO SAVEPOINT)');

EXCEPTION

WHEN OTHERS THEN

ROLLBACK;

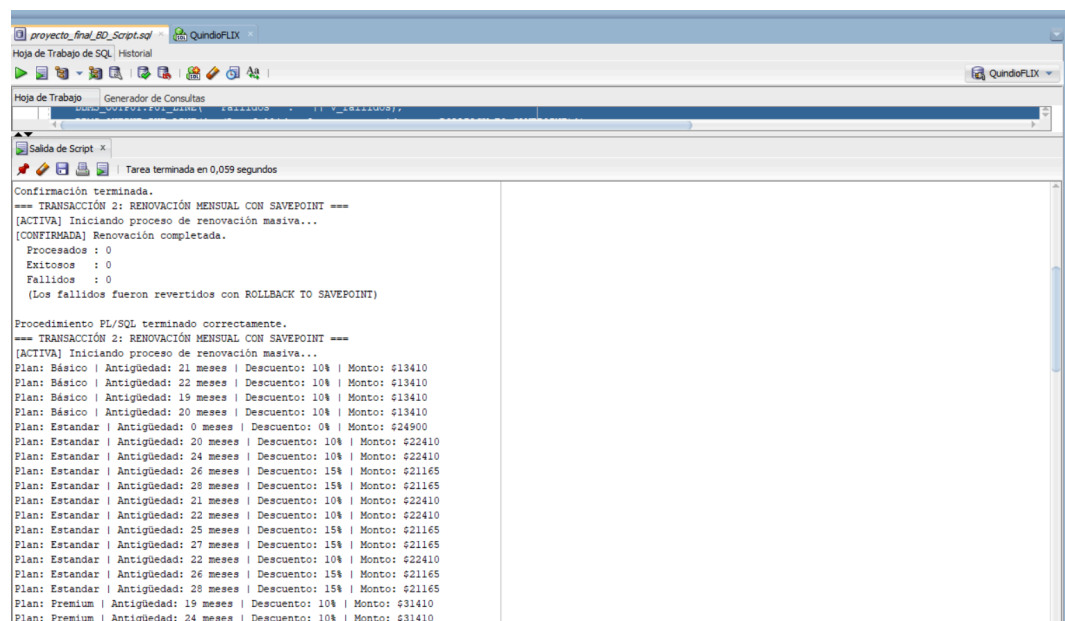
DBMS_OUTPUT.PUT_LINE('[ABORTADA] Error grave: ' || SQLERRM);

RAISE;

END;

/

Resultado:



```

Tarea terminada en 0,059 segundos

Confirmación terminada.
=== TRANSACCIÓN 2: RENOVACIÓN MENSUAL CON SAVEPOINT ===
[ACTIVA] Iniciando proceso de renovación masiva...
[CONFIRMADA] Renovación completada.
Procesados : 0
Exitosos : 0
Fallidos : 0
(Los fallidos fueron revertidos con ROLLBACK TO SAVEPOINT)

Procedimiento PL/SQL terminado correctamente.
=== TRANSACCIÓN 2: RENOVACIÓN MENSUAL CON SAVEPOINT ===
[ACTIVA] Iniciando proceso de renovación masiva...
Plan: Básico | Antigüedad: 21 meses | Descuento: 10% | Monto: $13410
Plan: Básico | Antigüedad: 22 meses | Descuento: 10% | Monto: $13410
Plan: Básico | Antigüedad: 19 meses | Descuento: 10% | Monto: $13410
Plan: Básico | Antigüedad: 20 meses | Descuento: 10% | Monto: $13410
Plan: Estandar | Antigüedad: 0 meses | Descuento: 0% | Monto: $24900
Plan: Estandar | Antigüedad: 20 meses | Descuento: 10% | Monto: $22410
Plan: Estandar | Antigüedad: 24 meses | Descuento: 10% | Monto: $22410
Plan: Estandar | Antigüedad: 26 meses | Descuento: 15% | Monto: $21165
Plan: Estandar | Antigüedad: 28 meses | Descuento: 15% | Monto: $21165
Plan: Estandar | Antigüedad: 21 meses | Descuento: 10% | Monto: $22410
Plan: Estandar | Antigüedad: 22 meses | Descuento: 10% | Monto: $22410
Plan: Estandar | Antigüedad: 25 meses | Descuento: 15% | Monto: $21165
Plan: Estandar | Antigüedad: 27 meses | Descuento: 15% | Monto: $21165
Plan: Estandar | Antigüedad: 22 meses | Descuento: 10% | Monto: $22410
Plan: Estandar | Antigüedad: 26 meses | Descuento: 15% | Monto: $21165
Plan: Estandar | Antigüedad: 28 meses | Descuento: 15% | Monto: $21165
Plan: Premium | Antigüedad: 19 meses | Descuento: 10% | Monto: $31410
Plan: Premium | Antigüedad: 24 meses | Descuento: 10% | Monto: $31410

```

3. Transacción 3 - Eliminación de cuenta con política de auditoría financiera

Código:

DECLARE

```
p_id_usuario CONSTANT NUMBER := 31; -- ID del usuario a eliminar
(prueba)

v_count    NUMBER;

BEGIN

    -- Verificar que el usuario existe

    SELECT COUNT(*) INTO v_count FROM Usuario WHERE id_usuario =
p_id_usuario;

    IF v_count = 0 THEN

        DBMS_OUTPUT.PUT_LINE('Usuario ' || p_id_usuario || ' no existe.
Transacción no necesaria.');
```

RETURN;

```
END IF;

    DBMS_OUTPUT.PUT_LINE('=== TRANSACCIÓN 3: ELIMINACIÓN DE
CUENTA ===');
```

DBMS_OUTPUT.PUT_LINE('[ACTIVA] Iniciando eliminación del usuario ' ||
p_id_usuario);

-- PASO 1: Eliminar calificaciones de todos sus perfiles

```
DELETE FROM Calificacion

WHERE Perfil_id_perfil IN (

    SELECT id_perfil FROM Perfil WHERE Usuario_id_usuario = p_id_usuario

);

    DBMS_OUTPUT.PUT_LINE('[ACTIVA] Paso 1: ' || SQL%ROWCOUNT || '
calificación(es) eliminada(s)');
```

-- PASO 2: Eliminar favoritos de todos sus perfiles

```
DELETE FROM Favorito

WHERE Perfil_id_perfil IN (

    SELECT id_perfil FROM Perfil WHERE Usuario_id_usuario = p_id_usuario

);

    DBMS_OUTPUT.PUT_LINE('[ACTIVA] Paso 2: ' || SQL%ROWCOUNT || '
favorito(s) eliminado(s)');
```

-- PASO 3: Eliminar reportes de sus perfiles

DELETE FROM Reporte

WHERE Perfil_id_perfil IN (

SELECT id_perfil FROM Perfil WHERE Usuario_id_usuario = p_id_usuario

);

DBMS_OUTPUT.PUT_LINE('[ACTIVA] Paso 3: ' || SQL%ROWCOUNT || '
reporte(s) eliminado(s)');

-- PASO 4: Eliminar reproducciones de sus perfiles

DELETE FROM Reproduccion

WHERE Perfil_id_perfil IN (

SELECT id_perfil FROM Perfil WHERE Usuario_id_usuario = p_id_usuario

);

DBMS_OUTPUT.PUT_LINE('[ACTIVA] Paso 4: ' || SQL%ROWCOUNT || '
reproducción(es) eliminada(s)');

-- PASO 5: Eliminar los perfiles

DELETE FROM Perfil WHERE Usuario_id_usuario = p_id_usuario;

DBMS_OUTPUT.PUT_LINE('[ACTIVA] Paso 5: ' || SQL%ROWCOUNT || '
perfil(es) eliminado(s)');

-- PASO 6: Marcar pagos históricos como auditados (no se eliminan por ley)

-- En lugar de DELETE, se registra en logs (aquí solo se documenta la política)

DBMS_OUTPUT.PUT_LINE('[ACTIVA] Paso 6: Pagos conservados por
política de auditoría financiera');

-- PASO 7: Marcar la suscripción como inactiva

UPDATE suscripcion

SET estado = 'Inactiva'

WHERE Usuario_id_usuario = p_id_usuario;

DBMS_OUTPUT.PUT_LINE('[ACTIVA] Paso 7: ' || SQL%ROWCOUNT || '
suscripción(es) desactivada(s)');

-- PASO 8: Marcar la cuenta como inactiva (REGLA: no se borra, solo se desactiva)

UPDATE Usuario

SET estado_cuenta_activa = 'Inactiva'

WHERE id_usuario = p_id_usuario;

DBMS_OUTPUT.PUT_LINE('[PARCIALMENTE CONFIRMADA] Paso 8:
Cuenta marcada como Inactiva');

-- — CONFIRMAR toda la transacción

COMMIT;

DBMS_OUTPUT.PUT_LINE('[CONFIRMADA] Eliminación de cuenta
completada.');

EXCEPTION

WHEN OTHERS THEN

ROLLBACK;

DBMS_OUTPUT.PUT_LINE('[ABORTADA] Error en eliminación: ' ||
SQLERRM);

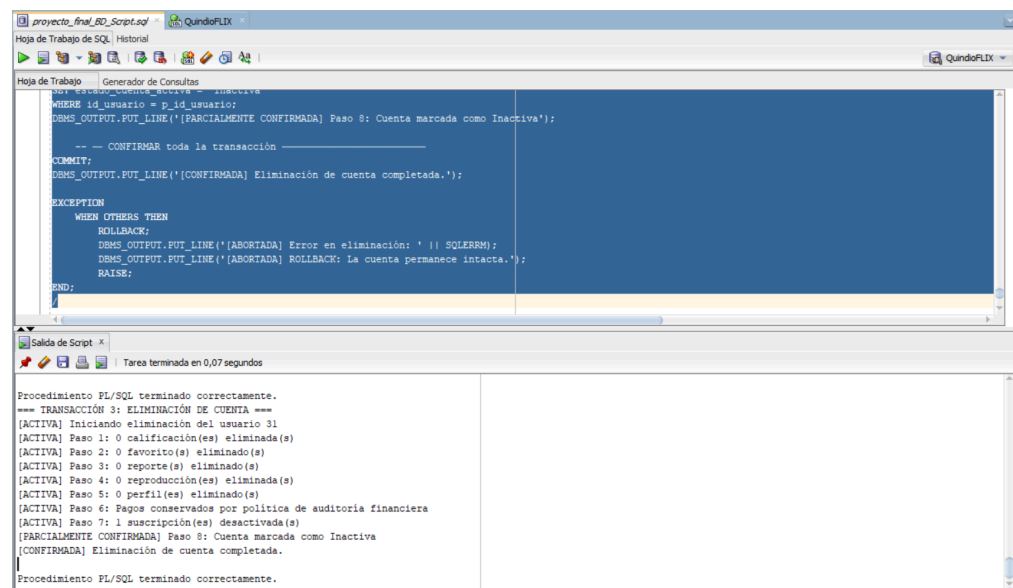
DBMS_OUTPUT.PUT_LINE('[ABORTADA] ROLLBACK: La cuenta
permanece intacta.');

RAISE;

END;

/

Resultado:



```

proyecto_final_RQ_Script.sql  QuindioPLIX
Hoja de Trabajo de SQL Historial
Hoja de Trabajo  Generador de Consultas
-- estado_cuenta_activa = inactiva
WHERE id_usuario = p_id_usuario;
DBMS_OUTPUT.PUT_LINE('PARCIALMENTE CONFIRMADA) Paso 8: Cuenta marcada como Inactiva');
-- -- CONFIRMAR toda la transacción
COMMIT;
DBMS_OUTPUT.PUT_LINE('CONFIRMADA) Eliminación de cuenta completada. ');
EXCEPTION
  WHEN OTHERS THEN
    ROLLBACK;
    DBMS_OUTPUT.PUT_LINE('ABORTADA) Error en eliminación: ' || SQLERRM);
    DBMS_OUTPUT.PUT_LINE('ABORTADA) ROLLBACK: La cuenta permanece intacta. ');
    RAISE;
END;

Salida de Script:
Tarea terminada en 0,07 segundos

Procedimiento PL/SQL terminado correctamente.
===== TRANSACCIÓN 3: ELIMINACIÓN DE CUENTA =====
[ACTIVA] Iniciando eliminación del usuario 31
[ACTIVA] Paso 1: 0 calificación(es) eliminada(s)
[ACTIVA] Paso 2: 0 favorito(s) eliminado(s)
[ACTIVA] Paso 3: 0 reporte(s) eliminado(s)
[ACTIVA] Paso 4: 0 reproducción(es) eliminada(s)
[ACTIVA] Paso 5: 0 perfil(es) eliminado(s)
[ACTIVA] Paso 6: Pagos conservados por políticas de auditoría financiera
[ACTIVA] Paso 7: 1 suscripción(es) desactivada(s)
[PARCIALMENTE CONFIRMADA] Paso 8: Cuenta marcada como Inactiva
[CONFIRMADA] Eliminación de cuenta completada.
Procedimiento PL/SQL terminado correctamente.
  
```

4. Concurrencia - Escenario de bloqueo con SELECT FOR UPDATE en dos sesiones

Código:

-- SESIÓN A (ejecutar primero):

SET SERVEROUTPUT ON;

DECLARE

v_id_sus NUMBER;

v_plan NUMBER;

BEGIN

DBMS_OUTPUT.PUT_LINE('SESIÓN A: Iniciando bloqueo usuario 5...');

SELECT id_suscripcion, Plan_suscripcion_id_plan

INTO v_id_sus, v_plan

FROM suscripcion

WHERE Usuario_id_usuario = 3

AND estado = 'Activa'

FOR UPDATE;

DBMS_OUTPUT.PUT_LINE('SESIÓN A: Bloqueo adquirido. Esperando 10 segundos...');

DBMS_LOCK.SLEEP(15);

```
UPDATE suscripcion

SET Plan_suscripcion_id_plan = 3

WHERE Usuario_id_usuario = 3 AND estado = 'Activa';

COMMIT;

DBMS_OUTPUT.PUT_LINE('SESIÓN A: COMMIT realizado. Bloqueo
liberado.');
```

END;

/

```
SET SERVEROUTPUT ON;

DECLARE

    v_id_sus NUMBER;

    v_plan NUMBER;

BEGIN

    DBMS_OUTPUT.PUT_LINE('SESIÓN B: Intentando cambiar plan del
    mismo usuario 5...');
```

```
    SELECT id_suscripcion, Plan_suscripcion_id_plan

    INTO v_id_sus, v_plan

    FROM suscripcion

    WHERE Usuario_id_usuario = 3

        AND estado = 'Activa'

    FOR UPDATE NOWAIT;

    DBMS_OUTPUT.PUT_LINE('SESIÓN B: Bloqueo obtenido.');
```

```
    UPDATE suscripcion

    SET Plan_suscripcion_id_plan = 2

    WHERE Usuario_id_usuario = 3 AND estado = 'Activa';

    COMMIT;

EXCEPTION

    WHEN OTHERS THEN
```

```
DBMS_OUTPUT.PUT_LINE('SESIÓN B ERROR: ' || SQLERRM);

DBMS_OUTPUT.PUT_LINE('Oracle bloqueó la fila. Sesión A tiene el
control.');
```

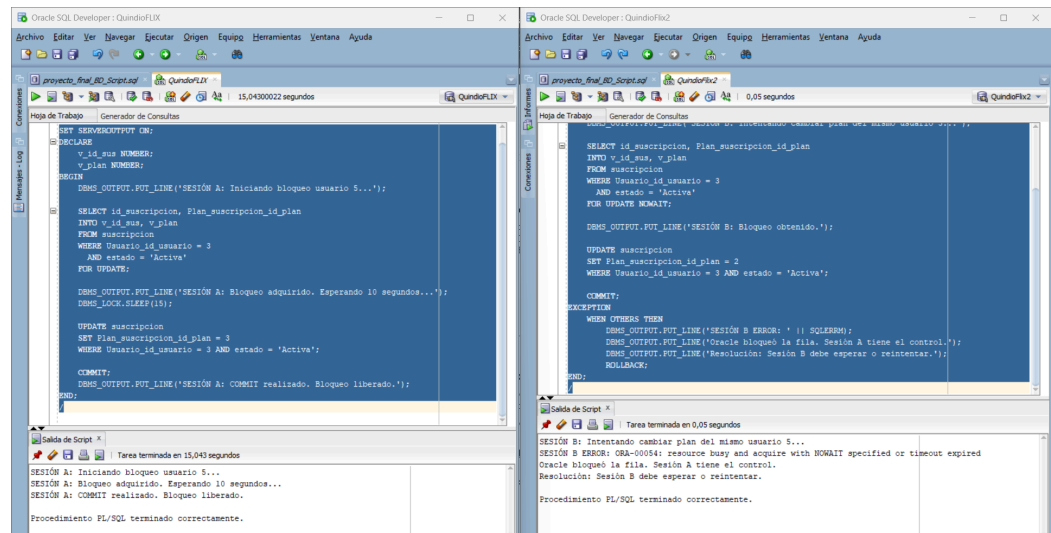
```
DBMS_OUTPUT.PUT_LINE('Resolución: Sesión B debe esperar o
reintentar.');
```

```
ROLLBACK;
```

```
END;
```

```
/
```

Resultado:



7.6 Índices

1. IDX_REPROD_PERFIL_FECHA - Índice compuesto sobre Reproduccion por perfil y fecha

Código:

-- ANTES: eliminar el índice

DROP INDEX IDX_REPROD_PERFIL_FECHA;

CREATE INDEX IDX_REPROD_PERFIL_FECHA

ON Reproduccion (Perfil_id_perfil, fecha_inicio);

--prueba

EXPLAIN PLAN FOR

SELECT * FROM Reproduccion

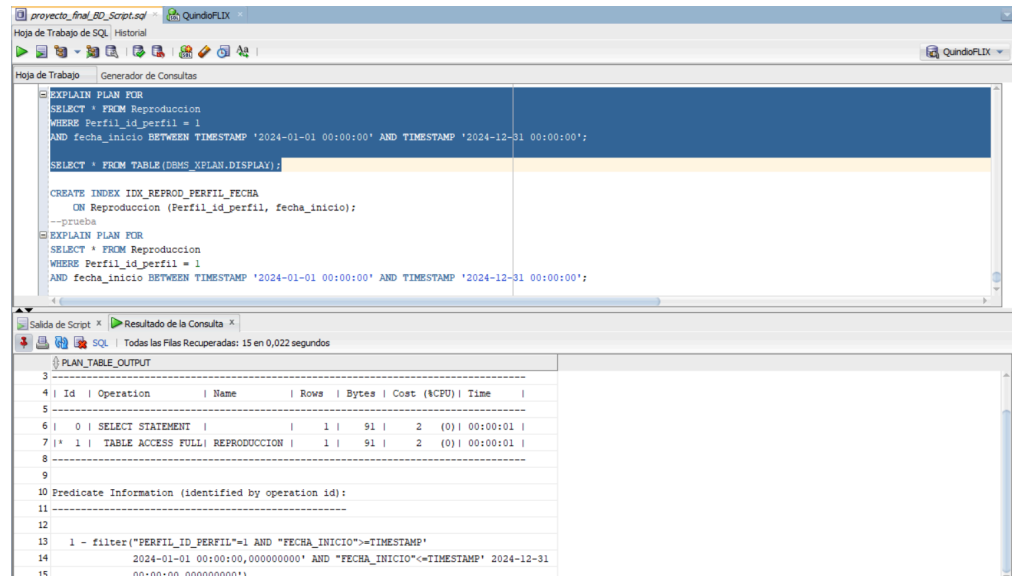
WHERE Perfil_id_perfil = 1

AND fecha_inicio BETWEEN TIMESTAMP '2024-01-01 00:00:00' AND
TIMESTAMP '2024-12-31 00:00:00';

SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);

Resultado:

Sin índice:



```

EXPLAIN PLAN FOR
SELECT * FROM Reproduccion
WHERE Perfil_id_perfil = 1
AND fecha_inicio BETWEEN TIMESTAMP '2024-01-01 00:00:00' AND TIMESTAMP '2024-12-31 00:00:00';

SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);

CREATE INDEX IDX_REPROD_PERFIL_FECHA
ON Reproduccion (Perfil_id_perfil, fecha_inicio);
--prueba
EXPLAIN PLAN FOR
SELECT * FROM Reproduccion
WHERE Perfil_id_perfil = 1
AND fecha_inicio BETWEEN TIMESTAMP '2024-01-01 00:00:00' AND TIMESTAMP '2024-12-31 00:00:00';

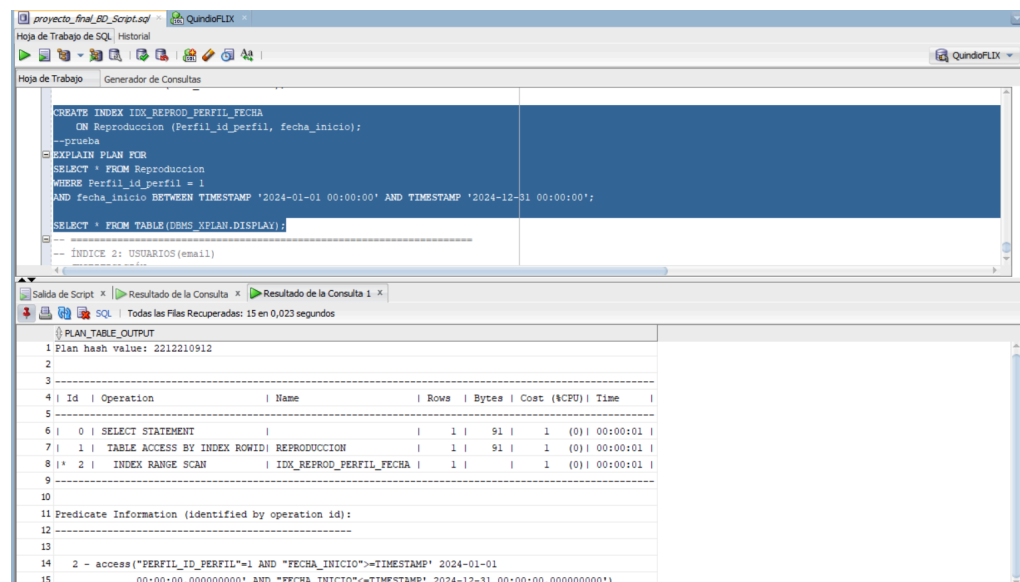
```

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		1	91	2 (0)	00:00:01
1	TABLE ACCESS FULL	REPRODUCCION	1	91	2 (0)	00:00:01

Predicate Information (identified by operation id):

1 - filter("PERFIL_ID_PERFIL"=1 AND "FECHA_INICIO">=TIMESTAMP'
2024-01-01 00:00:00,000000000' AND "FECHA_INICIO"<=TIMESTAMP' 2024-12-31
00:00:00,000000000')

Con índice:



```

CREATE INDEX IDX_REPROD_PERFIL_FECHA
ON Reproduccion (Perfil_id_perfil, fecha_inicio);
--prueba
EXPLAIN PLAN FOR
SELECT * FROM Reproduccion
WHERE Perfil_id_perfil = 1
AND fecha_inicio BETWEEN TIMESTAMP '2024-01-01 00:00:00' AND TIMESTAMP '2024-12-31 00:00:00';

SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);

--
-----
-- INDICE 2: USUARIOS(email)
-----

```

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		1	91	1 (0)	00:00:01
1	TABLE ACCESS BY INDEX ROWID	REPRODUCCION	1	91	1 (0)	00:00:01
2	INDEX RANGE SCAN	IDX_REPROD_PERFIL_FECHA	1		1 (0)	00:00:01

Predicate Information (identified by operation id):

2 - access("PERFIL_ID_PERFIL"=1 AND "FECHA_INICIO">=TIMESTAMP' 2024-01-01
00:00:00,000000000' AND "FECHA_INICIO"<=TIMESTAMP' 2024-12-31 00:00:00,000000000')

La combinación es útil porque las consultas de historial siempre filtran por las dos columnas juntas: primero identifican a qué perfil pertenece el historial y luego filtran por rango de fechas.

Sin el índice Oracle tiene que leer las 200 filas de Reproduccion completas para encontrar las del perfil 1 en enero 2024. Con el índice compuesto Oracle va directo a las filas del perfil 1 y dentro de esas busca solo las del rango de fechas, saltándose todo lo demás.

La razón de poner primero Perfil_id_perfil y luego fecha_inicio es porque Oracle usa el índice de izquierda a derecha. Primero agrupa todas las reproducciones del mismo perfil y luego dentro de ese grupo ordena por fecha, lo que hace que el rango de fechas sea eficiente. Si estuviera al revés el índice sería menos útil.

En el sistema QuindioFlix este índice beneficia directamente al SP_REPORTE_CONSUMO y a la función FN_CONTENIDO_RECOMENDADO, que son las que más frecuentemente consultan el historial de un perfil por fecha.

2. IDX_USUARIO_EMAIL - Índice único sobre email de usuario en mayúsculas

Código:

```
DROP INDEX IDX_USUARIO_EMAIL;  
  
CREATE UNIQUE INDEX IDX_USUARIO_EMAIL  
  
    ON Usuario (UPPER(email)); -- Función-based: email case-insensitive  
  
--prueba  
  
EXPLAIN PLAN FOR  
  
SELECT * FROM Usuario  
  
WHERE UPPER(email) = UPPER('andres.m@email.com');  
  
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
```

Resultado:

Sin índice:

QuindioFLIX

Hoja de Trabajo de SQL | Historial

Hoja de Trabajo | Generador de Consultas

```

--prueba
CREATE UNIQUE INDEX IDX_USUARIO_EMAIL
ON Usuario (UPPER(email)); -- Función-based: email case-insensitive

--prueba
EXPLAIN PLAN FOR
SELECT * FROM Usuario
WHERE UPPER(email) = UPPER('andres.m@email.com');

```

Salida de Script x | Resultado de la Consulta x

Todas las Filas Recuperadas: 17 en 0,014 segundos

PLAN_TABLE_OUTPUT

1 Plan hash value: 2351522803

2

3

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		1	559	4 (0)	00:00:01
1	TABLE ACCESS FULL	USUARIO	1	559	4 (0)	00:00:01

4

5

6

7

8

9

10 Predicate Information (identified by operation id):

11

12

13 1 - filter(UPPER("EMAIL")=ANDRES.M@EMAIL.COM)

14

15 Note

16

17 - dynamic sampling used for this statement (level=2)

Con índice:

QuindioFLIX

Hoja de Trabajo de SQL | Historial

Hoja de Trabajo | Generador de Consultas

```

--prueba
EXPLAIN PLAN FOR
SELECT * FROM Usuario
WHERE UPPER(email) = UPPER('andres.m@email.com');

SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);

```

Salida de Script x | Resultado de la Consulta x

Todas las Filas Recuperadas: 14 en 0,031 segundos

PLAN_TABLE_OUTPUT

1 Plan hash value: 1569299178

2

3

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		1	761	1 (0)	00:00:01
1	TABLE ACCESS BY INDEX ROWID	USUARIO	1	761	1 (0)	00:00:01
2	INDEX UNIQUE SCAN	IDX_USUARIO_EMAIL	1		0 (0)	00:00:01

4

5

6

7

8

9

10

11 Predicate Information (identified by operation id):

12

13

14 2 - access(UPPER("EMAIL")=ANDRES.M@EMAIL.COM)

El email es el campo de autenticación principal en QuindioFlix. Cada vez que un usuario inicia sesión el sistema ejecuta una búsqueda por email. Sin el índice Oracle tendría que recorrer todos los usuarios uno por uno para encontrar el que coincide, lo que en producción con millones de usuarios sería muy lento.

El índice tiene dos beneficios simultáneos. Primero acelera el login porque Oracle va directo al registro en lugar de escanear toda la tabla. Segundo garantiza que no existan dos usuarios con el mismo email porque es UNIQUE, lo que reemplaza la validación manual que hace el SP_REGISTRAR_USUARIO con el COUNT.

Se creó con UPPER(email) para que la búsqueda sea insensible a mayúsculas, es decir que "ANDRES@EMAIL.COM" y "andres@email.com" se traten como el mismo email, evitando duplicados por diferencia de

capitalización.

3. IDX_CONTENIDO_TIPO_ANIO - Índice compuesto sobre contenido por tipo y año

Código:

```
DROP INDEX IDX_CONTENIDO_TIPO_ANIO;

CREATE INDEX IDX_CONTENIDO_TIPO_ANIO
    ON contenido (tipo_contenido, anio_lanzamiento);

-- EXPLAIN PLAN sin índice

EXPLAIN PLAN FOR

SELECT * FROM contenido

WHERE tipo_contenido = 'PELICULA'

AND anio_lanzamiento = 2023;

SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
```

Resultado:

Sin índice:

The screenshot displays the QundoFLIX SQL IDE interface. The top window, titled 'proyecto_final_80_Script.sql', shows a SQL query in the 'Hoja de Trabajo' (Worksheet) tab. The query is as follows:

```
-- EXPLAIN PLAN sin indice
EXPLAIN PLAN FOR
SELECT * FROM contenido
WHERE tipo_contenido = 'PELICULA'
AND anio_lanzamiento = 2023;

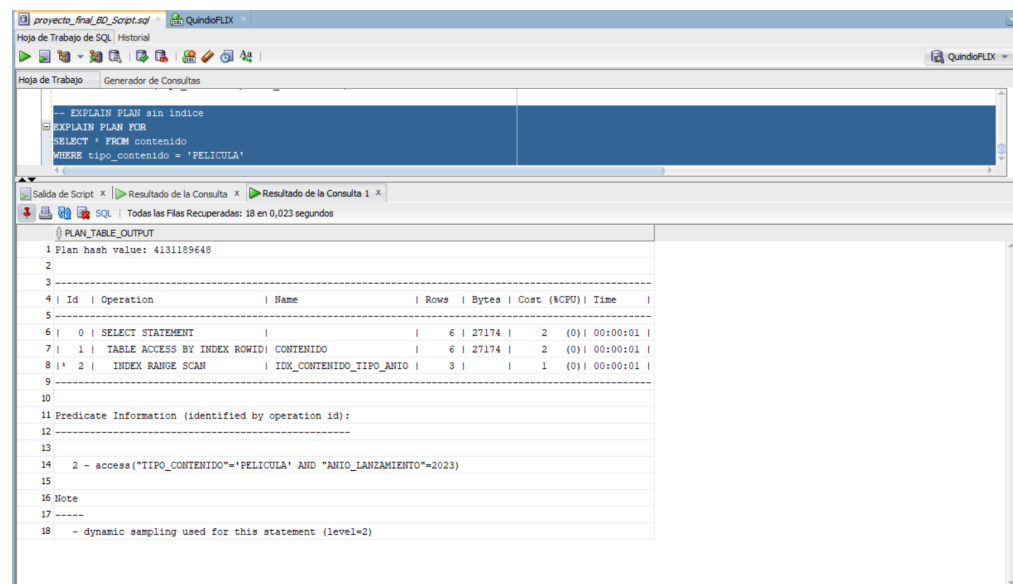
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
```

The bottom window, titled 'Resultado de la Consulta', shows the execution plan output. It includes a table with columns: Id, Operation, Name, Rows, Bytes, Cost (CPU), and Time. The output shows two operations: a SELECT STATEMENT and a TABLE ACCESS FULL operation on the CONTENIDO table.

Id	Operation	Name	Rows	Bytes	Cost (CPU)	Time
0	SELECT STATEMENT		6	27174	3 (0)	00:00:01
1	TABLE ACCESS FULL	CONTENIDO	6	27174	3 (0)	00:00:01

Below the table, the output includes 'Predicate Information (identified by operation id):' and a filter condition: '1 - filter("TIPO_CONTENIDO"='PELICULA' AND "ANIO_LANZAMIENTO">2023)'. The status bar at the bottom indicates 'Todas las Filas Recuperadas: 17 en 0,023 segundos'.

Con índice:



Hoja de Trabajo | Generador de Consultas

```
-- EXPLAIN PLAN sin indice
EXPLAIN PLAN FOR
SELECT * FROM contenido
WHERE tipo_contenido = 'PELICULA'
```

Salida de Script x | Resultado de la Consulta x | Resultado de la Consulta 1 x

Todas las Filas Recuperadas: 18 en 0,023 segundos

PLAN_TABLE_OUTPUT

1 Plan hash value: 4131189648

2

3

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		6	27174	2 (0)	00:00:01
1	TABLE ACCESS BY INDEX ROWID	CONTENIDO	6	27174	2 (0)	00:00:01
2	INDEX RANGE SCAN	IDX_CONTENIDO_TIPO_ANIO	3		1 (0)	00:00:01

11 Predicate Information (identified by operation id):

12

13

14 2 - access("TIPO_CONTENIDO"='PELICULA' AND "ANIO_LANZAMIENTO"=2023)

15

16 Note

17

18 - dynamic sampling used for this statement (level=2)

El catálogo de QuindioFlix se filtra constantemente por tipo de contenido y año de lanzamiento. Cuando un usuario busca películas del 2023 o series del 2022, el sistema ejecuta consultas con esos dos filtros combinados.

Sin el índice Oracle tiene que recorrer los 40 títulos completos para encontrar los que coinciden con el tipo y año buscado. Con el índice compuesto Oracle agrupa primero por tipo de contenido y dentro de ese grupo ordena por año, permitiendo ir directo a las películas del 2023 sin revisar documentales, series ni podcasts.

Se puso primero tipo_contenido y luego anio_lanzamiento porque en QuindioFlix siempre se filtra primero por categoría y luego se refina por año, nunca al revés. Eso hace que el índice sea óptimo para ese patrón de búsqueda específico.

En producción con miles de títulos este índice marcaría una diferencia enorme en el tiempo de carga del catálogo.

4. **IDX_PAGO_SUS_EST_FECHA** - Índice compuesto sobre Pago por suscripción, estado y fecha

Código:

```
DROP INDEX IDX_PAGO_SUS_EST_FECHA;
```

```
CREATE INDEX IDX_PAGO_SUS_EST_FECHA
```

```
ON Pago (suscripcion_id_suscripcion, estado_pago, fecha_pago DESC);
```

```
-- EXPLAIN PLAN sin índice
```

```
EXPLAIN PLAN FOR
```

```
SELECT * FROM Pago
```

WHERE suscripcion_id_suscripcion = 1

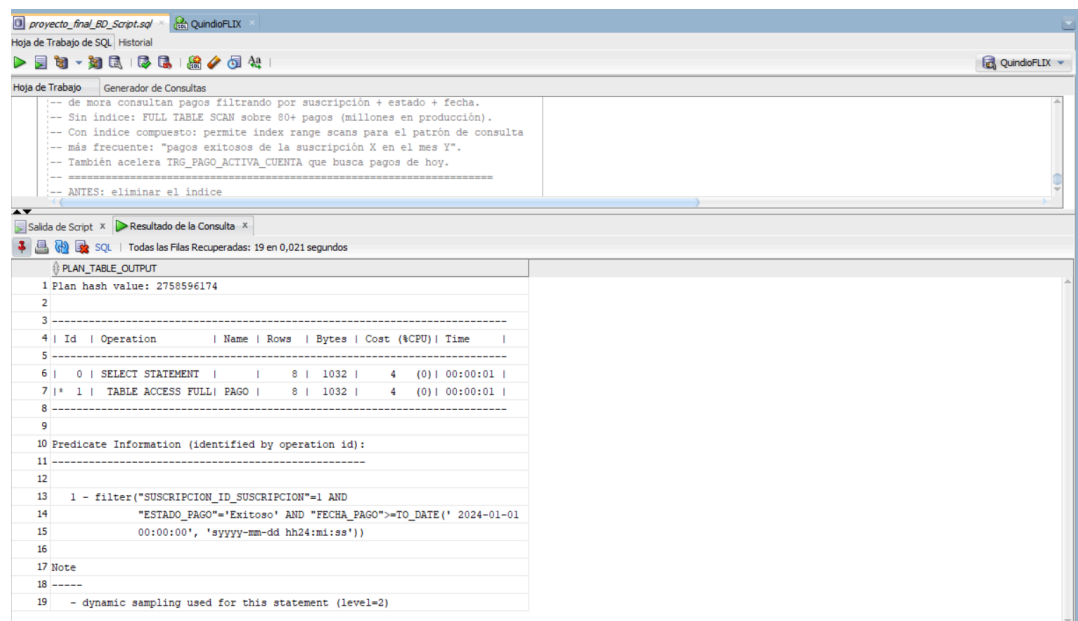
AND estado_pago = 'Exitoso'

AND fecha_pago >= DATE '2024-01-01';

SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);

Resultado:

Sin índice:



Hoja de Trabajo | Generador de Consultas

```
-- de mora consultan pagos filtrando por suscripción + estado + fecha.
-- Sin índice: FULL TABLE SCAN sobre 80+ pagos (millones en producción).
-- Con índice compuesto: permite index range scans para el patrón de consulta
-- más frecuente: "pagos exitosos de la suscripción X en el mes Y".
-- También acelera TRG_PAGO_ACTIVAR_CUENTA que busca pagos de hoy.
--=====
-- ANTES: eliminar el índice
```

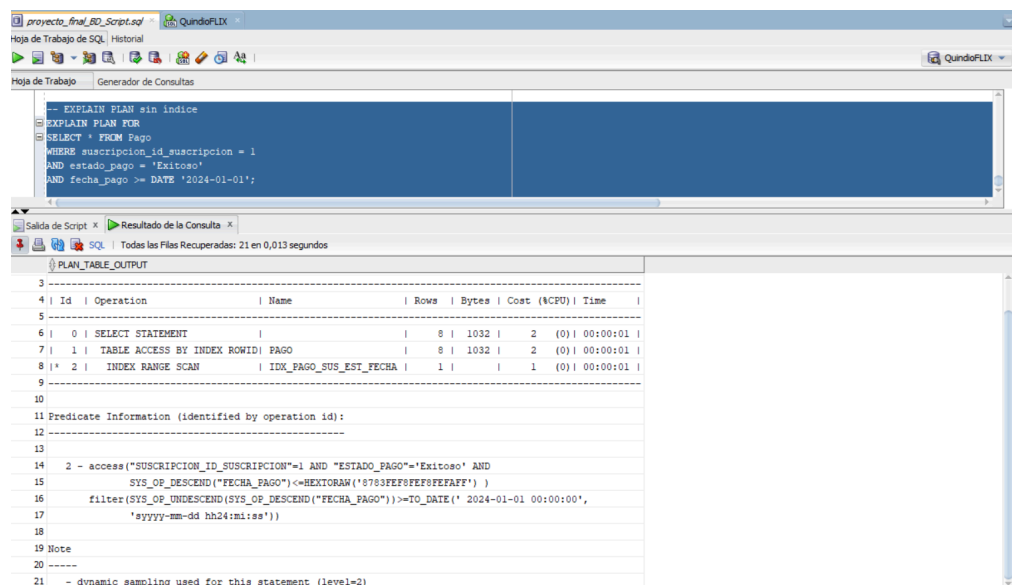
Salida de Script | Resultado de la Consulta

Todas las Filas Recuperadas: 19 en 0,021 segundos

PLAN_TABLE_OUTPUT

```
1 Plan hash value: 2759596174
2
3
4 | Id | Operation | Name | Rows | Bytes | Cost (%CPU) | Time |
5 -----
6 | 0 | SELECT STATEMENT | | | 8 | 1032 | 4 (0) | 00:00:01 |
7 |* | 1 | TABLE ACCESS FULL | PAGO | 8 | 1032 | 4 (0) | 00:00:01 |
8
9
10 Predicate Information (identified by operation id):
11 -----
12
13 1 - filter("SUSCRIPCION_ID_SUSCRIPCION"=1 AND
14 "ESTADO_PAGO"='Exitoso' AND "FECHA_PAGO">=TO_DATE(' 2024-01-01
15 00:00:00', 'yyyy-mm-dd hh24:mi:ss'))
16
17 Note
18 -----
19 - dynamic sampling used for this statement (level=2)
```

Con índice:



Hoja de Trabajo | Generador de Consultas

```
-- EXPLAIN PLAN sin índice
-- EXPLAIN PLAN FOR
-- SELECT * FROM PAGO
-- WHERE suscripcion_id_suscripcion = 1
-- AND estado_pago = 'Exitoso'
-- AND fecha_pago >= DATE '2024-01-01';
```

Salida de Script | Resultado de la Consulta

Todas las Filas Recuperadas: 21 en 0,013 segundos

PLAN_TABLE_OUTPUT

```
3
4 | Id | Operation | Name | Rows | Bytes | Cost (%CPU) | Time |
5 -----
6 | 0 | SELECT STATEMENT | | | 8 | 1032 | 2 (0) | 00:00:01 |
7 | 1 | TABLE ACCESS BY INDEX ROWID | PAGO | 8 | 1032 | 2 (0) | 00:00:01 |
8 |* | 2 | INDEX RANGE SCAN | IDX_PAGO_SUS_EST_FECHA | 1 | | 1 (0) | 00:00:01 |
9
10
11 Predicate Information (identified by operation id):
12 -----
13
14 2 - access("SUSCRIPCION_ID_SUSCRIPCION"=1 AND "ESTADO_PAGO"='Exitoso' AND
15 SYS_OP_DESCEND("FECHA_PAGO")<=HEXTORAW('8783FEF8FEF8FEFAFF'))
16 filter(SYS_OP_UNDESCEND(SYS_OP_DESCEND("FECHA_PAGO"))>=TO_DATE(' 2024-01-01 00:00:00',
17 'yyyy-mm-dd hh24:mi:ss'))
18
19 Note
20 -----
21 - dynamic sampling used for this statement (level=2)
```

Este índice fue elegido porque los pagos son la tabla de mayor consulta frecuente en QuindioFlix. Tres componentes del sistema lo usan

constantemente: el reporte financiero MV_INGRESOS_MENSUALES que agrupa pagos por suscripción y estado, el cursor de morosos que busca pagos exitosos después del vencimiento, y el trigger TRG_PAGO_ACTIVA_CUENTA que verifica pagos de la suscripción recién pagada.

La combinación de las tres columnas suscripcion_id_suscripcion, estado_pago y fecha_pago DESC responde exactamente al patrón de consulta más frecuente del sistema: "dame los pagos exitosos de la suscripción X a partir de la fecha Y". Sin el índice Oracle escanea los 80+ pagos completos. Con el índice va directo a los pagos de esa suscripción, filtra por estado exitoso y ordena por fecha descendente en un solo paso.

El DESC en fecha_pago es intencional porque siempre se quiere ver el pago más reciente primero.

8. CONCLUSIONES

Construir QuindioFlix desde cero a partir de una descripción de negocio fue el ejercicio más completo del curso, porque obligó a aplicar todos los núcleos temáticos sobre un mismo sistema con dependencias reales entre ellos. El punto de partida el modelo de datos, condicionó todo lo demás: un modelo mal diseñado habría roto las consultas, vaciado de sentido los reportes y generado conflictos entre triggers y procedimientos. Traducir el contexto empresarial a entidades y relaciones requirió una lectura analítica del enunciado que va mucho más allá de identificar sustantivos. Relaciones como la jerarquía de supervisión entre empleados, el programa de referidos entre usuarios o la asociación de contenidos relacionados no están explicadas explícitamente como relaciones: hay que deducirlas del funcionamiento del negocio, y eso fue lo que más tiempo y discusión tomó al inicio del proyecto.

En el primer núcleo, implementar PIVOT, UNPIVOT, ROLLUP, CUBE y GROUPING SETS sobre datos propios hizo que estos operadores cobraran sentido real. No es lo mismo aplicarlos sobre una tabla de ejemplo que sobre un modelo con 21 tablas relacionadas donde hay que construir los JOIN correctos antes de pensar en la agregación. El particionamiento de la tabla Reproduccion fue el primer contacto real con la administración física de Oracle, y entender por qué tiene sentido hacerlo por rango de fecha y no por otro criterio consolida la idea de que las decisiones de almacenamiento tienen un fundamento lógico detrás.

El segundo núcleo fue el más extenso y el que más dificultades presentó. Escribir procedimientos que validan reglas de negocio, manejan excepciones personalizadas y operan dentro de transacciones integra prácticamente todo el curso al mismo tiempo. Un aprendizaje concreto de este núcleo fue entender que los triggers no son redundantes respecto a los procedimientos: SP_REGISTRAR_USUARIO valida el correo antes de insertar, pero TRG_PERFIL_LIMITE_PLAN bloquea la creación de un perfil adicional sin importar desde dónde venga esa inserción, ya sea desde el procedimiento, desde SQL Developer o desde cualquier cliente externo. Esa diferencia entre validar en la capa de aplicación y restringir en la capa de datos quedó completamente clara al implementar ambos sobre el mismo modelo. Los

cursores también mostraron algo que un SELECT plano no puede producir: la capacidad de recorrer resultados fila por fila y generar reportes con estructura jerárquica, con subtotales por perfil y por categoría, tal como lo hace SP_REPORTE_CONSUMO.

En el tercer núcleo, la transacción de renovación masiva con SAVEPOINT por usuario mostró un patrón que existe en sistemas reales de facturación: si falla el cobro de un usuario, no tiene sentido revertir el trabajo ya hecho sobre los demás. El escenario de concurrencia con SELECT FOR UPDATE y NOWAIT fue la parte más distinta a todo lo trabajado hasta ese punto, porque no se trata de escribir SQL correcto sino de entender qué sucede cuando dos sesiones compiten por el mismo recurso y cómo Oracle administra esa situación automáticamente. Eso fue lo que más acercó el proyecto a condiciones de producción reales. Sobre los índices, ver el EXPLAIN PLAN antes y después de crearlos es la forma más directa de entender por qué importan: que Oracle pase de un full table scan sobre Reproduccion a un index range scan por perfil y fecha es una diferencia que en producción, con millones de registros, se traduce directamente en tiempos de respuesta. El índice funcional sobre UPPER(email) fue el ejemplo más claro de que no se trata de indexar todas las columnas, sino de entender el patrón de consulta específico que se quiere optimizar.

Finalmente, definir cuatro roles con privilegios diferenciados conectó el proyecto con una dimensión del diseño de bases de datos que normalmente queda fuera del alcance de los cursos introductorios. Que ROL_CONTENIDO pueda administrar el catálogo completo pero no vea ningún dato de usuario, o que ROL_ANALISTA tenga acceso de lectura a todo pero no pueda modificar nada, son restricciones que en un entorno real tienen implicaciones legales. En general, QuindioFlix demostró que un sistema de base de datos bien construido no solo almacena información correctamente: la protege, la organiza para acceso eficiente y la expone de forma controlada. Esa visión integral es el aprendizaje más importante que deja el proyecto.